



Technische Universität München
TUM School of Computation, Information and Technology

Learning-Based Security Analysis: Contextual Representations and Adversarial Evaluation

Mohammad Reza Norouzian

Vollständiger Abdruck der von der TUM School of Computation, Information and Technology der Technischen Universität München zur Erlangung des akademischen Grades eines

Doktors der Naturwissenschaften (Dr. rer. nat.)

genehmigten Dissertation.

Vorsitz: Prof. Dr. Claudia Eckert

Prüfende der Dissertation:

1. Prof. Dr. First Reviewer
2. Prof. Dr. Second Reviewer
3. Prof. Dr. Optional Third Reviewer

Die Dissertation wurde am DD.MM.YYYY bei der Technischen Universität München eingereicht und durch die TUM School of Computation, Information and Technology am DD.MM.YYYY angenommen.

Mohammad Reza Norouzian

Learning-Based Security Analysis: Contextual Representations and Adversarial Evaluation

Dissertation, Draft

Technische Universität München

TUM School of Computation, Information and Technology

Your Chair or Institute

Boltzmannstraße 3

85748 Garching bei München

Abstract

This dissertation examines two questions in learning-based security analysis. The first concerns representation: which observations of a program or a network a detector should receive, and how they should be structured. The second concerns evaluation, namely how a defence can be measured when an adversary may choose the test data.

In a collaborative study of IoT microservices, each node stores the service relations observed at start-up as a whitelist that the site owner can revise; detection added about 6 ms to a round trip of 24 ms. In industrial control systems, the modular framework NADICS places the learning stage behind fixed interfaces. A random forest trained on an S7-300 corpus from the IUNO project reached an attack F_1 of 94.50 per cent at a false-positive rate of 0.484 per cent under a session-disjoint split. A GAN-based detector at the same interface flagged 92.00 per cent of anomalous SWaT timestamps at a false-positive rate of 5.12 per cent on later test data.

Two Android systems form the main contribution. Hybroid combines an embedding of the function-call graph with 13 features of the network flows that an application produces. On CICAndMal2017, fusion added one or two points of detection F_1 over the better single modality, and between minus two and plus three points in malware-category classification. HGANN-Mal lifts the function-call graph to a hypergraph whose hyperedges include the backward slices of security-sensitive call sites, and it learns hyperedge membership weights by attention. Across Drebin and CICMalDroid, every hypergraph operator exceeded both pairwise graph operators in all four task and corpus combinations. HGANN-Mal led the non-attentive hypergraph operators by 1.2 to 4.1 points of accuracy in three combinations and was level in the fourth. Since construction and attention changed together, an ablation that isolates attention remains open.

Within the SPARTA project, an adversary model was translated into a contest on a hidden test set and then into a modular benchmark that combines attacks and defences by configuration. The accuracy-loss score used for ranking assigns zero loss to any constant classifier under label-preserving attacks; under PGD, the defence with zero loss had an attacked accuracy 12.9 points below that of adversarial training. The four contest results show no general relation between clean performance and robustness, so the two should be reported together.

Kurzfassung

Diese Dissertation untersucht zwei Fragen der lernbasierten Sicherheitsanalyse. Die erste betrifft die Repräsentation: Welche Beobachtungen eines Programms oder Netzwerks soll ein Detektor erhalten, und wie sollen sie strukturiert sein? Die zweite betrifft die Evaluation, also die Messung eines Abwehrverfahrens, wenn ein Angreifer die Testdaten wählen kann.

In einer kollaborativen Studie zu IoT-Microservices speichert jeder Knoten die beim Start beobachteten Dienstbeziehungen als vom Betreiber anpassbare Positivliste; die Erkennung kostete etwa 6 ms bei einer Umlaufzeit von 24 ms. In industriellen Steuerungssystemen kapselt das modulare Framework NADICS die Lernstufe hinter festen Schnittstellen. Ein Random Forest erreichte auf einem S7-300-Korpus aus dem Projekt IUNO unter sitzungsdisjunkter Aufteilung einen F_1 -Wert der Angriffsklasse von 94,50 % bei einer Falsch-Positiv-Rate von 0,484 %. Ein GAN-basierter Detektor an derselben Schnittstelle erkannte auf späteren Testdaten 92,00 % der anomalen SWaT-Zeitstempel bei einer Falsch-Positiv-Rate von 5,12 %.

Den Hauptbeitrag bilden zwei Android-Systeme. Hybroid kombiniert eine Einbettung des Funktionsaufrufgraphen mit 13 Merkmalen der Netzwerkflüsse einer Anwendung. Auf CI-CAndMal2017 erhöhte die Fusion den F_1 -Wert der Erkennung um ein bis zwei Prozentpunkte gegenüber der besseren Einzelmodalität; bei der Klassifikation nach Schadsoftwarekategorien lag der Effekt zwischen minus zwei und plus drei Punkten. HGANN-Mal überführt den Funktionsaufrufgraphen in einen Hypergraphen, dessen Hyperkanten auch Rückwärts-Slices sicherheitsrelevanter Aufrufstellen umfassen, und lernt Zugehörigkeitsgewichte mit Attention. Auf Drebin und CICMalDroid übertraf jeder Hypergraphoperator in allen vier Kombinationen aus Aufgabe und Korpus beide paarweisen Operatoren. HGANN-Mal lag in drei Kombinationen um 1,2 bis 4,1 Prozentpunkte Genauigkeit vor den Hypergraphoperatoren ohne Attention und in der vierten gleichauf. Da Konstruktion und Attention gemeinsam verändert wurden, fehlt noch eine Ablation, welche die Attention isoliert.

Im Projekt SPARTA wurde ein Angreifermodell in einen Wettbewerb mit verdecktem Testdatensatz und dann in einen modularen Benchmark überführt, der Angriffe und Abwehrverfahren per Konfiguration kombiniert. Der zur Rangbildung genutzte Genauigkeitsverlust weist jedem konstanten Klassifikator unter labelerhaltenden Angriffen den Wert null zu; unter PGD lag das verlustfreie Abwehrverfahren in der Genauigkeit unter Angriff 12,9 Prozentpunkte hinter adversarialem Training. Die vier Wettbewerbsergebnisse zeigen keinen allgemeinen Zusammenhang zwischen Genauigkeit und Robustheit, sodass beide gemeinsam berichtet werden sollten.

Acknowledgements

Replace this text with your acknowledgements.

Contents

Abstract	iii
Kurzfassung	v
Acknowledgements	vii
1 Introduction	1
1.1 Research Context	1
1.2 Problem Statement and Scope	2
1.3 Research Questions	3
1.4 Contributions	5
1.5 Publications, Projects, and Individual Contributions	6
1.6 Dissertation Structure	7
2 Background	9
2.1 Android Applications and Their Analysis	9
2.1.1 Dalvik Bytecode, Manifest, and Permissions	10
2.1.2 Static, Dynamic, and Hybrid Analysis	10
2.1.3 Obfuscation, Packing, and Repackaging	11
2.2 Graph Representations of Programs	12
2.2.1 Control-Flow and Function-Call Graphs	12
2.2.2 Hypergraphs and Higher-Order Relations	13
2.3 Learning over Graphs	14
2.3.1 Message Passing	14
2.3.2 Attention	14
2.3.3 Hypergraph Convolution	15
2.4 Detection in Networked and Cyber-Physical Systems	16
2.4.1 Network Flows as a Unit of Observation	16
2.4.2 Anomaly-Detection Method Families	16
2.4.3 Communication Structure in Industrial and IoT Networks	16
2.4.4 Datasets and Measurement Limits	17
2.5 Adversarial Machine Learning	17
2.6 Evaluation Methodology and Experimental Bias	18
2.7 Summary and Research Gaps	19
3 Anomaly Detection in IoT and Industrial Control Systems	21
3.1 Scope, Research Question, and Contributions	21
3.2 Observation Models and Datasets	21
3.3 Communication Graphs for IoT Services	22
3.4 The NADICS Framework and IUNO Integration	24

3.4.1	The S7-300 Corpus	26
3.4.2	Detection Results	30
3.4.3	Benign Operation and Alarm Burden	31
3.4.4	Extractor Throughput on Constrained Hardware	32
3.5	A GAN-Based Multivariate Detector for SWaT	33
3.5.1	Model and Training	35
3.5.2	Input Representations and Settings	37
3.5.3	Detection Results	39
3.5.4	Interpretation and Evaluation Scope	41
3.6	Limitations, Validity, and Individual Contribution	42
3.7	Chapter Summary	43
4	Multimodal Android Malware Analysis with Hybroid	45
4.1	Research Problem and Contribution	45
4.2	Related Work	46
4.3	Observation and Threat Model	48
4.4	Dataset and Experimental Protocol	48
4.5	Opcode and Method-Level Representation	49
4.6	Program-Graph Representation	51
4.7	Network-Flow Representation	53
4.8	Multimodal Fusion and Classification	55
4.9	Binary Detection Results	56
4.10	Malware-Category Classification	58
4.11	Modality Comparison	60
4.12	Limitations and Threats to Validity	60
4.13	Chapter Summary	62
5	Higher-Order Android Malware Analysis with HGANN-Mal	65
5.1	Research Problem and Relation to Hybroid	65
5.2	Related Work	66
5.3	Observation and Threat Model	68
5.4	APK Processing and Function-Call Graph Extraction	69
5.5	Security-Sensitive API Identification and Program Slicing	70
5.6	Node Features	70
5.7	Hyperedge Construction	73
5.8	Hypergraph Attention Model	76
5.9	Datasets, Tasks, and Split Policy	81
5.10	Binary Detection Results	84
5.11	Malware-Family Classification on Drebin	87
5.12	Malware-Category Classification on CICMalDroid	88
5.13	Ablation, Efficiency, and Interpretability Analysis	89
5.14	Limitations and Threats to Validity	91
5.15	Chapter Summary	93
6	Evaluating Adversarial Robustness: Benchmark Design and Metric Analysis	95
6.1	Research Problem and Contribution	95
6.2	Threat Analysis and Evaluation Scope	96
6.3	Evaluation Requirements and Threat Models	96

6.4	Tasks and Evaluation Protocol	97
6.4.1	Prediction Tasks and Permitted Perturbations	97
6.4.2	SAFAIR Contest Scoring and Evaluation Protocol	97
6.5	Data and Evaluation Measures	98
6.5.1	Corpora and Hidden Evaluation Samples	98
6.5.2	Accuracy and Task-Specific Attack Success	99
6.5.3	Aggregation over Attacks	100
6.5.4	Biometric Data and Data Protection	100
6.6	Benchmark Architecture and Reproducibility	101
6.7	Defence Mechanisms and Integration	102
6.7.1	Training and Representation Transformations	102
6.7.2	History and Activation Detectors	103
6.7.3	From an Alarm to a Classification	104
6.8	Results and Metric Analysis	105
6.8.1	Contest Evidence	105
6.8.2	SD Development Configurations	106
6.8.3	Histopathology Benchmark	107
6.8.4	Sensitivity to the Scoring Rule	108
6.8.5	Comparison with Constrained PDF-Malware Features	110
6.9	Threats to Validity	111
6.10	Individual Contribution and Project Attribution	112
6.11	Chapter Summary	112
7	Cross-Contribution Discussion	115
7.1	Communication Context and Program Context	115
7.2	Multimodal and Higher-Order Representations	115
7.3	Observation Requirements and Data-Acquisition Cost	116
7.4	Predictive Evaluation and Adversarial Evaluation	117
7.5	Dataset Limitations and Extraction Failures	117
7.6	Scope of Generalisation	118
7.7	Implications for Android Malware Analysis	119
8	Conclusion and Future Work	121
8.1	Answers to the Research Questions	121
8.2	Summary of Contributions	122
8.3	Future Work	122
	References	123
A	Feature Importance of the NADICS Extraction Fields	137

List of Figures

2.1	Android package contents and analysis modes	11
2.2	Pairwise projection of a hypergraph	13
3.1	The two measurements reported for the service-graph design. Panel (a) gives the inter-service round-trip time with and without detection for ten to forty services. Panel (b) gives the vertex and edge counts of one node's graph over a single run, with the learning phases and the interval of anomalous behaviour shaded. Reproduced from [32], to whose measurement design the candidate contributed.	23
3.2	Architecture of the NADICS framework. Malicious traffic is generated in a real industrial environment, dissected into protocol-characteristic features, and classified by the anomaly detection engine. The stage structure follows the one reported in the IUNO deliverable [33].	25
3.3	The IUNO test environment at the Volkswagen plant in Wolfsburg. Four servers, three network segments and a Siemens plant network reproduce the production setting in which NADICS ran as the TUM detection tool on the virtualisation host. The network TAPs fitted at the relevant interfaces are omitted from the drawing [41].	27
3.4	Random-forest importance of the 63 extraction fields. (a) The fifteen most important fields, coloured by group. (b) Each group's share of the total importance against its share of the field count. (c) Cumulative share over the ranked fields.	29
3.5	SWaT process and control architecture. Network levels follow the SWaT convention. The field-network symbols indicate sensors and actuators within each stage; the detector consumes their recorded values. Adapted from the supplied diagram and the dataset description [123].	34
3.6	SWaT attack categories and their counts in the dataset paper [123]. Stage and point locations are schematic examples. The arrangement describes attack scope and carries no ranking of detection difficulty.	34
3.7	SWaT collection campaign and the partition used in the GAN study. The campaign follows the dataset paper [123]. After the initial normal-operation exclusion, the first 56,240 attack-period timestamps support training and validation; the later 393,679 timestamps form the test set.	35

3.8	Multivariate GAN detection pipeline. (a) The four networks are trained together on earlier windows containing normal and attack observations; validation data select the representation, the settings and the threshold. (b) The later 393,679 test timestamps are scored with the trained networks; the latent critic has no part in the score a_t of Equation 3.1. The blue tag marks the input representation, the only factor varied between runs, and orange tags mark the steps that used attack ground truth. The window length is fixed at 60 samples. The scaler and PCA are fitted on training data and reused for validation and testing. The candidate implemented every processing step shown.	36
3.9	Observed signals (blue) and their reconstructions (orange) for two dimensions, reproduced at the original embedded resolution from Figure 4.15 of the internal report. The repeated red trace is the aggregate reconstruction discrepancy across dimensions, shared by the two displayed signals. It precedes the combination of Equation 3.1.	37
4.1	The Hybroid architecture. Every application sample provides a pair of inputs: the application package and a capture of the traffic it generates (1). Both are analysed in parallel (2). Androguard decompiles the package into opcodes, basic blocks and a control-flow graph, which are embedded into a 64-dimensional code representation, while Argus turns the capture into bidirectional flows that are aggregated, normalised and reduced to 13 network-flow features. The two representations are concatenated into one joint vector per sample (3), on which a gradient-boosting classifier is trained and cross-validated (4) for binary detection and for categorisation into four malware categories (5). Redrawn by the candidate from the architecture published in [35].	46
4.2	Opcode frequency across the pre-training corpus. The upper panel names the twenty most common operations, and the lower panel plots all 216 in rank order, where the shape of the decay is the property the embedding relies on. Reproduced from the candidate's own publication [35], as is Figure 4.5.	50
4.3	The program-code branch. The static analyzer extracts the control-flow graph, opcodes and basic blocks from each APK. Opcodes are embedded first, every method is represented from the tagged control-flow graph of the basic blocks it contains, and a graph embedding reduces the resulting function-call graph to a single 64-dimensional feature vector. Redrawn by the candidate from the arrangement published in [35].	51
4.4	The network branch. Argus reduces a raw capture to bidirectional flow events, the events belonging to one application are aggregated into a single record, every attribute is normalised to a common range, and feature selection retains the 13 most informative features. Redrawn by the candidate from the arrangement published in [35].	53
4.5	Kendall correlation among the retained flow features, used to check that the selected features are not redundant with one another.	54
4.6	Receiver operating characteristic curves for binary malware detection, reproduced from the published vector content of Figure 7 in [35]. The source labels the curves as macro averages and reports the AUC values in the legend. It provides neither a selected operating threshold nor fold-level variation.	57

4.7	Binary detection by learner and observation modality, with each reported metric in its own panel. Every row carries one marker per modality, joined by a rule spanning the values it reports. The values are the five-fold means printed to two decimal places in Figure 8 of [35], checked against the bar geometry of that figure. The score axis is truncated to 0.86–0.98, the interval those values occupy. No fold-level scores are published, so the figure carries no error bars. Redrawn by the candidate.	58
4.8	Five-class malware-category classification by learner and observation modality, drawn on the same plan as Figure 4.7. The score axis is truncated to 0.78–0.96, because this task occupies a lower interval. The values are the five-fold means printed to two decimal places in Figure 9 of [35], checked against the bar geometry of that figure. Decision-tree recall is 0.86 for program code and for the fused representation, so those two markers are drawn concentrically. No fold-level scores are published, so the figure carries no error bars. Redrawn by the candidate.	58
4.9	Per-class receiver operating characteristic curves for gradient boosting on the fused representation, reproduced from the published vector content of Figure 10 in [35]. The legend reports a macro-average AUC of 0.976 and the AUC of each class. The paper provides no selected operating threshold or fold-level variation.	59
5.1	Construction of the node feature vector in Stage 3	72
5.2	Hyperedge generation in Stage 4	74
5.3	Class distribution of the two corpora	82
5.4	F1 of the five methods on both corpora and both tasks	85
5.5	Accuracy against the majority baseline and the strongest baselines	86
6.1	Benchmark architecture, redrawn after D7.6 Figure 2, p. 22 [39]. Box labels follow the original. The five modules separate task definitions and data loading from attack execution.	101
6.2	Autoencoder placement in the partner classifier, redrawn after D7.2 Figures 13 and 11, p. 14 [171]. In the top variant (a), the autoencoder r_{η_x} transforms the input before feature extraction. In the middle variant (b), r_{η_z} acts between the feature extractor ϕ_ω and the prediction head c_ψ , as in Equation 6.8. The legend follows D7.2, which keeps the original weights of VGG16 and the dense network in both variants. The diagrams specify placement, not a matched ablation of all training choices.	103
6.3	Partner detection mechanisms, redrawn after D7.2 Figures 15 and 16, p. 16 [171]. Panel (a) uses preceding queries, whereas panel (b) uses internal activations of the classifier. The recorded parameters, the activation features of Equation 6.9 and the per-class detector follow D7.2. The response is the label reversal of Equation 6.10, which D7.6 applies in the benchmark [39]. The final action is part of the defended system and must be included when task accuracy is measured.	104
6.4	SD development configurations from D7.6 Sections 2.8.1.2–2.8.1.3, pp. 18–19 [39]. The interval 84–85% is a reported approximate range, not a confidence interval. The source describes attribute outputs despite its re-identification heading. These values are not additional official contest rows.	107

6.5	Accuracy losses computed from D7.6 Table 2 [39]. Each cell is $100(A_{\text{clean}} - A_{\text{attack}})$ in percentage points; the original accuracies remain in Table 6.4. The separation of attack rows does not imply matched budgets. In particular, zero loss for the activation detector describes its aggregate score, not a demonstrated mechanism or robustness certificate.	109
6.6	Original iterative-FGSM transfer curves from the partner PDF-malware study, D7.5 Figure 15, p. 23 [38]. Both panels retain the published legends and values. The horizontal axis is the source's $\log_{10}(\delta)$ perturbation budget, distinct from the accuracy loss Δ used in this chapter. These are feature-space results, with no uncertainty bands or demonstrated construction of modified PDFs with preserved malicious functionality.	110

List of Tables

1.1	Previously disseminated parts of this dissertation.	6
1.2	Research questions, chapters, and the material each answer rests on.	8
3.1	Observation units used in Chapter 3. The rows are not numerically comparable because their labels, sampling processes, and validation protocols differ.	22
3.2	Composition of the S7-300 corpus and its partitions. Whole capture sessions were assigned to the partitions, four to training and one to each of the other two, and every attack episode lies entirely within one partition. The episode column counts executed attacks, not records.	28
3.3	Duration of the 25 attack episodes in minutes and seconds. Each attack type has three training episodes, one validation episode and one test episode.	28
3.4	Six-class confusion matrix on the 19,841-record test session, with rows giving the true class. Predicted classes are abbreviated: N normal, AB authentication bypass, CS CPU stop and start, RC reconnaissance, DS denial of service, BF brute force. The last three columns give precision, recall and F_1 for the class of that row.	31
3.5	Extraction workloads on the Raspberry Pi 3 prototype. Input frames are the frames presented to the extraction pipeline, and processed frames exclude those dropped before processing. Rates and CPU utilisation are averages over the replay, and CPU utilisation refers to the capacity of the whole device. The emission delay runs from the close of a window to the emission of its record.	32
3.6	Measured timestamp-level performance of the input representations. Each timestamp is counted once and no point adjustment is applied.	39
3.7	Timestamp-level confusion matrix for the final 24-component PCA configuration. Counts refer to individual timestamps after aggregation of overlapping window outputs.	40
4.1	CICAndMal2017 source-corpus composition. The category counts are recomputed from the dataset family listing and differ from the published values of 124, 112, 109 and 101, which do not sum to the stated total of 426.	49
4.2	The 13 network-flow features retained after selection, grouped by the kind of quantity each one measures.	54
4.3	Share of flows by protocol, per malware category. The two columns do not sum to 100 per cent and the source publication does not say what the remainder is.	55
4.4	Detection compared with four reimplemented systems, read from the published figures. The four systems differ considerably in weight: DREBIN is a full conference paper, Adagio and CIC 2017 are papers of ten and seven pages, and the SVM entry is a two-page poster. The reported performance values come from Figures 6 and 7 of [35].	56

4.5	Area under the ROC curve per class, gradient boosting on the combined representation, with the pre-exclusion counts of Table 4.1. Values are read from Figure 10 of [35].	59
4.6	Difference between the fused score and the better of the two single-modality scores. Positive values favour fusion. The differences are computed from Figures 8 and 9 of [35], whose values are printed to two decimal places.	60
5.1	The four groups of node feature. The width of the first group is the size of the opcode vocabulary.	71
5.2	Binary detection, in per cent, as published. The source labels the first column Macro-F1 and states no averaging convention. Section 5.14 shows that the HGANN-Mal values on Drebin are positive-class values for the malicious class, and that several rows cannot follow one convention. Best value in each column in bold.	84
5.3	Multiclass classification, in per cent, as published. The Drebin task assigns a malicious application to one of twenty families; the CICMalDroid task assigns an application to one of five categories, of which Benign is one. The two tasks are not comparable with each other. Section 5.14 identifies two rows whose three columns cannot follow one averaging convention.	88
6.1	Planned and reported SAFAIR evaluation procedures.	98
6.2	Evaluation units and data provenance. Stated sample sizes describe the printed procedures, not recovered per-example result files.	99
6.3	Reported contest scores, including the worst-perturbed column, from D7.6 Table 1, p. 16 [39]. Task names follow that table. The SD task label conflicts with its detailed attribute-model description; best and worst values are not identified as paired outcomes for one checkpoint under a common attack suite.	105
6.4	Published accuracies for the five partner defence configurations, from D7.6 Table 2, pp. 28–29 [39]. Attack hyperparameters and the evaluated subset are unspecified. Each entry is a reported aggregate, with no dispersion estimate. . .	107
7.1	Evaluation conditions and reported binary class shares	118
A.1	Importance of the 63 fields of the NADICS extraction schema in the random forest of Section 3.4.2, in rank order. Importance is the normalised share of the total, and the cumulative column sums the importances down to that rank. The letter of the identifier gives the group of Table A.2, and the bar colours follow Figure 3.4.	137
A.2	Importance by field group. Both shares are percentages of the 63-field total. Importance per field is the group share divided by the number of fields in the group, and the last column gives the rank of the group’s highest field in Table A.1.139	

Introduction

1.1 Research Context

Detection systems built on machine learning are ordinary practice in security engineering. Drebin classified Android applications from statically extracted feature sets in 2014 [1], MaMaDroid modelled sequences of abstracted API calls as Markov chains soon afterwards [2], and a systematic review published in 2023 identified 132 studies applying deep learning to Android malware defence between 2014 and 2021 [3]. The same shift took place in network defence, where anomaly detection moved from statistical baselining [4] towards deep architectures trained directly on traffic [5], and in program analysis, where graph neural networks are now applied to code structure [6, 7, 8]. Android alone reports more than three billion active devices [9], and the number of applications submitted for analysis each year is far larger than any team of analysts can inspect by hand. That imbalance is what makes automated classification attractive.

Industrial and Internet of Things (IoT) networks reached the same point by a different route. Their traffic is comparatively regular, since the devices are few and their interactions are prescribed by the process they control, which is what makes learned models of normal behaviour plausible there at all. Testbeds built for the purpose supply the labelled attack data that operational plants cannot [10]. The difficulty is that a model of normal behaviour has to be kept current as the deployment changes, and in a service-oriented architecture it changes often.

The methods were developed for settings that differ from security in two ways that matter. A classifier trained on one sample of data and deployed on another assumes that both come from the same distribution. Malware violates the assumption by construction: families rise and fall, developers repackage applications specifically to defeat the detectors of the previous year, and the label an antivirus engine assigns to a file may change as engines are updated. Jordaney et al. measured this drift directly and showed that a classifier's confidence gives little warning of it [11]. The second difference is that the data are chosen by someone who benefits when classification fails.

How much reported performance survives a stricter protocol has been measured. Published F1 scores for Android malware classification reach 0.99, which leaves little room for improvement and invites the question of what the number means. Pendlebury et al. traced the figure to two forms of experimental bias: training and test data whose composition does not match a real deployment, and time splits that let a classifier learn from samples that appeared after the ones it is tested on [12]. Arp et al. then examined thirty papers from the four leading security conferences against a list of ten methodological pitfalls. Every paper exhibited at least three of

them [13]. Artefacts in the datasets add to the effect. Drebin, the most widely used Android corpus, carries duplicate samples produced by repackaging [14], although the systematic study of what those duplicates cost found the effect limited for supervised classification and considerably larger for unsupervised family clustering [15]. None of these findings concerns a single weak paper. They concern the standard protocol.

A second body of evidence concerns what happens once the adversary is treated as active. Small, deliberately chosen perturbations flip the output of trained models [16], and the observation predates deep learning: Biggio et al. demonstrated gradient-based evasion against detectors at test time in 2013 [17], with a line of work on adversarial classification running back further still [18]. The defensive response has not held up well. Athalye et al. re-examined the nine non-certified defences accepted at ICLR 2018 and found that seven owed their apparent robustness to obfuscated gradients, of which six were then circumvented completely [19]. Tramèr et al. broke a further thirteen defences published at leading machine-learning venues, all of which had already been evaluated against adaptive attacks by their own authors [20]. Carlini et al. responded by writing down what an adequate robustness evaluation requires [21]. In the malware setting the constraint is tighter than in image classification, because an adversarial example must be a file that still installs and still runs [22].

Two questions follow from this, and they organise the present work. The first is a question about representation: what should be observed about a program or a network, and how should those observations be structured, so that a model is given the evidence that separates malicious from benign behaviour? The second is a question about evaluation: how should a defence be measured when the party generating the test data is trying to defeat it? The two are usually pursued by different communities. This dissertation treats them together, because a representation that is never tested against an adaptive opponent and a benchmark that never varies the representation are weak in complementary ways.

1.2 Problem Statement and Scope

The representation problem

An analysis system sees only what its observation model admits. Static analysis reads the application package and reasons about code that may never execute, which gives broad coverage at the cost of accuracy against obfuscation and dynamic loading; the limits are well understood and were formalised early [23]. Dynamic analysis observes what a sample actually does, using instrumentation such as taint tracking [24], which resists obfuscation but reaches only the paths that the harness triggers [25]. Neither observation model dominates the other, so systems that combine them should be able to describe behaviour that neither describes alone. What such a combination buys, measured on the same samples under one protocol, is the first open question this dissertation addresses.

Structure raises a second problem. Program behaviour is commonly encoded as a graph, with functions or basic blocks as nodes and calls or transfers as edges, and graph neural networks then learn over that structure. An edge relates exactly two elements. Behaviour that

is security-relevant frequently involves more: a permission request, a sequence of API calls, and a network write may be innocuous separately and malicious in combination. Encoding such a group as a set of pairwise edges loses the fact that the members belong to one group. Hypergraphs remove that restriction by allowing an edge to join any number of nodes [26], and hypergraph neural networks provide the learning machinery [27, 28]. Whether the higher-order encoding pays for itself on Android program structure, and whether weighting hyperedges by attention adds anything beyond the encoding itself, is the second open question. The question is worth asking sceptically, since attention mechanisms in graph learning have been shown to be less expressive than their formulation suggests [29].

The evaluation problem

Robustness claims are only as good as the attacks used to test them, and the record shows that attacks chosen by the defender's own authors are too weak [19, 21]. An evaluation that is credible has to begin from an explicit statement of what the adversary knows and what the adversary may change, and it has to be reproducible by a third party. Building such an evaluation as a piece of infrastructure, rather than as a section of a paper, is the third open question treated here.

Scope

The empirical work concerns Android. Applications are analysed from the package as distributed, using Dalvik bytecode and, where the design calls for it, traffic recorded while the application runs. Native code, kernel-level instrumentation, and iOS lie outside the scope. The industrial and IoT material concerns communication behaviour observed at the network level and, in the SWaT study, sensor and actuator values of the physical process. It was recorded in testbeds and project deployments, not at operator scale.

Three limits on the claims should be stated here rather than discovered later. First, the work on communication graphs for IoT services was disseminated as a two-page contribution to PAM 2018 on which the candidate is the fourth of four authors. The full paper that reports anomaly detection on that system was written by two of his co-authors without him [30], as was the companion short paper on graph-based access control [31]. Chapter 3 identifies both and scopes the claim accordingly. Second, the graph work on IoT services did not lead to the Android work. The two are compared as representations in Chapter 7, and no developmental sequence between them is claimed. Third, the adversarial material is a contribution to evaluation methodology and to benchmark construction. No new attack algorithm and no certified defence is proposed.

1.3 Research Questions

Four questions organise the dissertation. The first concerns the representation of behaviour in networked and industrial systems and is divided into two subquestions. The second and third concern the representation of Android program behaviour. The fourth concerns evaluation.

Each question takes up one of the four gaps stated in Section 2.7, and Chapter 8 answers each of them in the same words in which they are asked here.

RQ1: Behaviour in networked and industrial systems

How can behaviour in networked and industrial systems be represented to support anomaly detection?

RQ1 takes up gap G1. Its two subquestions organise three empirical strands over service relations, network records, and industrial process data. The strands are complementary rather than cumulative, and no experiment compares all of them.

RQ1.1: Communication graphs

How can service relationships be represented as communication graphs to identify unexpected interactions?

Chapter 3 answers RQ1.1 through a relation model for service-to-service communication, a procedure for keeping the graph current as services change, and measurements of the overhead this imposes. Detection accuracy against a labelled attack set is not reported, and the chapter states why.

RQ1.2: Modular industrial anomaly detection

How can a modular learning framework analyse network and process features for anomaly detection in industrial control systems?

Chapter 3 answers RQ1.2 through NADICS, a modular framework for network and process features with interchangeable preprocessing and learning components. The chapter reports its IUNO integration and a multivariate reconstruction detector on SWaT. Here, reconstruction means that the model reproduces its input readings and uses the discrepancies to help detect anomalies.

RQ2: Multimodal Android malware analysis

To what extent does fusing static code-graph representations with dynamic network-flow features improve Android malware detection and category classification compared with either modality alone?

RQ2 takes up gap G2. Chapter 4 answers it through Hybroid, which extracts features from program structure and from recorded traffic and classifies their combination. The chapter reports binary detection and malware-category classification, and it compares each modality against the fusion under one protocol.

RQ3: Higher-order Android malware analysis

To what extent do attention-weighted hypergraph representations of static Android program structure improve binary malware detection and malware classification over pairwise graph models and non-attentive hypergraph models?

RQ3 takes up gap G3. Chapter 5 answers it through HGANN-Mal, which represents an application as a hypergraph over its call structure and applies attention across hyperedges. Two classification tasks are distinguished throughout and are not interchangeable: malware-family classification on Drebin, and malware-category classification on CICMalDroid, in which benign applications form one of the categories. Binary detection is a third, separate task.

RQ4: Adversarial robustness evaluation

How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness?

RQ4 takes up gap G4. Chapter 6 answers it through a threat analysis of machine-learning systems, a public contest run on a hidden test set, and a modular benchmark in which attacks and defences are combined by configuration. It examines how task definitions, scoring and attack validity affect the interpretation of the reported results, and identifies the experimental records needed for reproduction.

1.4 Contributions

A communication-graph model for IoT service interaction. Service-to-service relations are represented as a graph that is built from observed traffic and updated as the deployment changes, together with a measurement of the cost of maintaining it. The contribution answers RQ1.1 and is reported in Chapter 3. Its scope is limited by the evidence available: the disseminated artefact runs to two pages, and the chapter states what the candidate contributed to it.

A modular framework for anomaly detection in industrial networks. NADICS decomposes packet parsing, feature construction, and classification into interchangeable components, so that learning methods can be exchanged without rebuilding the pipeline, and it fed the visual security control room built by the IUNO project. The later work adds a GAN-based detector for multivariate SWaT process data. The contribution answers RQ1.2 and is reported in Chapter 3.

Multimodal Android malware analysis. Hybroid combines a static representation of program structure with features derived from observed network flows, and evaluates the combination against each modality alone on the same samples. The contribution answers RQ2 and is reported in Chapter 4.

Higher-order Android malware analysis. HGANN-Mal encodes groups of related program elements as hyperedges rather than as sets of pairwise edges, and weights those hyperedges by attention. The contribution answers RQ3 and is reported in Chapter 5, which compares the result against pairwise graph models and against non-attentive hypergraph models. Because the encoding and the weighting were changed together, the chapter does not attribute the gain to attention alone.

Threat-guided evaluation of adversarial defences. An adversary model for machine-learning systems is translated into an evaluation protocol, realised first as a contest with a hidden test set and then as a modular benchmark in which attacks and defences are composed by configuration. The chapter derives the behaviour of accuracy-loss scoring under constant predictions and distinguishes gradient masking from prediction collapse. It analyses the response policy of binary adversarial detectors and compares the published clean and attacked accuracies, with unresolved experimental mechanisms identified as such. The contribution answers RQ4 and is reported in Chapter 6. Where the underlying project deliverables were written collectively, the chapter states what the candidate designed and implemented and what he synthesised from the work of partners.

A comparison across the contributions. Chapter 7 relates the representations to one another along the axes of observability, data-acquisition cost, and validity of evaluation. The comparison is conceptual. Numerical results from the individual studies are not pooled, because their datasets and validation protocols differ.

1.5 Publications, Projects, and Individual Contributions

Parts of this dissertation have been disseminated previously, in accordance with the doctoral regulations of the Technical University of Munich. Table 1.1 lists every such item, with the candidate’s position in the author list and the chapter that draws on it. Where a chapter reuses previously published material, it rewrites it rather than reprinting it, and the corresponding results are restated from the original experimental records.

Table 1.1. Previously disseminated parts of this dissertation.

Item	Venue or type	Year	Position	Chapter
Graph-based Anomaly Detection for IoT Microservices [32]	PAM, two pages	2018	4 of 4	3
IUNO work package 4, visual security control room [33]	Project deliverable	2018	contributor	3
IUNO work package 7, security tools [34]	Project deliverable	2018	contributor	3
Hybroid [35]	ISC, Springer LNCS	2021	1 of 4	4
HGANN-Mal [36]	ISC, Springer LNCS	2025	1 of 2	5
SPARTA D7.1, AI systems threat analysis [37]	Public deliverable	2021	contributor	6
SPARTA D7.5, AI systems security mechanisms [38]	Public deliverable	2021	contributor	6
SPARTA D7.6, Validation and evaluation report [39]	Public deliverable	2022	editor	6
SAFAIR AI Contest development toolkit [40]	Software repository	2021	sole author	6

Individual contribution

The two-page PAM 2018 contribution on graph-based anomaly detection for IoT microservices [32] arose from a collaboration in which the candidate is the last of four authors. The peer-reviewed full paper on anomaly detection in that system was published by two of those

co-authors without him [30], and so was the short paper on graph-based access control in the same architecture [31]. Chapter 3 therefore presents the communication-graph material with its authorship stated explicitly, and it limits the originality claim to the parts the candidate designed and implemented.

Two final deliverables of the IUNO project [33, 34] report work the candidate carried out at the chair. Both are consortium documents whose author lists sit at the level of the document, so neither assigns any individual passage to one contributor. Within the project the candidate designed and implemented NADICS, together with the extractor that derives connection features from S7 and PROFINET traffic. He also produced the labelled attack corpus the deliverables describe, recorded between Siemens S7-300 controllers and their control server and labelled for binary detection as well as for five classes of attack. An earlier internal deliverable of June 2017 [41] documents the first version of that tool and names him among its authors. That document was not published, which is why it is absent from Table 1.1. Chapter 3 states which components are his, and it records the passages in which the deliverable text credits the implementation with functionality it did not yet have.

The candidate also designed, implemented, evaluated, and documented an unpublished extension reported in Chapter 3. It combines a multivariate GAN architecture with reconstruction and critic scoring for SWaT process data. He carried out all of its experiments and wrote the internal research report, which was not disseminated and is therefore absent from Table 1.1.

For Hybrid [35] the candidate is the first of four authors, and the paper records that the first two share first authorship. He designed the pipeline as a whole and the network-traffic analysis within it, together with the framework through which the dataset was analysed, from preprocessing through learning to the reported results. He carried out the experiments and wrote the paper, and he contributed in part to the treatment of the graph data. HGANN-Mal [36] is the work of the candidate with his supervisor. The candidate designed the method, implemented it, carried out the experiments and wrote the paper.

Within the SPARTA project, the candidate contributed to the threat analysis of machine-learning systems reported in [37] and to the contest concept in [38], and he is the editor of [39], in which the benchmark and the contest results are reported. The first two deliverables were written by a consortium and attribute contributors at the level of the document rather than the chapter. Chapter 6 is therefore written afresh, cites each taxonomy it draws on in place, and confines its contribution claim to the evaluation design and to the benchmark implementation, which are the candidate's own.

1.6 Dissertation Structure

Chapter 2 sets out the foundations shared by the later chapters: Android applications and their analysis, graph and hypergraph representations of programs, learning over graphs, detection in networked and cyber-physical systems, adversarial machine learning, and evaluation methodology. It closes with a statement of the gaps that the four research questions address. The work against which each contribution is positioned is treated in the chapter that reports it. Chapter 3 reports communication-graph, network-flow, and industrial process-anomaly

studies and answers RQ1. Chapter 4 presents Hybroid and answers RQ2. Chapter 5 presents HGANN-Mal and answers RQ3. These two chapters carry the main Android empirical weight of the dissertation. Chapter 6 presents the threat-guided evaluation work and answers RQ4. Chapter 7 sets the contributions against one another, and Chapter 8 answers each research question in turn and states what remains open.

Every contribution chapter carries its own account of the work specific to its method, its own evaluation, and its own statement of limitations. There is no shared methodology chapter, because the studies have no common experimental method to share.

Table 1.2 maps each research question to the chapter that answers it, to the material that supports the answer, and to the candidate's role in producing that material. The reader who wants to know what any particular claim rests on should start there.

Table 1.2. Research questions, chapters, and the material each answer rests on.

Question	Chapter	Supporting material	Role
RQ1.1	3	PAM 2018, 2 pp. [32], project artefacts	contributor
RQ1.2	3	NADICS, internal experimental records, IUNO deliverables [33, 34]	design, implementation, evaluation
RQ2	4	ISC 2021 [35]	first author
RQ3	5	ISC 2025 [36]	first author
RQ4	6	SPARTA D7.1, D7.5, D7.6 [37, 38, 39]	contributor, editor

Background

This chapter sets out the material the later chapters share: the artefacts an Android application exposes to analysis, the graph and hypergraph representations used to describe programs, the learning operators defined over those representations, detection in networked and cyber-physical systems, adversarial machine learning, and the methodology by which any of it is assessed. Positioning against competing systems is not attempted here. Every contribution chapter carries its own account of the work specific to its method, and that is where published systems are compared.

Learning-based security analysis has a common shape. Samples are collected and labelled, a representation is derived from each sample, a model is fitted, its output is thresholded, and the result is measured on data held back from training. A methodological defect can enter at any of these stages, and the defects that recur have been catalogued stage by stage [13]. Four assumptions carried over from other application domains do not hold in this setting. Training and test data are taken to come from one stationary distribution, which the evolution of malware denies. The available sample is taken to represent the deployment, which its composition and its timeline usually deny. Errors are treated as symmetric and the classes as roughly balanced, whereas operational traffic is neither [42, 43]. The data are assumed not to be chosen by an opponent. Section 2.6 takes up the first three and Section 2.5 the fourth.

The Android sections open the chapter, because Android malware analysis is the centre of the dissertation and because a hyperedge is easier to read once function-call graphs have been defined. One consequence of that ordering should be signalled here. Section 2.4 supports Chapter 3 although it appears after the Android material, so a reader following chapter order meets the foundations later than the chapter that draws on them. Section 3.1 repeats the pointer.

2.1 Android Applications and Their Analysis

Chapters 4 and 5 both take an Android application package as input and derive features from it. This section fixes what that package contains, what each mode of analysis is able to observe in it, and the two mechanisms that most often prevent observation altogether. Network-flow representation is deferred to Section 2.4, and the graph objects the two chapters build are defined in Section 2.2.

2.1.1 Dalvik Bytecode, Manifest, and Permissions

An Android application is distributed as an application package, a ZIP archive whose security-relevant members are a manifest and one or more files of compiled bytecode [3]. Source written in Java or Kotlin is compiled to bytecode for the Dalvik virtual machine [1] and stored in `classes.dex`, with further numbered files such as `classes2.dex` for larger applications. The container is specified by the platform [44], as is the instruction set [45]. Since Android 5 that bytecode is compiled ahead of time and executed by the Android Runtime rather than interpreted by Dalvik [46], although the instructions an analyst reads are the same in either case. Section 4.5 depends on this distinction, because the operand values attached to an instruction vary with the runtime while the opcode does not.

The manifest declares the components an application provides, the permissions it requests, the intents its components filter, and the hardware features it expects. It is recoverable without any reasoning about code, which is what makes it the cheapest security-relevant metadata available: four of Drebin’s eight feature sets are taken from the manifest and the remaining four from the disassembled bytecode [1].

Access to sensitive functionality is mediated by permissions, each carrying a protection level, of which only the level marked *dangerous* guards capability able to cause real harm [47, 48]. Two properties of this system bear on the later chapters. The mapping from a permission to the API calls it guards is documented incompletely and has to be recovered by analysing the platform: PScout extracted it from roughly 3.4 million lines of Android source across four releases and found more than 75 permissions whose guarded API the documentation did not describe [49]. The specification is also unstable, since fine-grained least privilege trades against stability across releases [49], and deprecation on the platform side has been measured as one source of the drift reported for permission-based detection [50]. Section 5.5 relies on both properties, because it selects permissions by protection level and maps flagged API calls back to their call sites.

2.1.2 Static, Dynamic, and Hybrid Analysis

An application may be analysed without running it, by running it, or by combining the two. Figure 2.1 sets the three modes against the artefacts of the previous subsection. The static mode dominates practice: across the studies of deep learning for Android malware defence reviewed by Liu et al., 73 per cent worked statically, 17 per cent dynamically, and 10 per cent combined both [3].

Static analysis reads the package and reasons about code that may never execute, which yields coverage of everything shipped at modest computational cost. The engineering is mature. Disassembly and call-graph construction are supplied by standard toolkits [51, 52], and taint analysis that models Android’s component lifecycle is available [53]. What no amount of engineering removes is a ceiling on what static reasoning can decide. An adversary is able to construct obfuscation whose analysis reduces to an NP-hard problem, which defeats semantics-aware static detection [23]. That construction was demonstrated on x86 binaries,

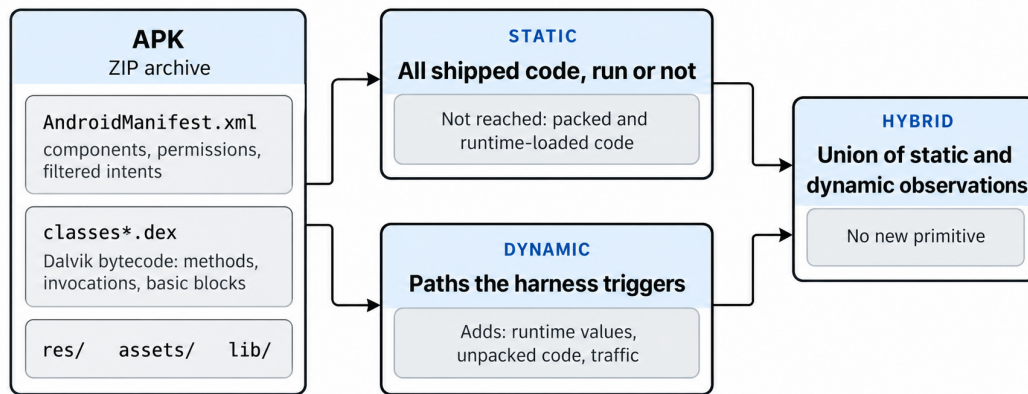


Figure 2.1. What an application package holds and what each mode of analysis reaches. Static observation covers all shipped code and not what the code does; dynamic observation covers what the code did, but only along the paths the harness triggered, and it alone reaches network traffic, which is the channel Chapter 4 uses. The graph objects built from the bytecode are named in Section 2.2.

and the argument carries to Dalvik by the same reasoning rather than having been shown there directly.

Dynamic analysis observes the application while it runs. Instrumentation ranges from taint tracking on the device [24] to whole-system emulation that reconstructs the operating-system and Dalvik views together [54]. Traffic recorded during execution is a further channel, and it is the one Chapter 4 uses. Two limits apply throughout. Only the paths the harness triggers are observed, and an application may detect the analysis environment and withhold the behaviour under investigation [55, 56].

Neither mode dominates the other. Measured under one protocol, static analysis proved at least as effective as dynamic analysis, depending on how the applications were stimulated during execution, while the combination of the two measured best [25]. Hybrid pipelines are consequently an established design [57], and the question Chapter 4 takes up is what such a combination buys on one set of samples under a single protocol.

2.1.3 Obfuscation, Packing, and Repackaging

Obfuscation is ordinary developer practice as much as attacker practice. Of 1.7 million free applications on Google Play, 24.92 per cent had been obfuscated by their developers [58], and a measurement across 114,560 applications drawn from Google Play, third-party markets and malware corpora found string encryption more frequent in malware and packing more frequent outside Google Play [59]. The effect on detection is measurable. Three thousand benign and three thousand malicious applications, expanded into 73,362 obfuscated variants by 29 strategies, left most commercial anti-malware products severely affected, and trivial transformations sufficed [60]. A later study of twelve detectors found that none held its ideal-setting performance once obfuscation, concept drift and adversarial examples were introduced [61].

Packing is a distinct mechanism and is kept separate here. A packed application ships an encrypted original together with a wrapper that decrypts and loads it at runtime, so static extraction fails outright instead of degrading. Commercial packers are used by malware authors, and custom packers are written specifically to defeat the available unpackers [62].

The packaging model also allows an attacker to graft a malicious rider onto a benign carrier [63]. Applications produced this way are near-identical outside the injected payload, and corpora assembled from them therefore contain duplicates. What those duplicates cost depends on the task: the measured effect is limited for supervised classification and substantially larger for unsupervised family clustering [15]. Section 2.6 returns to this as a problem of evaluation rather than of extraction.

2.2 Graph Representations of Programs

Representations are defined here, before the operators that learn over them. Two families are needed. Chapters 4 and 5 both derive a graph from program structure, and Chapter 5 then lifts that graph to a hypergraph. The notation introduced in this section is the notation those chapters use, and neither redefines it.

2.2.1 Control-Flow and Function-Call Graphs

A *control-flow graph* is a directed graph over the basic blocks of a single procedure. Its vertices are maximal instruction sequences that contain no branch except possibly at the end, and a directed edge records that control may pass from one block to the next. The object is intra-procedural by definition [8]. A *call graph* takes the procedures of a program as its vertices, with an edge from f to g recording that a call site in f may invoke g . It is inter-procedural, and a statically recovered call graph over-approximates, since the analysis admits edges that no execution takes. For Android the call graph is taken at method granularity over the recovered bytecode, and this dissertation calls that object a *function-call graph*: vertices are methods, and directed edges are method invocations [64, 36]. The two objects are kept distinct throughout, because their vertex sets and their edge semantics differ.

The two compose. Embeddings computed on per-method control-flow graphs may be attached to the vertices of a function-call graph, which makes intra-procedural and inter-procedural structure available together [8]. Dropping the call edges from that composition leaves a third object: a set of independently embedded control-flow graphs, pooled to application level, with no invocation structure at all. The three are named here so that each contribution chapter can state which of them it builds without introducing new machinery, and so that a claim made about one is not read as a claim about another.

Whichever object is chosen, it is an artefact of the analyser rather than a property of the program. Recovery depends on the toolkit and on how the component lifecycle is modelled [52, 53, 51], and abstracting call targets to package or family level shrinks the vertex set at the cost of the callee's identity [2]. Vertex features may equally be learned rather than hand-built, from the block structure of a procedure [65, 66] or from the body of a method [67].

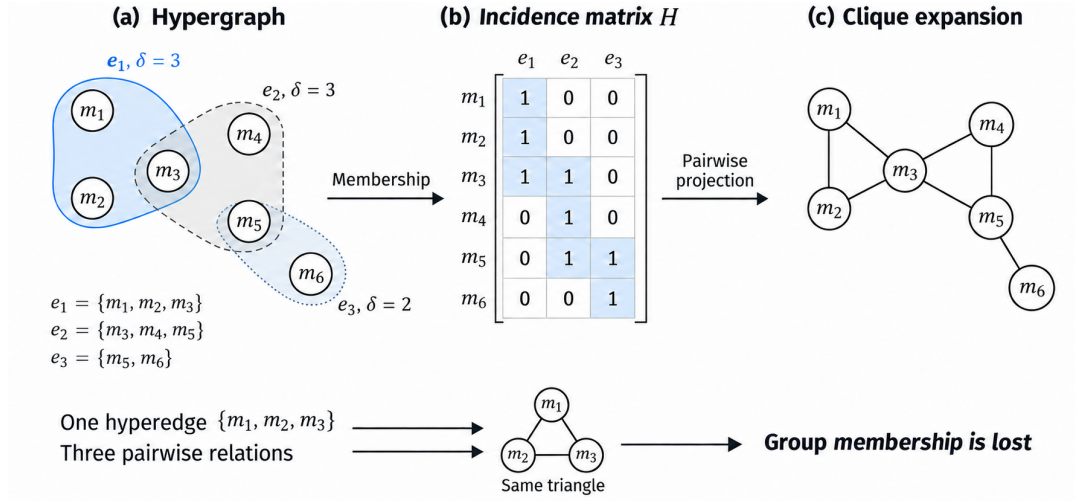


Figure 2.2. The same six methods represented as (a) a hypergraph, (b) its incidence matrix H , and (c) its unweighted clique expansion. A matrix entry is one precisely when the method belongs to the corresponding hyperedge. Hyperedge e_3 joins two vertices and is therefore an ordinary graph edge. The lower example shows that the triangle over m_1, m_2, m_3 arises both from the single hyperedge e_1 and from three separate pairwise relations. The projection preserves pairwise adjacency but does not uniquely determine the original hyperedge memberships [26].

2.2.2 Hypergraphs and Higher-Order Relations

A hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E}, w)$ has a finite vertex set $\mathcal{V}$ and a family $\mathcal{E}$ of subsets of $\mathcal{V}$, each subset a hyperedge e carrying a weight $w(e) > 0$. Membership is recorded by the incidence matrix $H \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{E}|}$, where $h(v, e) = 1$ exactly when $v \in e$. The degree of a vertex is $d(v) = \sum_{e \in \mathcal{E}} w(e) h(v, e)$ and the degree of a hyperedge is $\delta(e) = \sum_{v \in \mathcal{V}} h(v, e) = |e|$. Writing D_v and D_e for the diagonal matrices of vertex and hyperedge degrees and W for the diagonal matrix of hyperedge weights completes the apparatus [26]. A graph is the special case in which $\delta(e) = 2$ for every hyperedge, so every graph is a hypergraph and the converse fails.

An edge relates exactly two vertices, and encoding a group of several as a set of pairwise edges loses the fact that its members belonged to one group. The projected graph cannot distinguish a single hyperedge on three vertices from three separate pairwise relations among the same vertices [26]. Two projections are standard. *Clique expansion* replaces each hyperedge by the complete graph on its members, and *star expansion* introduces a new vertex per hyperedge joined to each member [68]. Figure 2.2 shows the hyperedge memberships and the information lost in their pairwise projection.

The sceptical reading of these definitions deserves stating, because it is the stronger position to argue from. Agarwal et al. show that the hypergraph learning formulations of their period reduce to the Laplacian of one expansion or the other, that the two coincide for uniform hypergraphs, and that for non-uniform ones they differ only in how hyperedges of unequal size are weighted; their conclusion is that graphs, not hypergraphs, lie at the heart of the problem [68]. Chitra and Raphael sharpen the boundary. A random walk on a hypergraph whose vertex weights do not vary between the hyperedges a vertex belongs to, the *edge-independent* case, is equivalent to a walk on a weighted clique expansion, so an operator built on such

a walk carries no higher-order information. Only *edge-dependent* vertex weights escape the equivalence [69].

What the projection costs has since been measured rather than asserted. An evaluation framework that expands, learns and then reconstructs reports depressed performance at vertex, hyperedge and hypergraph level across eight datasets [70]; two recurring hyperedge patterns have been identified as the source of the loss in every projection, together with a combinatorial impossibility result on recovering the discarded structure [71]; and an expressivity hierarchy names the invariants a clique expansion cannot represent [72]. Section 2.3 describes the operators built on this apparatus, and Chapter 5 develops the one it uses.

2.3 Learning over Graphs

The operators described here learn representations over a structure whose construction was fixed in Section 2.2. Three families are relevant: neighbourhood aggregation on pairwise graphs, attention-weighted aggregation, and convolution over an incidence structure. This section states what each does and what is known about its limits. The notation stays as defined above, and the operator Chapter 5 builds on is developed there rather than here.

2.3.1 Message Passing

Most graph neural networks share one scheme. Each vertex holds a state, that state is updated from the states of its neighbours by a permutation-invariant aggregation, and the update is applied for a fixed number of rounds [73]. The convolutional instance fixes the aggregation coefficients in advance from the degrees of the two endpoints, so a vertex combines its neighbours in proportions that the graph determines and training does not alter [6].

Two limits on this family are established. The first is a bound on what it can tell apart: no network in the family distinguishes two graphs that one-dimensional Weisfeiler-Leman colour refinement assigns the same colours, and an architecture reaching that bound is as discriminative as the scheme permits [74]. The second concerns depth. Repeated propagation drives the representations within a connected component towards one another, so adding rounds to widen a vertex's receptive field costs discriminability between vertices [75]. Both bear on program graphs, where the structures compared are often large and locally similar.

2.3.2 Attention

Attention replaces fixed coefficients with learned ones. A scoring function is applied to each vertex and its neighbour, the scores across a neighbourhood are normalised, and the resulting weights determine the combination [7]; the mechanism was introduced for sequences before it was carried to graphs [76]. The motivation is that some neighbours matter more than others, and that which ones matter should be learned rather than read off the degree.

Three results qualify that motivation, and they should be kept apart. Brody et al. show that the ranking the standard formulation induces over a vertex's neighbours is the same for every query vertex, so the mechanism selects globally important neighbours rather than neighbours important to a particular vertex; a revised scoring function removes the restriction [29]. Fang et al. bound both scoring functions, the linear form used by the original mechanism and the multi-layer form introduced by its revision, showing that each realises only finitely many neighbour rankings at bounded width and depth [77]. The same work reports that on node-property benchmarks over pairwise graphs, evaluated with GCN, GraphSAGE and GIN as non-attentive baselines, no unmodified attentive model outperformed all of those baselines on any dataset or task [77]. The defensible reading of that last result is negative about the evidence rather than about the mechanism: an advantage for attention over fixed aggregation has not been demonstrated on those benchmarks, and the expressive power of the standard scoring functions is bounded.

Whether either finding transfers to operators defined over hyperedges is not established in the literature, since both analyses concern pairwise operators. Section 5.8 shows that the static-ranking argument carries over to the additive hyperedge score used there. For other scoring functions over hyperedges, and for the finite-ranking bound, the question is open, and Section 2.3.3 returns to it.

A methodological point follows, and it concerns how such architectures are designed rather than how they perform. Where a model changes its relational encoding and its aggregation weighting at the same time, the contribution of each change cannot be recovered from the reported totals. The two have to be varied separately for either to be attributed.

2.3.3 Hypergraph Convolution

Operators over a hypergraph propagate in two stages. Signal moves from the vertices to the hyperedges that contain them, is combined at each hyperedge, and returns to the member vertices, with the incidence structure and the degree matrices of Section 2.2.2 supplying the normalisation [27, 28]. Two design routes exist. One reduces the hypergraph to a graph and applies a pairwise operator to the result [78], which inherits the losses recorded in Section 2.2.2; the other treats a hyperedge as a multiset and learns the functions that aggregate it [79]. Attention has been carried across to this setting by making the incidence structure itself learnable, so that membership is weighted by a real value instead of a binary indicator [80].

Three observations bound what is known. Standard hypergraph convolution averages over the members of a hyperedge and therefore attenuates variation between them, a low-pass bias whose discarded high-frequency component has been characterised [81]. The operator most widely used has since been superseded by a stability-oriented successor from its own authors [82], which is the reference point for any claim about its limitations. No theoretical analysis of attention scoring in hypergraph operators has appeared since 2024; the nearest treatments predate that, in work on equivariant hypergraph networks and on generalising set transformers to hypergraphs [83, 84]. The transfer question raised in Section 2.3.2 therefore remains open on the theoretical side as well as the empirical one.

2.4 Detection in Networked and Cyber-Physical Systems

This section covers the unit of observation used in network detection, the families of method defined over it, the particular character of industrial and IoT deployments, and the corpora on which results are reported. Chapter 3 draws on it, although the Android material precedes it for the reason given at the head of this chapter.

2.4.1 Network Flows as a Unit of Observation

Detection at the flow level summarises the packets sharing a five-tuple into counts, durations and byte totals. Working at that level lowers processing cost and survives payload encryption, at the price of discarding content [85]. Network intrusion detection is conventionally divided into signature matching against descriptions of known attacks and anomaly detection against a learned model of normal behaviour [86], and the second is what the rest of this section concerns. Section 4.7 states the particular flow feature set Chapter 4 uses, rather than drawing it from here.

2.4.2 Anomaly-Detection Method Families

Anomalies are conventionally classed as point, contextual or collective, and detectors differ in whether labels are available for both classes, for one, or for neither [4]. Classical methods score a point by the local density around it [87] or by how few partitions are needed to isolate it [88]. Deep detectors replace a hand-built feature space with a learned one, and the shallow and deep families admit a common formulation [89, 5]. Where the data are relational, the anomaly is a property of structure rather than of any single record, and the object flagged may be a vertex, an edge or a subgraph [90]. That observation is what connects this section to Section 2.2.

The security setting resists the standard formulation. The cost of a false alarm is high, network traffic is more variable than a model of normal behaviour admits, and data adequate for evaluation are scarce [42].

2.4.3 Communication Structure in Industrial and IoT Networks

Industrial and IoT traffic is comparatively regular, because the devices are few and their interactions are prescribed by the process under control. A whitelist of permitted communication relations is therefore a workable model of normal behaviour. Barbosa et al. built such whitelists in SCADA networks from four packet properties and reported both that the whitelists stabilise and that rare legitimate flows continue to raise false positives [91]; Yoon and Ciocarlie modelled communication patterns per host pair for the same purpose [92].

Neither of the two ideas that follow from this originates with service-oriented deployments. Scoring behaviour against a graph of which host talks to which was established for carrier traffic [93, 94], and the dependency graph between services was already being recovered automatically from passively observed traffic in enterprise networks [95, 96, 97].

Where the system is cyber-physical, the observation may be the process rather than the traffic, and detectors are then fitted to sensor and actuator series recorded from testbeds [10, 98]. In IoT deployments the unit modelled is often the individual device. Models of that kind hold under the conditions they were fitted to and lose a substantial part of their measured performance when the deployment changes: reproduced figures fall by roughly a quarter to a half beyond three months, and by a comparable margin between test sites [99].

2.4.4 Datasets and Measurement Limits

The corpora this field reports on are known to be defective. KDD 99 was shown to contain redundant records that bias measured accuracy [100], and a later audit of CICIDS2017, UNSW-NB15 and comparable sets catalogues construction faults that allow trivial features to separate attack traffic from benign [101]. Two further limits bear on any figure reported here. Detectors reporting detection rates above 99 per cent have been shown to be trained and tested on the same attack type, with performance not surviving attack types absent from training [102]. And when 70 published approaches were re-run on one evaluation platform, four deliberately simple tests on process values, using minima, maxima and rate of change, performed on par with them [103]. Section 2.6 treats the protocol-level causes.

2.5 Adversarial Machine Learning

Chapter 1 has already set out what the record shows about published defences. This section supplies the vocabulary that account assumed, and it is the vocabulary Chapters 4, 5 and 6 use to state what they claim and what they do not.

Threat models. A threat model states four things: the attacker's goal, the attacker's knowledge of the system, the attacker's capability to act on it, and the constraints on what may be modified [104, 105]. A claim about an attack or about robustness cannot be interpreted without all four [21]. Knowledge is conventionally divided into three levels. Under *white box* the attacker knows the training data, the feature representation, the learning algorithm, the trained parameters and any deployed defence [18]. Under *black box* the attacker knows neither parameters nor architecture and interacts through the prediction interface, either by transferring from a substitute model trained on its outputs or by searching directly with queries [106]. *Grey box* lies between the two: some components of the white-box list are known and training cannot be influenced. Because the term is used inconsistently in the literature, every use in this dissertation names which components the attacker is assumed to know. One limit on the taxonomy should be recorded with it. A survey of 271 practitioners found that the knowledge assumptions of academic threat models are more generous to the attacker than deployment conditions usually allow [107].

Classes of attack. *Evasion* modifies an input at test time so that a trained model misclassifies it, leaving the model unchanged [17]. *Poisoning* manipulates the training data or the training process so that the resulting model is degraded or carries behaviour the attacker chose [108, 109], and the *backdoor* is its sub-case, in which the implanted behaviour is conditioned on a trigger [110]. Confidentiality attacks form a third class: a functional copy of a deployed model can be recovered through its prediction interface [111]. The word adversarial covers all three and is not a synonym for evasion.

Feature space and problem space. A *feature-space* attack perturbs the feature vector directly, with no requirement that any real input maps to the result. A *problem-space* attack modifies the artefact itself and must satisfy four constraints: the transformation has to be available, semantics have to be preserved, the result has to be plausible, and it has to survive preprocessing. Moving between the two is the inverse feature-mapping problem, and in the malware setting the modified sample must still install and still run [22]. The distinction is load-bearing for Chapters 4 and 5, because attacks on static Android detectors operate at code level, upstream of feature extraction, so a representation derived from that extraction inherits the attacker's influence whatever the learning architecture [112]. Which operators reach which representation depends on what is extracted, and each chapter settles that where it states its threat model. Early work on Android established the feature-space case [113].

Adaptive attacks and robustness. An *adaptive* attack is constructed in knowledge of the defence and tuned against that specific defence, in contrast to a fixed attack applied unchanged [20]. The recurring reason a defence appears to hold and does not is gradient masking, where the defence obstructs the optimisation an attack relies on without reducing the attacker's reach [19]. *Robustness* is therefore defined relative to a stated threat model and a stated attack budget. It is a property of the pair, never of a model alone, and a robustness figure reported without those parameters cannot be compared with another [21]. Gradient-based attacks at fixed perturbation budgets [114, 115] are the reference instruments against which the defences of Chapter 6 are measured.

2.6 Evaluation Methodology and Experimental Bias

Chapter 1 reported what happens to published performance under a stricter protocol. This section states the mechanisms behind those results and fixes the standard this dissertation holds itself to.

Sampling bias in space and time. *Temporal bias* is a split that allows a model to learn from samples postdating the ones it is tested on, which reports a figure no deployment reproduces. *Spatial bias* is a test set whose composition differs from the deployment, which leaves precision uninterpretable. A sound evaluation constrains both: training data must precede test data, and the ratio between classes in the test set must reflect the setting the system is claimed for [12]. Drift is the mechanism that makes a randomly ordered split optimistic [11]. Curation of the corpus decides as much as the split does, and guidance published since those constraints were formalised revises parts of them [116, 117].

Leakage. Any transformation fitted before the split moves information from the test set into training. Resampling, feature selection, normalisation and model selection all have this property when applied to the whole corpus, and the resulting figure measures the procedure rather than the model [13]. Duplicate and near-duplicate samples are a second channel, since repackaging produces applications that differ only in the injected payload and copies of one application may fall on both sides of the split. The measured cost is uneven: limited for supervised classification, and substantially larger for unsupervised family clustering [15, 14].

What a score means. Axelsson derived the constraint that binds every detector deployed against a rare event: because the base rate of intrusions is low, the probability that an alarm signals a genuine intrusion is dominated by the false-alarm rate, and a detector must therefore reach a false-alarm rate that is difficult to attain before its alarms carry information [43]. Accuracy and precision are consequently uninterpretable without the class ratio that produced them [13], and a reported accuracy is read against the floor set by a majority-class or constant-output predictor. In time-series detection a further convention distorts comparison: *point adjustment* marks an entire anomalous window as detected once any single point inside it is flagged, which inflates F1 to the point that randomly generated scores outperform published detectors [118, 119]. Some standard benchmarks also contain mislabelled or trivially separable anomalies, so a high score on them is weak evidence [120].

Baselines and adaptive evaluation. A reported gain is not established until a simple baseline has been run under the same protocol. Where that has been done, the gap has often disappeared: re-implementing eight provenance-based intrusion detectors in one framework left a simple neural network matching the state of the art on most of the datasets tested [121]. For a defence the corresponding rule concerns the attack. A robustness claim is bounded by the strongest attack applied to it, so the defence has to be evaluated against an attack adapted to it [21], and defences evaluated only by their own authors have repeatedly failed under adapted attacks [19, 20].

The standard adopted here. Every empirical chapter of this dissertation states its split policy and uses time-aware splits where the timestamps permit, reports the class ratio alongside any aggregate score, reports no point-adjusted figure, places a trivial baseline beside each headline result, and states preprocessing and random seeds [122]. Results carried over from the source publications cannot be retrofitted to all of this, and each chapter says in its Limitations which parts were not met.

2.7 Summary and Research Gaps

Three threads run through the preceding sections. The first concerns what an analysis system is able to observe. An Android package exposes a manifest and bytecode to static reading, exposes executed behaviour and recorded traffic to dynamic instrumentation, and neither mode dominates the other. Network deployments expose flows, and industrial and IoT systems additionally expose process values and a comparatively regular pattern of who communicates with whom. The second thread concerns how those observations are structured. Program behaviour is encoded as control-flow and function-call graphs, communication as graphs over

hosts or services, and both may be lifted to hypergraphs when a relation holds among more than two elements. The operators that learn over these structures are bounded in what they can distinguish, and the projection from a hypergraph back to a graph discards information that has now been measured. The third thread concerns measurement. Reported performance depends on the split, on the corpus, on the baseline it is compared against, and, where a defence is claimed, on the strength of the attack applied to it.

Four questions are left open by this material, and each is taken up by one of the research questions stated in Section 1.3.

G1. How behaviour in networked and industrial systems should be represented as deployments change. Communication whitelists, service-dependency graphs, flow records, and process time series have each supported anomaly detection under the conditions in which they were fitted [91, 92, 123]. The evidence is split across different observation units and system assumptions, and traffic-based models lose a substantial part of their measured performance once a deployment moves in time or place [99]. The literature lacks a common account of how these representation choices support detection as the observed system changes. RQ1 takes up the question through communication graphs and modular analysis of network and process features.

G2. What combining program structure with observed behaviour buys. Static and dynamic analysis are usually set against one another as alternatives. Where they have been measured under one protocol, static analysis proved at least as effective as dynamic analysis and the combination measured best [25], but that comparison has rarely been repeated, and hybrid systems are seldom evaluated against each modality alone on the same samples [57]. The gain attributable to the combination, as distinct from the gain attributable to either part, is therefore not established. RQ2 takes up this question.

G3. Whether higher-order representation of program structure pays for itself. Program structure is represented pairwise, and a relation holding among several functions at once is flattened into a set of edges that cannot record it as one group [26]. Hypergraphs remove the restriction and the cost of the projection has been quantified [70, 71], yet the sceptical result also stands: an operator whose vertex weights do not vary between the hyperedges a vertex belongs to is equivalent to one acting on a weighted clique expansion [68, 69]. Whether higher-order encoding, and the attention weighting often applied with it, improves detection and classification of Android malware is open, and no theoretical analysis of attention scoring in hypergraph operators has appeared since 2024 [83, 84]. RQ3 takes up this question.

G4. How an adversary model becomes a reproducible benchmark. Defences are commonly evaluated against attacks chosen by the party proposing them, and a robustness claim is bounded by the strongest attack applied [21]. Threat models are stated inconsistently, and the knowledge assumptions used in academic evaluations are more generous than deployment usually allows [107]. How an explicit adversary model is translated into a benchmark that a third party can reproduce, and what such a benchmark then shows about the relation between clean predictive performance and robustness, has no accepted answer. RQ4 takes up this question.

Anomaly Detection in IoT and Industrial Control Systems

3.1 Scope, Research Question, and Contributions

This chapter answers RQ1.1 and RQ1.2, the two subquestions of RQ1 stated in Section 1.3, and takes up gap G1. Three pieces of work support the answer. The first represents service-to-service communication in an IoT middleware as a graph and flags relations absent from that graph. The second is NADICS, a learning framework for network and process data, together with its S7 and PROFINET extractor and its integration in IUNO. The third studies a multivariate reconstruction detector over SWaT process data by combining elements of MAD-GAN and TadGAN.

Each contribution observes its own setting. The service-graph study represents inter-service messages as graph relations. In the industrial work, the NADICS extractor reduces packet captures to connection records, while the SWaT study represents process measurements as multivariate time windows. No common protocol evaluates all three settings, so the chapter treats them as related but separate experimental strands.

The evidence also differs in kind. The service-graph contribution was disseminated at the 2018 Passive and Active Measurement Conference (PAM) [32]. The original industrial framework is supported by released code and IUNO deliverables [33, 34]. The SWaT study is recorded in an unpublished internal report written by the candidate. Because that report is a research record, it is not a bibliography item. The candidate designed the method, implemented the experimental pipeline, ran and analysed the experiments, and wrote the report. Section 3.5 reports the empirical evaluation of the SWaT detector.

3.2 Observation Models and Datasets

The contribution strands observe different events. In the IoT middleware, the monitor sits at the Knowledge Agent, the entry point of the Virtual State Layer through which services exchange data [124]. It observes an inter-service message rather than a packet. Vertices are service addresses, and an edge records an observed communication relation. The message also exposes metadata such as the operation performed [31].

NADICS receives tabular records. Its network path takes connection features derived from S7, PROFINET, or conventional network traffic. The SWaT path does not model communication.

Table 3.1. Observation units used in Chapter 3. The rows are not numerically comparable because their labels, sampling processes, and validation protocols differ.

Study	Unit observed	Representation	Decision
IoT service graph	inter-service message	vertices and whitelisted relations	known or unknown relation
NADICS in IUNO	packet-derived connection record	protocol and flow features	binary or attack-class label
SWaT GAN study	sensor and actuator sample	multivariate time window	anomalous or normal timestamp

It consumes the sensor and actuator values of a six-stage water-treatment process, sampled once per second [123, 10].

The studies use different decision rules. The service-graph method stores the inter-service relations observed during its learning phase and flags a later relation that is absent from the graph. The GAN study uses mixed normal and attack windows for fitting, combines reconstruction and critic scores, and applies an adaptive threshold that assigns a binary prediction to each timestamp. Both methods depend on representative fitting data: an attack present during learning may be incorporated into the learned representation. The NADICS evaluation on the S7-300 corpus uses labelled classification instead, so it addresses a different decision problem. It holds out whole capture sessions. Its attack share of 4.54 per cent was set by the executed attacks and does not reproduce the class ratio of routine operation.

3.3 Communication Graphs for IoT Services

The lineage set out in Section 2.4.3 places communication whitelisting before service-oriented IoT deployments. Flow whitelists were built for control networks [91], and communication patterns were modelled per host pair soon afterwards [92]. The service-graph contribution acknowledges that lineage and identifies its proposed difference: an IoT deployment changes as services are added or removed, so its communication model must support interactive revision rather than a fixed learning phase [32].

The model is compact. Services are vertices, observed relations are edges, and each node maintains its own graph. A learning phase runs when a service starts and labels every relation observed during that period as normal. A relation absent from the model later raises a notification. The site owner may legitimate it, after which the system relabels the edge as normal. Detection runs on the node hosting the service, so the design does not require central collection [32].

The intended distinguishing property was a portable model attached to service metadata. A service moved to another node would then carry its learned relations with it. The study also states that the implementation identifies services by concrete local addresses, which prevents that portability. Semantic service discovery would have been required for site-independent identities.

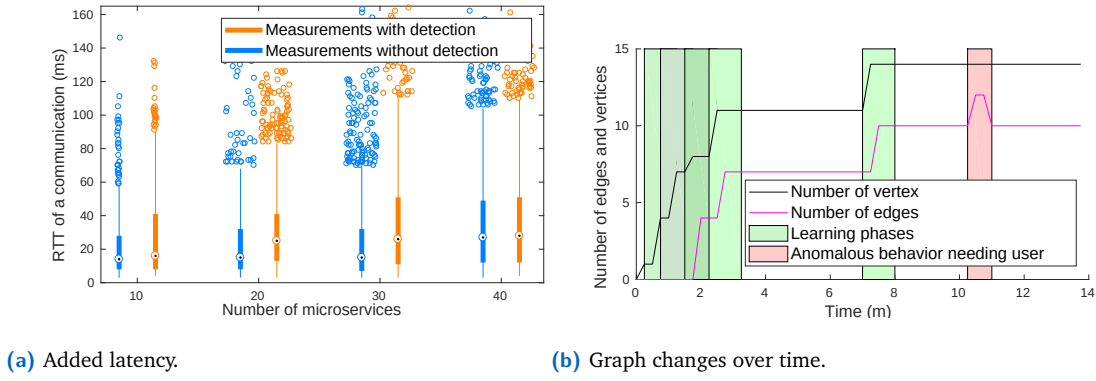


Figure 3.1. The two measurements reported for the service-graph design. Panel (a) gives the inter-service round-trip time with and without detection for ten to forty services. Panel (b) gives the vertex and edge counts of one node’s graph over a single run, with the learning phases and the interval of anomalous behaviour shaded. Reproduced from [32], to whose measurement design the candidate contributed.

Two costs were reported. Memory is modelled per vertex as 100 bytes plus 88 bytes for every incident edge, and a factor of 1.33 accounts for the stacked hash tables that hold the graph. That factor enters twice, once inside the per-vertex expression and once over the vertex set, so twenty services with ten relations each occupy 33.7 KB. The second cost is latency: about 6 ms added to an inter-service round trip of 24 ms, measured over configurations of ten to forty services [32].

Figure 3.1 reproduces both measurements. In the latency panel the two distributions overlap heavily. Medians without detection rise from roughly 14 ms at ten services to roughly 27 ms at forty, and every configuration carries a long upper tail past the 160 ms axis limit. Neither the repetition count nor the statistic behind the quoted 6 ms and 24 ms is stated, so both values are best read as an aggregate over the four configurations. The added cost does not grow between ten and forty services.

Panel (b) covers one run of about fourteen minutes. Learning phases cluster in the first three minutes as services start, and another follows at minute seven when a service is added. The graph then settles at fourteen vertices and ten edges. One anomalous interval is recorded shortly after minute ten, and the edge count returns to its earlier value once it ends, so the trace records the detection but no legitimation by the site owner.

No labelled attack set was used, and that interval holds a previously unseen connection rather than an executed attack, so the study reports no detection rate and no false-positive rate. A whitelist emits a verdict, so even a labelled trace yields no threshold curve without a change of method. RQ1.1 is answered here at the level of representation, update procedure, and measured overhead.

3.4 The NADICS Framework and IUNO Integration

NADICS, Network Anomaly Detection for Industrial Control Systems, is a Python framework built at the Chair of IT Security of the Technical University of Munich to detect attacks against PLC and SCADA installations from network traffic. By design, it was limited to detection. The framework is organised in three modules. Preprocessing turns recorded traffic into a labelled feature table, encodes the string columns, scales the numeric ones, and resamples a corpus whose classes are imbalanced. The learning module holds the estimators behind one interface, so that classical models and neural models can be exchanged by name without the pipeline being rewritten: the released repository contains the scikit-learn estimators, the IUNO deliverable additionally reports a deep neural network and a convolutional network [33], and the generative adversarial detector of Section 3.5 was added later at the same interface for multivariate process data. Postprocessing carries the verdict to whoever acts on it. The released code leaves this stage open, because alarm handling depends on the security policy and the network layout of the individual site. The S7 and PROFINET extractor existed as a separate project artefact, so the released repository takes a feature table as its input.

The design follows from the data situation in industrial control. No public corpus of labelled traffic from a real industrial network was available [33], so the estimator that performs well depends on the plant and on how its labels were produced. Fixed interfaces around the learning stage let the models be compared under a single protocol on whatever corpus a site can supply. They also let feature importances be inspected, so that dimensionality and the false-positive rate can be reduced.

Figure 3.2 shows the pipeline as it was deployed in IUNO. Two data artefacts separate its three stages. The first stage produced the corpus. Attacks of five classes (authentication bypass, CPU stop and start, reconnaissance, denial of service and brute force) were executed in a real industrial environment against Siemens S7-300 controllers that exchanged S7comm traffic with a control server, and the traces were labelled both in binary form and by attack class [33]. In the second stage, the public analysers TShark, Bro-IDS and Argus were combined with dissectors written for S7 and PROFINET, and the TUM feature extractor reduced each network connection to one labelled feature vector, written as CSV or JSON [41, 33]. The third stage is the framework proper. It reads that dataset, fits one of the registered estimators, applies the resamplers and the outlier tests, and reports which features carried the decision. Detections reached the visual security control room as JSON event messages over the authenticated message bus of the IUNO aggregation system [33], and acting on them belonged to the control room [34].

The modules map onto these stages at the dataset. Preprocessing spans the second stage and the input side of the third, where encoding, scaling and resampling are applied to the table, while learning and postprocessing both sit in the third stage. Corpus generation lies outside the framework and appears in the figure because the labels had to be produced somewhere. Inside the engine, a layout file declares the full columns, the feature subset, the label column, and the string features that require encoding. Learning methods sit in registries that map a name to an estimator. The original repository holds thirteen classifiers, four novelty or outlier detectors, fifteen resamplers, and two boosting meta-estimators. The framework calls only

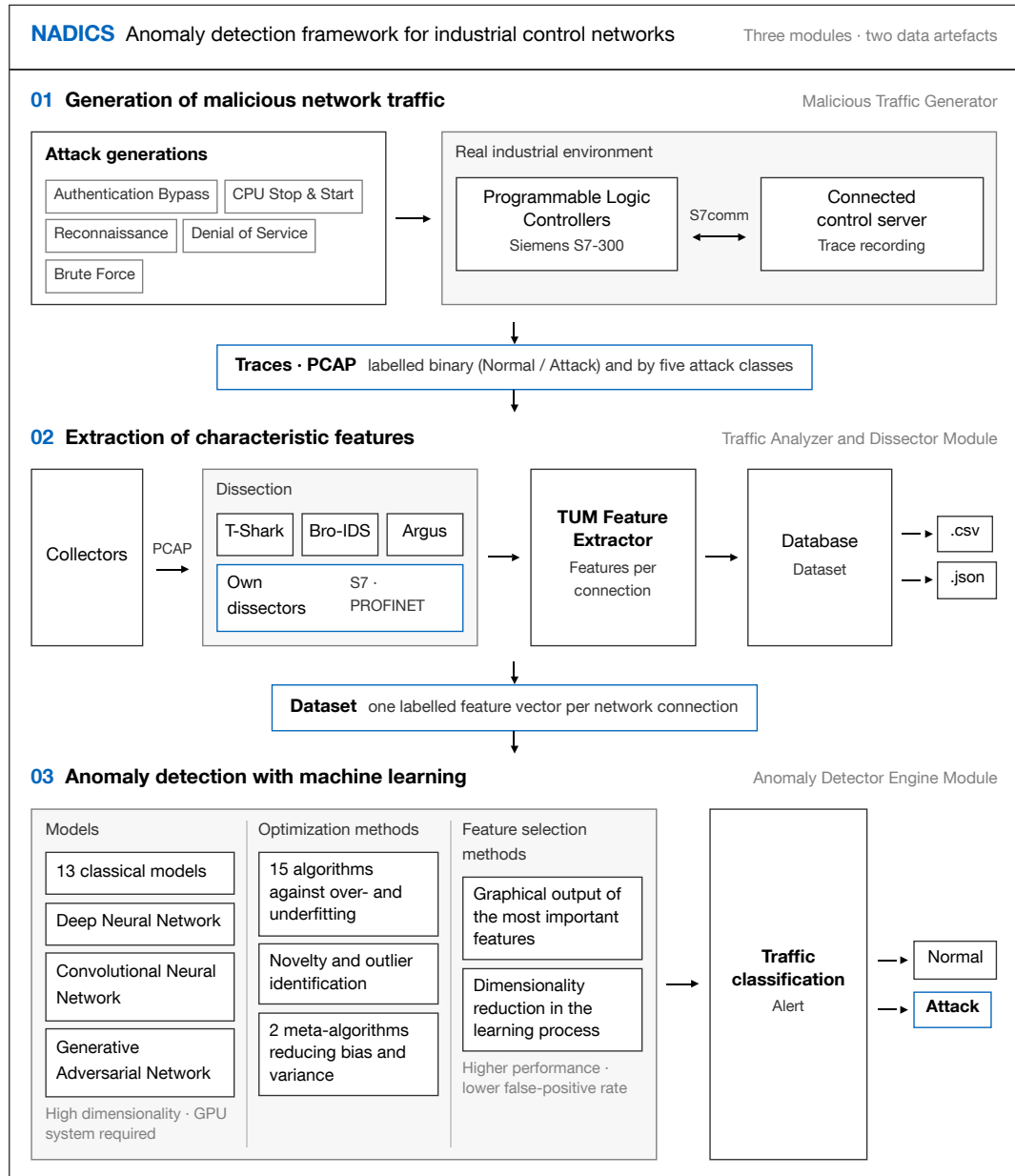


Figure 3.2. Architecture of the NADICS framework. Malicious traffic is generated in a real industrial environment, dissected into protocol-characteristic features, and classified by the anomaly detection engine. The stage structure follows the one reported in the IUNO deliverable [33].

fit and predict, following the scikit-learn estimator protocol [125]. It fits min-max scaling on the training partition and applies the fitted transformation to the test partition.

Figure 3.3 shows where the framework ran. The environment was built at the Volkswagen plant in Wolfsburg from hardware of the kind used in regular production [41]. A Hirschmann industrial switch carries three segments, the intranet, the demilitarised zone and the plant network, and manually configured Eagle routers separate them. The first of four servers runs the WinCC and OPC UA clients that originate test traffic, together with a Nagios instance and the database of the aggregation system. The second is the virtualisation host for the detection tools contributed by AISEC, DFKI, SIT and TUM, and the third serves the visual control room to the intranet. The fourth combines the WinCC server with the collector, which gathers traffic from the plant network and passes it, bundled and pre-filtered, to the detection tools, so NADICS was one of four detectors reading the same collector output. Siemens controllers occupy the plant network behind a Scalance switch. The collector observed that network passively at the plant switch, never in line with the traffic, which matches the detection-only design: an alert could reach the control room, but nothing in the path could stop a packet.

The IUNO deliverables describe the tool and its environment but print no measured result for it [33]. The values below come from the experimental records of the project and cover the corpus and its feature schema, the detection results on the held-out session, the alarm burden over a benign monitoring period, and the throughput of the extractor on the prototype collector.

3.4.1 The S7-300 Corpus

The corpus was recorded in 2018 in the environment of Figure 3.3, where six Siemens S7-300 controllers exchanged routine polling and control traffic with WinCC-based services. Complete selected channels were kept, together with ARP and DCP traffic and the connection attempts belonging to those channels. No random packet sampling was applied.

Recording opened with a two-hour clean baseline, which was excluded from the corpus totals. That interval fixed the communication relationships, the polling behaviour and the protocol activity expected under the operating modes present at the time. Six capture sessions followed, with 24 hours of labelled traffic in total. The collector recorded 12,616,875 Ethernet frames in that period, an average of 146.03 frames per second. The captured frames hold 2,317,074,962 bytes, and the six capture files occupy about 2.519 GB in the classic PCAP format.

Twenty-five controlled attack episodes were executed during the sessions, five repetitions of each of the five classes the deliverable names [33]. Background communication continued while they ran. Table 3.3 lists the duration of every episode. The shortest lasted 3 min 07 s and the longest 9 min 55 s, and together they cover 2 h 51 min 05 s. Episodes also differed in the channels they touched and in how often a connection was re-established. Labels were assigned from the execution logs and from inspection of the packet traces, and only the communications an episode actually affected received an attack label, so background traffic concurrent with an attack keeps the normal label. Two attack types were never run against one channel at the same time.

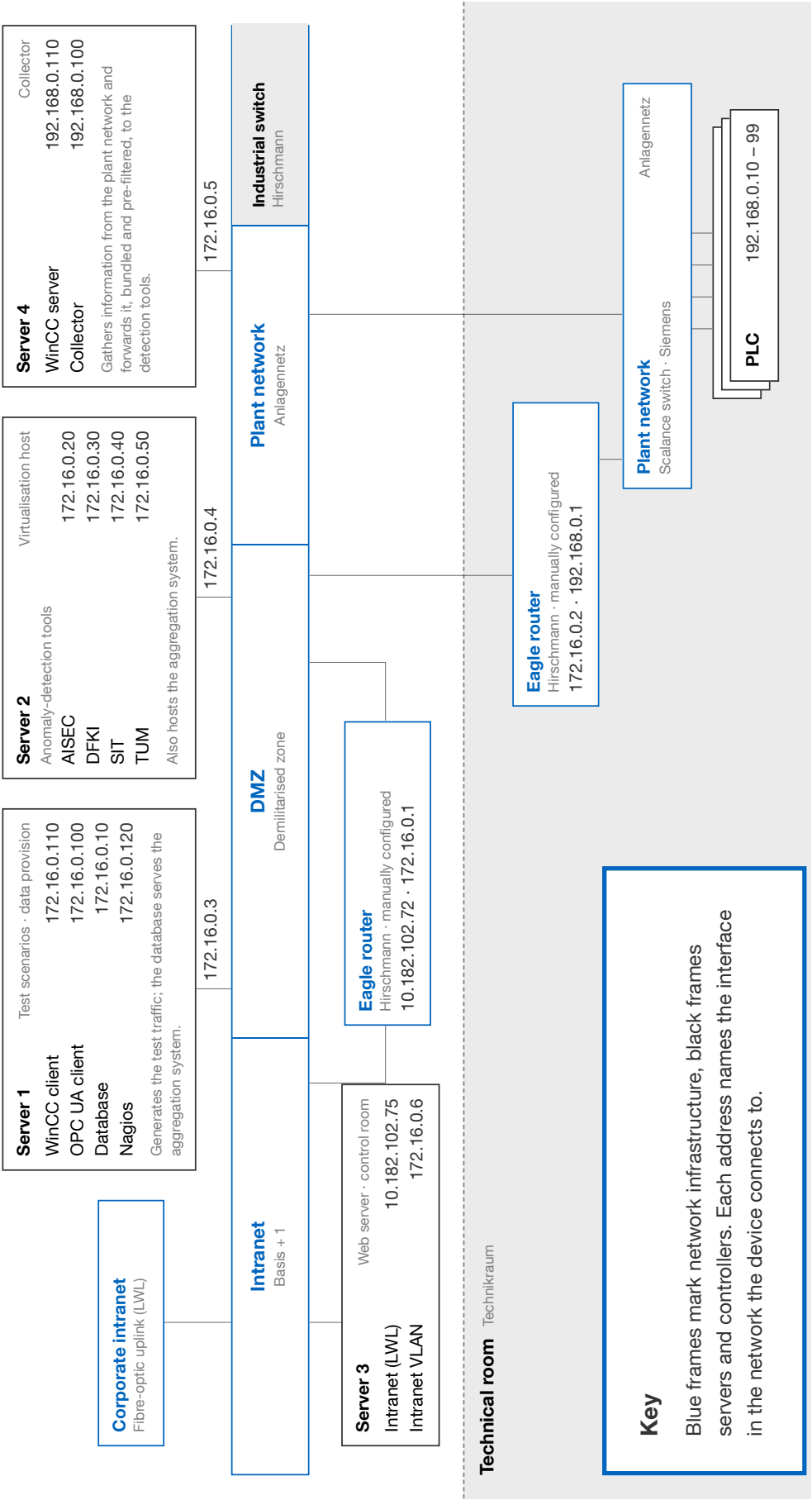


Figure 3.3. The IUNO test environment at the Volkswagen plant in Wolfsburg. Four servers, three network segments and a Siemens plant network reproduce the production setting in which NADICS ran as the TUM detection tool on the virtualisation host. The network TAPs fitted at the relevant interfaces are omitted from the drawing [41].

Table 3.2. Composition of the S7-300 corpus and its partitions. Whole capture sessions were assigned to the partitions, four to training and one to each of the other two, and every attack episode lies entirely within one partition. The episode column counts executed attacks, not records.

Class	Corpus	Train	Validation	Test	Episodes
Normal	113,751	76,556	18,587	18,608	
Authentication bypass	1,058	441	269	348	5
CPU stop and start	593	359	90	144	5
Reconnaissance	1,803	1,070	366	367	5
Denial of service	1,230	765	278	187	5
Brute force	725	445	93	187	5
Total	119,160	79,636	19,683	19,841	25
Capture time (h:min:s)	24:00:00	16:01:57	3:56:27	4:01:36	

Table 3.3. Duration of the 25 attack episodes in minutes and seconds. Each attack type has three training episodes, one validation episode and one test episode.

Class	Training episodes	Validation	Test	All five
Authentication bypass	5:33, 4:01, 5:50	7:16	9:34	32:14
CPU stop and start	7:03, 8:23, 3:07	4:04	7:39	30:16
Reconnaissance	6:42, 9:40, 7:28	9:55	8:27	42:12
Denial of service	6:41, 9:39, 7:07	7:46	5:38	36:51
Brute force	8:05, 3:16, 4:06	4:15	9:50	29:32
Total				171:05

Extraction produced 119,160 feature records. Of these, 113,751 are normal and 5,409 carry one of the five attack labels, an attack share of 4.54 per cent. Table 3.2 gives the composition. The training sessions hold 79,636 records, the validation session 19,683 and the test session 19,841. Each attack type contributes three training episodes and one episode to each of the remaining partitions, and because episodes differ in duration and in the channels they touch, the attack records of a class are not divided between the partitions in a fixed proportion. An episode, its recovery interval and the windows that depend on it stay inside a single partition, and no rolling window crosses a partition boundary. The attack counts must not be read as a count of attacks. A single episode of a few minutes generates many records, and the corpus averages about 216 affected records per episode.

The extraction schema has 63 fields in five groups: 18 flow fields, 13 timing fields, 18 protocol fields, nine relational fields and five process fields. Forty-six are numerical, eleven categorical and six binary, and encoding of the categorical fields expands the input dimensionality of the model. The flow group follows the output of Argus and Bro-IDS, which the extractor wrapped. The UNSW-NB15 intrusion-detection corpus derives most of its features from the same two tools [126], so this group coincides with the flow columns of that corpus. Protocol fields cover S7 message and function codes, addressed memory areas, address entropy and error codes, together with COTP and OPC UA fields. Relational fields describe the communication partners, the age of an edge, and the number of distinct destinations and ports observed in the preceding minute. The process group reads one preconfigured anchor tag per channel, whose identity and units are held as metadata, so a value delta is never computed across unrelated tags. Unobservable protocol fields carry an explicit missing state, and an absent OPC UA security policy is not encoded as the policy *None*.

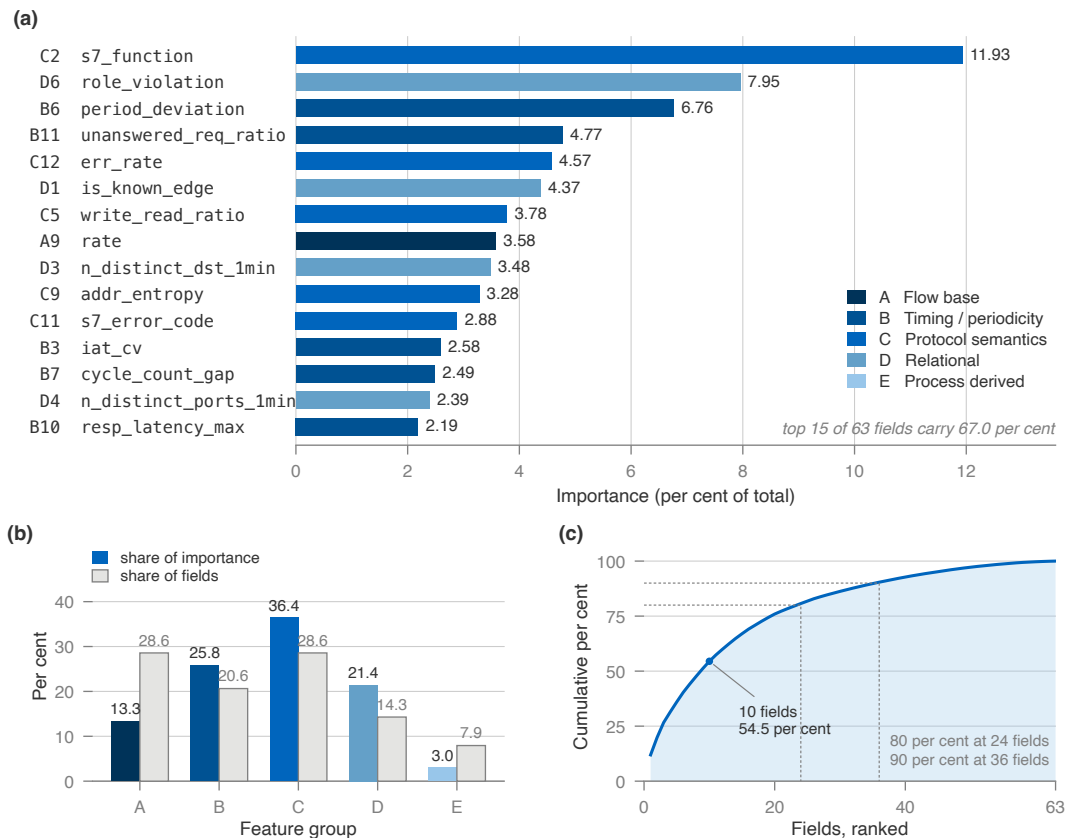


Figure 3.4. Random-forest importance of the 63 extraction fields. (a) The fifteen most important fields, coloured by group. (b) Each group's share of the total importance against its share of the field count. (c) Cumulative share over the ranked fields.

Figure 3.4 shows the importance of each field in the random forest of Section 3.4.2. The forest was fitted to the training partition; the impurity reductions from splits on each field were summed across its trees, normalised over all 63 fields, and ranked. Panel (a) shows the fifteen highest, and Appendix A lists all 63. `s7_function` leads at 11.9 per cent, because a stop or a start is an ordinary S7 operation whose function code is visible in the request, and `role_violation` follows at 8.0 per cent, because a write from a host whose role does not write is anomalous even over an established connection. Both signals have limits: legitimate maintenance issues the same function codes, and a compromised engineering station writes from a role that is permitted to write.

Panel (b) sets each group's share of the total importance against its share of the fields. Protocol semantics holds the largest share at 36.4 per cent, although it also holds the most fields. The comparison that carries information is the share per field: the nine relational fields average 2.38 per cent each, against 0.74 for the eighteen flow fields and 0.60 for the five process fields. Only the process fields receive less importance per field than the flow statistics. That is consistent with the position the schema takes: an industrial detector built on Argus and Bro flow features alone discards most of what separates an S7 attack from S7 polling. Panel (c) gives the cumulative share. Ten fields reach 54.5 per cent of the total importance, twenty-four reach 80 per cent and thirty-six reach 90 per cent, so the tail of the schema is thin and a reduced schema is plausible. NADICS exposes an importance ranking for exactly this purpose [33].

One record covers a non-overlapping ten-second window of an activity-bearing connection or channel, and a shorter final record is written when a session closes. PROFINET RT traffic is keyed at layer 2, since it carries no transport-layer connection. The windowing refines the per-connection unit of Figure 3.2 so that a change during a long-lived session can be scored while the session is still open. Channels that fall silent emit no record.

Three definitions were adjusted so that no record depends on information from after its own window closes. The duration field measures the exported segment and not the eventual lifetime of an open connection, while a separate field retains the time elapsed since the underlying connection opened. The ratio of unanswered requests counts only those requests whose one-second response deadline had already expired at window closure, and pending requests are excluded from both the numerator and the denominator. Trailing source and timing state is held over 60 seconds, and the period features are computed from past observations. Encoders, imputation and scaling were fitted on the training partition alone, and the classifier together with its preprocessing was then held fixed for validation and testing.

3.4.2 Detection Results

The classifier is a random forest of 300 trees with a maximum depth of 16 and a minimum leaf size of five. Class weighting was applied per bootstrap sample, and the training data were not resampled. Two decisions were read from the same fitted model: a binary verdict of normal against attack, and an assignment to one of six classes, the normal class and the five attack types. Every figure below comes from the held-out test session, which holds 18,608 normal and 1,233 attack records.

Table 3.4. Six-class confusion matrix on the 19,841-record test session, with rows giving the true class. Predicted classes are abbreviated: N normal, AB authentication bypass, CS CPU stop and start, RC reconnaissance, DS denial of service, BF brute force. The last three columns give precision, recall and F_1 for the class of that row.

True class	Predicted class						Score (%)		
	N	AB	CS	RC	DS	BF	Prec.	Rec.	F_1
Normal	18,518	21	5	31	18	15	99.74	99.52	99.63
Authentication bypass	22	292	8	12	1	13	88.75	83.91	86.26
CPU stop and start	3	1	134	2	3	1	90.54	93.06	91.78
Reconnaissance	9	7	0	337	6	8	84.25	91.83	87.87
Denial of service	5	0	0	4	178	0	85.99	95.19	90.36
Brute force	9	8	1	14	1	154	80.63	82.35	81.48

Table 3.4 reports the six-class confusion matrix together with the per-class scores. Collapsing the same predictions into the binary decision gives 18,518 true negatives, 90 false positives, 48 false negatives and 1,185 true positives. Binary accuracy is 99.30 per cent, attack precision 92.94 per cent and attack recall 96.11 per cent, for an attack F_1 of 94.50 per cent. The false-positive rate over normal records is 0.484 per cent and the balanced accuracy 97.81 per cent. On the six-class task, accuracy reaches 98.85 per cent, the macro- F_1 89.56 per cent and the weighted F_1 98.86 per cent.

Accuracy is the least informative of these numbers. Normal records make up 93.79 per cent of the test partition, so a detector that never fired would already score 93.79 per cent, and the gap between the weighted and the macro F_1 , 98.86 against 89.56 per cent, measures how much of the six-class figure the normal class carries. Per-class recall is highest for denial of service at 95.19 per cent and lowest for brute force at 82.35 per cent, with authentication bypass close behind at 83.91 per cent. Most errors of these two classes fall on other attack classes. Authentication bypass loses 22 of its 348 records to the normal class and 34 to the other attack types, while brute force loses nine of its 187 records to the normal class and 24 to the other attack types, 14 of them to reconnaissance. Both attacks reuse the protocol operations of legitimate engineering access, so a window holding only a few such operations offers little that separates them from ordinary traffic or from each other.

All of these scores are computed over records. They show that the extracted features separate the five executed attacks from the surrounding traffic of this plant under a session-disjoint split.

3.4.3 Benign Operation and Alarm Burden

A second exercise measured what the detector does when nothing malicious happens. It ran in the Volkswagen environment with four active PLCs and ordinary supervisory traffic. A separate two-hour baseline preceded 24 hours of benign monitoring. Over that day the collector recorded 8,472,913 frames, an average of 98.07 frames per second. The captured frames hold 2,186,039,471 bytes, about 2.322 GB in the classic PCAP format, and the extractor emitted 83,847 records. Engineering work and planned device restarts took place during the period and count as benign.

Table 3.5. Extraction workloads on the Raspberry Pi 3 prototype. Input frames are the frames presented to the extraction pipeline, and processed frames exclude those dropped before processing. Rates and CPU utilisation are averages over the replay, and CPU utilisation refers to the capacity of the whole device. The emission delay runs from the close of a window to the emission of its record.

Quantity	Light load	Moderate load	Overload
Replay duration (s)	600.14	599.87	600.09
Input frames	599,827	1,498,463	2,100,731
Dropped frames	4	361	245,897
Processed frames	599,823	1,498,102	1,854,834
Frame loss (%)	0.00067	0.02409	11.70531
Input rate (frames/s)	999.48	2,497.98	3,500.69
Processed rate (frames/s)	999.47	2,497.38	3,090.93
Feature records	5,842	5,887	5,781
Mean CPU utilisation (%)	22.6	43.1	50.4
Peak extractor memory (MiB)	302	381	436
95th-percentile emission delay (ms)	96	234	817

The detector classified 83,740 records as normal and 107 as attacks, all of them false, a record-level false-positive rate of 0.128 per cent. Positives were merged into alarm episodes before presentation to the control room: positives that share a source, a destination and a predicted attack class belong to one episode, and an episode closes once 60 seconds pass without a further positive. One notification opens an episode, and later positives update it. Nine alarm episodes were raised over the day, 0.375 per hour on average, and each was assigned a benign cause. Engineering workstation activity accounts for 43 positive records in three episodes and OPC UA reconnections for 28 records in two. Brief communication interruptions produced 23 records in two episodes, and planned device restarts 13 records in two.

No attack was executed during this period, so every alarm it raised was false. Its 0.128 per cent record rate lies below the 0.484 per cent of the test session, whose normal records also include the recovery intervals of executed attacks. The recovered causes share a structure that matters for deployment: each is a legitimate departure from routine polling, so the alarm burden of this design follows the rate at which such events occur in the plant.

3.4.4 Extractor Throughput on Constrained Hardware

The extractor was also exercised on a Raspberry Pi 3 Model B. The deliverable of June 2017 names Raspberry Pi boards as the prototype platform [41]. The board carries four cores at 1.2 GHz, 1 GB of RAM and 100 Mbit/s Ethernet. Only the extraction stage ran on it, comprising selected-field capture and dissection of S7 and PROFINET traffic, the bounded rolling feature computations and emission of CSV or JSON. Model fitting and inference remained on the server, and archival of full packet captures to the SD card was excluded. Each load replayed traffic over 100 configured communication channels with varying channel activity for about ten minutes.

Table 3.5 reports the three loads. At the moderate load, 1,498,463 frames were presented in 599.87 seconds and 361 were dropped, a loss of 0.024 per cent at a processed rate of 2,497.38

frames per second. This input rate is about 17 times the average frame rate of the corpus capture. The extractor used 43.1 per cent of the CPU capacity of the device on average and at most 381 MiB of memory. The 95th percentile of the delay from the close of a window to the emission of its record was 234 ms, a delay that excludes the ten seconds in which the window accumulates. The light load lost four of its 599,827 frames at 22.6 per cent CPU utilisation. The overload row does not describe a supported rate: 11.71 per cent of the input was dropped, and the processed rate stayed at 3,090.93 frames per second against an input rate of 3,500.69. Mean CPU utilisation reached 50.4 per cent there, and the 95th-percentile emission delay rose to 817 ms. The three runs emitted 5,842, 5,887 and 5,781 records, fewer than the 6,000 that 100 continuously active channels would give in ten-second windows over ten minutes, because channel activity varied.

3.5 A GAN-Based Multivariate Detector for SWaT

This section reports an empirical evaluation of a generative adversarial network (GAN) for detecting abnormal behaviour in an industrial water-treatment process. The study examines how the representation of process measurements and their temporal context influence the balance between detecting anomalous observations and generating false alarms. Its design combines the multivariate approach of MAD-GAN [127] with the encoder, cycle-consistency objective and Wasserstein critics described in TadGAN [128]. The reported results were measured on the SWaT evaluation cohort and compare five input representations.

The detector operates on process measurements from the Secure Water Treatment (SWaT) testbed. SWaT comprises six treatment stages, with programmable logic controllers coordinating sensors and actuators and a supervisory network connecting the controllers to SCADA (Figure 3.5). The process dataset contains 51 sensor and actuator variables recorded at a nominal frequency of one observation per second [123]. These variables describe the physical behaviour of the plant, allowing the detector to consider changes in individual measurements together with relationships among measurements across the treatment process.

The dataset documentation describes 36 attacks distributed across four categories (Figure 3.6): 26 single-stage, single-point attacks; four single-stage, multiple-point attacks; two multiple-stage, single-point attacks; and four multiple-stage, multiple-point attacks [123]. These categories describe the scope of the interventions. They do not imply that all attacks produce equally observable process changes. An intervention affecting one sensor or actuator may also influence measurements elsewhere in the plant, while an attack with little physical effect may remain difficult to identify from process data alone. The aggregate calculations in this section concern anomalous timestamps and do not establish the number of individual attacks detected.

The source collection campaign comprises seven days of normal operation followed by four days containing attacks [123]. Preprocessing removes the first 21,600 observations from the normal-operation trace, which cover the initial six-hour stabilisation period (Figure 3.7). The empirical study uses all 449,919 timestamps of the attack-period recording. Its first 56,240 timestamps are divided between the training and validation partitions, which also receive separate portions of the normal-operation recording. The later 393,679 attack-

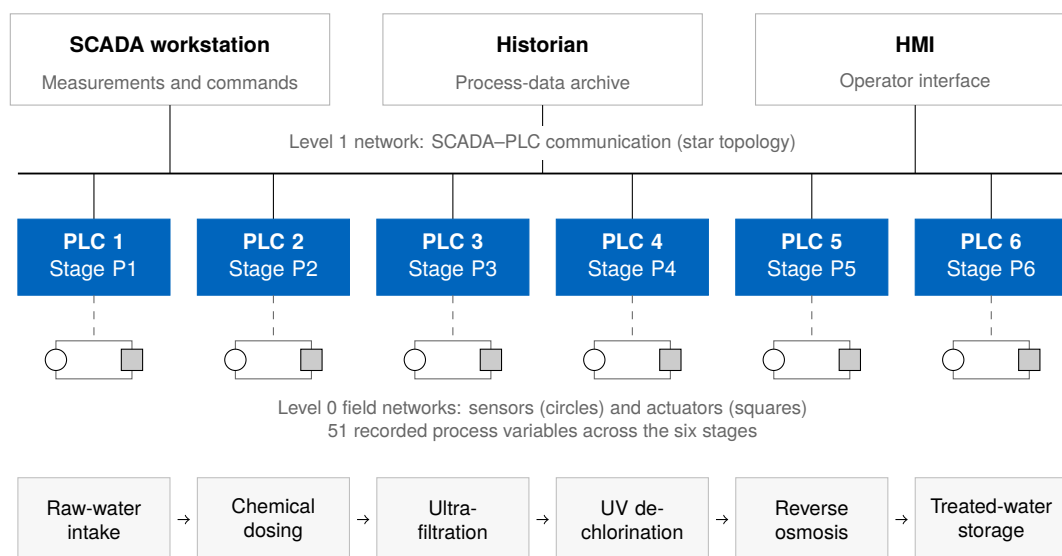
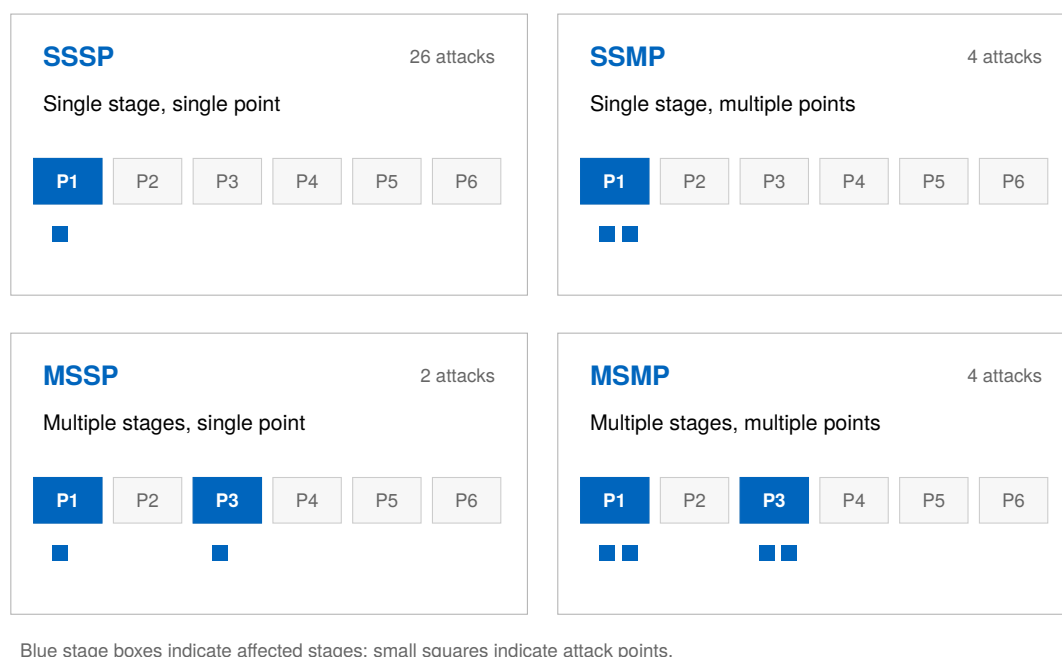


Figure 3.5. SWaT process and control architecture. Network levels follow the SWaT convention. The field-network symbols indicate sensors and actuators within each stage; the detector consumes their recorded values. Adapted from the supplied diagram and the dataset description [123].



Blue stage boxes indicate affected stages; small squares indicate attack points.

Figure 3.6. SWaT attack categories and their counts in the dataset paper [123]. Stage and point locations are schematic examples. The arrangement describes attack scope and carries no ranking of detection difficulty.

period timestamps form the test set. They comprise 51,453 anomalous and 342,226 normal timestamps, so anomalous observations account for 13.07% of the test cohort.

Collection campaign: 11 days, one sample per second

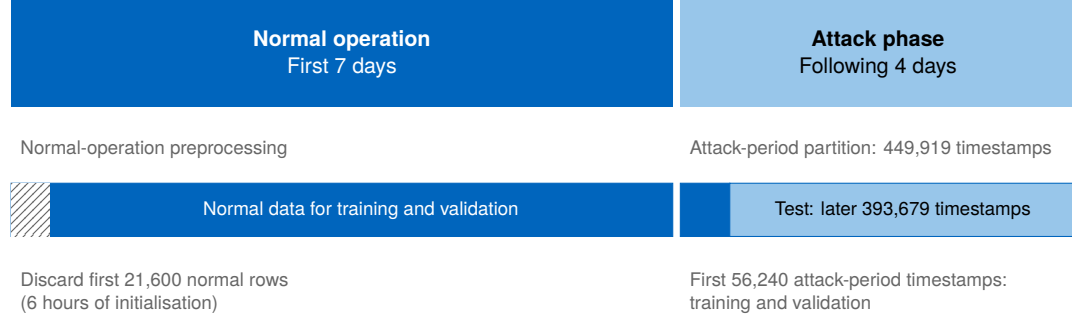


Figure 3.7. SWaT collection campaign and the partition used in the GAN study. The campaign follows the dataset paper [123]. After the initial normal-operation exclusion, the first 56,240 attack-period timestamps support training and validation; the later 393,679 timestamps form the test set.

All three partitions contain normal observations and labelled attack examples. Each attack episode is confined to one partition, and no window crosses a partition boundary or a discontinuity in the recordings. Partitioning precedes imputation, scaling, feature selection and window construction. Representations, model settings and detection thresholds are selected from the training and validation data. The model is therefore developed on earlier observations and evaluated on later observations that were not used for training or model selection.

The GAN objective does not require class annotations during gradient-based fitting. Nevertheless, the complete procedure includes supervised feature selection and model selection, and training includes attack observations. The resulting evaluation therefore concerns a detector developed with prior exposure to attacks. It does not establish detection performance for previously unseen attack types or a wholly unsupervised development process.

3.5.1 Model and Training

The model contains four jointly trained networks: an encoder, a generator, a data critic and a latent critic (Figure 3.8). For an input window X , the encoder E maps the observed process sequence to a latent representation $E(X)$. The generator G maps this representation back to the observation space, producing the reconstruction

$$\hat{X} = G(E(X)).$$

Latent samples z are drawn from a standard normal distribution. The data critic compares observed windows with generated windows $G(z)$, encouraging the generator to reproduce the distribution of the training sequences. The latent critic compares encoded observations $E(X)$

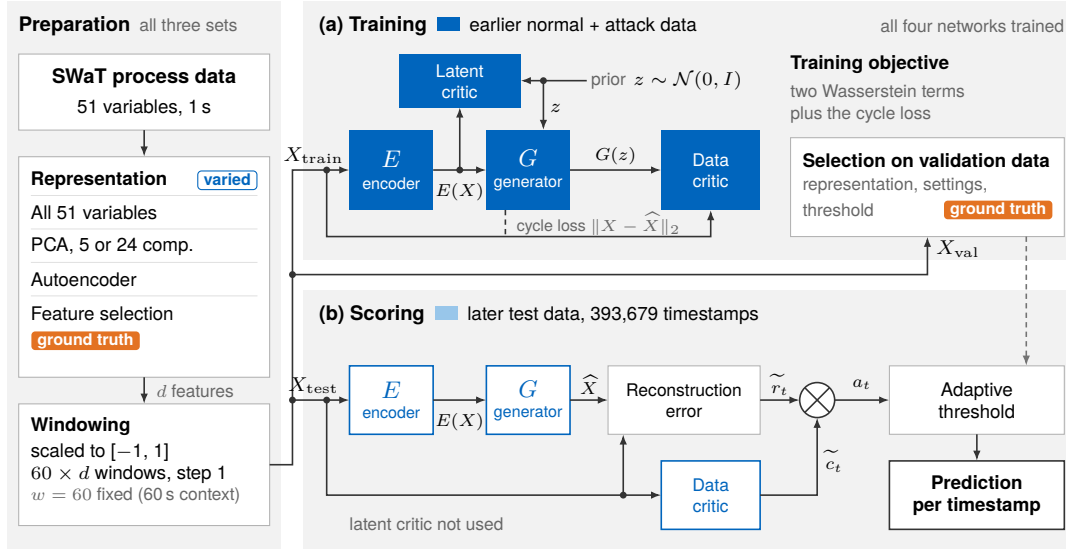


Figure 3.8. Multivariate GAN detection pipeline. (a) The four networks are trained together on earlier windows containing normal and attack observations; validation data select the representation, the settings and the threshold. (b) The later 393,679 test timestamps are scored with the trained networks; the latent critic has no part in the score a_t of Equation 3.1. The blue tag marks the input representation, the only factor varied between runs, and orange tags mark the steps that used attack ground truth. The window length is fixed at 60 samples. The scaler and PCA are fitted on training data and reused for validation and testing. The candidate implemented every processing step shown.

with samples from the latent prior. A cycle-consistency term penalises discrepancies between an observed window and its reconstruction,

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_X [\|X - G(E(X))\|_2].$$

Training combines the two Wasserstein objectives with this reconstruction constraint. The encoder provides a direct mapping from process observations to latent representations, while the cycle-consistency term encourages those representations to retain information required to reconstruct the input. The data critic supplies an additional measure of how closely an observed sequence resembles the learned training distribution. The latent critic contributes to training and is not included in the anomaly score at inference [128].

At inference, each test window is encoded and reconstructed using the fitted model. Discrepancies between the observed and reconstructed sequences provide a reconstruction score (Figure 3.9). The data critic supplies a second score for the observed window. Before combination, the scores require a common orientation, such that larger values indicate greater deviation from the reference behaviour. Their normalisation must also prevent a product of two negative standardised values from being interpreted as strong anomalous evidence. The reported score therefore uses non-negative, consistently oriented normalised components:

$$a_t = \tilde{r}_t \tilde{c}_t. \quad (3.1)$$

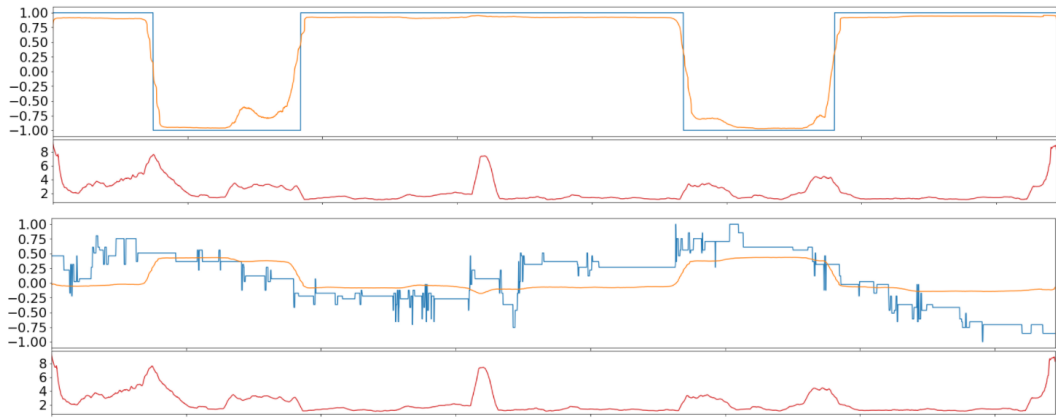


Figure 3.9. Observed signals (blue) and their reconstructions (orange) for two dimensions, reproduced at the original embedded resolution from Figure 4.15 of the internal report. The repeated red trace is the aggregate reconstruction discrepancy across dimensions, shared by the two displayed signals. It precedes the combination of Equation 3.1.

The transformation parameters are estimated from the development data and held fixed during evaluation. This scoring rule combines reconstruction discrepancy with the critic’s assessment of the observed sequence. A high reconstruction discrepancy alone does not establish that an attack occurred, because changes during normal operation may also be difficult to reconstruct. Similarly, a critic score is interpreted through the fitted scoring and thresholding procedure rather than as a calibrated probability of attack.

Overlapping windows produce several reconstructions and critic assessments for some timestamps. For the offline evaluation, reconstructed values are averaged at each corresponding timestamp and variable before calculating the multivariate reconstruction discrepancy. Critic outputs from windows covering the same timestamp are also averaged. Each observation then receives one combined anomaly score and one final prediction. Observations near a sequence boundary are aggregated over the windows available within their own partition. This convention ensures that overlapping windows do not cause a timestamp to be counted repeatedly in the confusion matrix.

An adaptive threshold converts the combined score into a binary anomaly prediction. The threshold rule, its historical context length and any interval-merging parameters are chosen using validation data and fixed before the test cohort is scored. Test ground truth is used only for the final performance calculation. The historical context used by the threshold is a separate parameter from the 60-second window used by the neural networks. Because the stated aggregation uses windows extending beyond the timestamp being scored, the evaluation measures offline detection. It does not measure causal online performance or real-time detection delay.

3.5.2 Input Representations and Settings

The representation comparison includes the complete set of 51 process variables, two principal component analysis (PCA) configurations, an autoencoder representation and a selected

feature subset. The full representation retains the original measurements and actuator states. It provides the broadest input coverage but also exposes the model to redundant relationships and normal operational variation across the process.

PCA represents the measurements through orthogonal combinations of the original variables. The five-component configuration imposes stronger compression, whereas the 24-component configuration retains 99% of the variance in the training data. Retained variance is a property of the fitted representation and does not measure the proportion of attack-relevant information preserved. A low-variance direction may still distinguish an attack from normal operation. The two PCA configurations therefore represent different balances between compression and retention of potentially informative changes.

The autoencoder provides a learned nonlinear representation before GAN processing. Its purpose is to retain relationships that support reconstruction while reducing the dimensionality of the input. The feature-selection branch combines correlation analysis with chi-squared selection, recursive feature elimination, LASSO and Random Forest importance. Supervised selectors use training ground truth only. Any non-negative transformation needed by chi-squared selection is fitted within the training partition, separately from the subsequent scaling used for GAN inputs. The identities of selected variables and the autoencoder bottleneck dimension remain implementation specifications; the aggregate performance results do not identify them.

Imputation and scaling are fitted on the training partition and applied consistently to validation and test observations. PCA and the preprocessing autoencoder are likewise fitted using training data. Features supplied to the GAN are scaled using a training-derived transformation with a reference range of $[-1, 1]$. An observation outside the training range need not fall inside this interval. Automatic clipping is not applied because it could suppress an informative deviation. These transformations preserve the separation between model development and final evaluation.

For a representation with d features, each input window contains 60 consecutive observations and has dimensions $60 \times d$. The window advances by one observation. At the nominal sampling frequency, this corresponds to a 60-second context with a one-second stride. An uninterrupted sequence of 393,679 observations would yield 393,620 complete windows, but its performance denominator remains 393,679 timestamps after aggregation. The 60-second setting is adopted for the reported evaluation; no claim of optimality across other window lengths is made.

The final configuration uses Adam with a learning rate of 0.0005, a latent dimension of 20, a batch size of 64 and 50 training epochs. All representation variants use the same evaluation cohort and counting convention. Their input dimensions vary, so comparison requires explicit control of the architecture and development procedure. A difference between variants cannot automatically be attributed to dimensionality reduction if network capacity or tuning effort also differs.

The reference computing platform comprises an AMD EPYC 7551P processor with 32 cores, 125 GB of RAM and a TITAN X GPU with 12 GB of memory. These specifications describe

Table 3.6. Measured timestamp-level performance of the input representations. Each timestamp is counted once and no point adjustment is applied.

Input representation	Precision (%)	Recall (%)	F_1 (%)	False-positive rate (%)
All 51 variables	70.00	94.50	80.43	6.09
PCA: 5 components	75.00	86.50	80.34	4.34
PCA: 24 components	73.00	92.00	81.41	5.12
Autoencoder representation	72.00	86.50	78.59	5.06
Selected feature subset	71.00	84.00	76.96	5.16

the reported execution environment. They do not establish training duration, inference throughput or memory consumption. Reducing the input dimension from 51 variables to 24 components constitutes a 52.94% reduction in input dimensionality; computational savings would require direct measurement.

3.5.3 Detection Results

Table 3.6 presents the measured performance of the five input configurations. Each row uses the same class totals: 51,453 anomalous timestamps and 342,226 normal timestamps. Precision describes the proportion of predicted anomalous timestamps that fall within recorded attack periods. Recall describes the proportion of actual anomalous timestamps that are detected. F_1 is the harmonic mean of precision and recall, while the false-positive rate is the proportion of normal timestamps incorrectly flagged. All table entries are calculated from integer confusion counts and rounded to two decimal places.

The complete representation has the highest recall, at 94.50%, together with the highest false-positive rate, at 6.09%. This profile represents a detector that identifies a larger proportion of anomalous observations while also flagging more normal observations. The five-component PCA configuration has the highest precision, at 75.00%, and the lowest false-positive rate, at 4.34%, accompanied by lower recall of 86.50%. In these results, stronger compression therefore corresponds to fewer false-positive timestamps and more missed anomalous timestamps.

The 24-component PCA configuration provides an intermediate balance, with precision of 73.00%, recall of 92.00% and an F_1 score of 81.41%. Relative to the complete representation, its precision is 3.00 percentage points higher and its recall is 2.50 percentage points lower. Its F_1 score is approximately 0.98 percentage points higher. It is consequently adopted as the final configuration on the basis of its numerical F_1 balance. This designation does not imply a statistically established advantage or preference under every operational cost model.

The autoencoder representation has a precision of 72.00% and recall of 86.50%, corresponding to an F_1 score of 78.59%. The selected feature subset has precision of 71.00%, recall of 84.00% and F_1 of 76.96%. These values show lower detection coverage for the two configurations. They do not identify a causal mechanism: loss of relevant input information, differences in representation learning and changes in model capacity would require separate investigation.

Table 3.7. Timestamp-level confusion matrix for the final 24-component PCA configuration. Counts refer to individual timestamps after aggregation of overlapping window outputs.

Actual class	Predicted anomalous	Predicted normal	Total
Anomalous	47,337 (TP)	4,116 (FN)	51,453
Normal	17,508 (FP)	324,718 (TN)	342,226
Total	64,845	328,834	393,679

The final 24-component PCA evaluation produced the confusion matrix in Table 3.7: 47,337 of the 51,453 anomalous timestamps are detected and 17,508 of the 342,226 normal timestamps are flagged as anomalous.

The performance measures follow directly from these counts and are calculated from the unrounded values:

$$\begin{aligned}
 \text{Precision} &= TP / (TP + FP) &= 47,337 / 64,845 &= 73.00\%, \\
 \text{Recall} &= TP / (TP + FN) &= 47,337 / 51,453 &= 92.00\%, \\
 F_1 &= 2TP / (2TP + FP + FN) &= 94,674 / 116,298 &= 81.41\%, \\
 \text{FPR} &= FP / (FP + TN) &= 17,508 / 342,226 &= 5.12\%, \\
 \text{Accuracy} &= (TP + TN) / (TP + FN + FP + TN) &= 372,055 / 393,679 &= 94.51\%.
 \end{aligned}$$

The recall of 92.00% means that the detector flags approximately 92 of every 100 anomalous timestamps. It does not mean that 92% of attack episodes are detected. Long attacks contribute more timestamps than short attacks, so the same timestamp-level recall could arise from different distributions of detection across attack episodes. Precision of 73.00% means that approximately 27% of flagged timestamps fall within recorded normal periods. This proportion differs from the false-positive rate of 5.12%, whose denominator includes all normal timestamps.

Accuracy is interpreted alongside the class distribution. Since normal observations account for 86.93% of the cohort, a rule assigning every observation to the normal class would achieve 86.93% accuracy while detecting no anomalous timestamps. The measured accuracy of 94.51% therefore provides supplementary information, whereas recall, precision and F_1 directly describe the balance relevant to identifying the anomalous class.

No point adjustment is applied. A timestamp inside an attack interval counts as a true positive only if that timestamp is predicted anomalous. Detecting one observation does not automatically convert all other observations in the interval into true positives. Contiguous positive predictions may be grouped into alarm intervals for display or operational analysis, but grouping does not retrospectively alter the timestamp-level classifications used in the matrix.

3.5.4 Interpretation and Evaluation Scope

The measured results describe a detector with high coverage of anomalous timestamps and a substantial false-positive burden. At the nominal one-second cadence, 17,508 false-positive observations correspond to approximately 4.86 hours of normal observations being flagged. This duration does not identify the number of alarms presented to an operator. A sustained interval and many isolated detections could contribute the same number of false-positive timestamps while requiring very different responses. Likewise, the 4,116 missed anomalous timestamps correspond to 68.6 minutes of anomalous observations, but their operational significance depends on which attacks and phases of process disruption they represent.

Attack-level assessment requires matching detected intervals to the retained attack episodes. Relevant outcomes include the number of episodes detected, the time between attack onset and the first qualifying detection, and the duration of detection within each episode. A minimum overlap or persistence rule must be specified before those outcomes are calculated. Attacks without a qualifying detection remain missed episodes and cannot be omitted from the interpretation of detection delay. The denominator is the number of attack episodes present in the evaluation cohort, which cannot be inferred solely from the 36 attacks described for the source dataset.

False alarm episodes similarly require an explicit grouping rule and a normal-operation exposure denominator. Their rate can be expressed as the number of distinct false alarm episodes per day of evaluated normal operation. Neither this rate nor the distribution of alarm durations follows from the confusion matrix alone. Timestamp-level precision also depends on the prevalence of anomalous observations: its value in a cohort containing 13.07% anomalous timestamps does not establish the precision expected during deployment with a different attack prevalence.

The comparison between representations specifies a modest difference in F_1 rather than evidence of a universal advantage for PCA. The complete input configuration retains the highest recall, while the five-component PCA representation produces the fewest false-positive timestamps relative to normal observations. The preferred configuration therefore depends on the relative consequences of missed anomalous behaviour and unnecessary alarms. Retention of 99% of training variance provides a description of the 24-component representation, not a guarantee of detection coverage or robustness.

The report gives no run-to-run variability from independently initialised training runs. A repeated-run analysis under the same partition and model-selection procedure should calculate precision, recall, F_1 and false-positive rate for each run, followed by their means and standard deviations. Resampling-based uncertainty estimates must account for temporal dependence and clustering within attack episodes because individual timestamps are not independent.

Further evaluation would separate the contributions of reconstruction scoring, critic scoring and their combination, and examine window length under a common validation procedure. Precision-recall curves would characterise changes across thresholds; their area cannot be derived from one confusion matrix. Training duration, inference throughput and peak memory use would establish computational cost on the stated hardware. The model topology, loss

weights, normalisation rule, threshold parameters, software versions, feature identities and partition records would form the implementation record needed for reproducibility.

Tables 3.6 and 3.7 report the empirical scope of this evaluation. The chronological partition prevents the later test observations from entering training or model selection. The report does not give the division of the first 56,240 attack-period timestamps between training and validation, and incomplete configuration records limit independent reproduction of the experiment.

3.6 Limitations, Validity, and Individual Contribution

Scope of the service-graph evidence. The artefact behind RQ1.1 is a two-page contribution to PAM 2018 on which the candidate is the fourth of four authors. The peer-reviewed anomaly-detection paper from the same group uses clustering rather than a graph, and the short paper that does use a graph is framed as access control [30, 31]. The portable service model was proposed but not implemented. RQ1.1 therefore carries no measured detection result.

Scope of the industrial evaluation. The S7-300 result rests on 25 attack episodes recorded in one plant over 24 hours, and the held-out partition holds a single episode of each class. Records taken from one episode are correlated, so the per-class scores of Table 3.4 summarise five attack events and not 1,233 independent trials. The benign monitoring day was a separate exercise with four active PLCs and executed no attack, so it bounds the false-alarm rate and says nothing about recall. No experiment reported here trains on one cell and tests for attacks on another, so transfer between installations is untested. The importances in Figure 3.4 are impurity decreases on the training partition, so they describe what this forest used and do not show which fields detection requires. Both exercises were labelled by the team that executed the attacks, and each two-hour baseline covers only the operating modes that occurred within it.

Sampling and selection. The GAN study develops the model on the first 56,240 attack-period timestamps and evaluates it on the later 393,679 timestamps. Training and validation also use separate normal-operation data, and attack episodes do not cross partition boundaries. This chronological split prevents test observations from entering training or model selection, but it remains within one SWaT campaign. The S7-300 evaluation holds out whole capture sessions. Neither study tests transfer to another installation or an operational class ratio. No repeated runs or uncertainty intervals are reported, so small differences between configurations cannot be interpreted as stable improvements.

Simple baselines. On SWaT, simple tests on process values have matched much more elaborate detectors under a common protocol [103]. Reimplementing published intrusion detectors in one framework has likewise left simple baselines competitive [121]. No such baseline is reported for the GAN study or for the S7-300 evaluation, so neither result shows that its model outperforms a simple rule on the same data.

Individual contribution. The candidate designed and implemented NADICS, its model interface, and the S7 and PROFINET extractor. He produced the labelled S7-300 attack corpus. For the process study, he designed and implemented the multivariate GAN pipeline, its preprocessing variants, dynamic architecture, training, scoring, and evaluation on SWaT. He analysed the results and wrote the internal report. His contribution to the service-graph study was the concept and the measurement design. The IUNO deliverables attribute authorship at document level [33, 34], so this account rests on the candidate’s attestation and on the surviving artefacts rather than on passage-level attribution.

3.7 Chapter Summary

RQ1.1 asks how service relationships can be represented as communication graphs. The reported design stores observed service relations as a node-local whitelist and permits a site owner to revise it as the deployment changes. Its memory and latency costs were measured. Detection quality was not.

RQ1.2 asks how a modular framework can analyse network and process features for anomaly detection in industrial systems. NADICS separates dataset layout, preprocessing, and learners behind fixed interfaces. On the S7-300 corpus recorded inside IUNO the framework reaches 94.50 per cent attack F_1 and 89.56 per cent macro- F_1 over six classes at a record false-positive rate of 0.484 per cent, and a benign day with four active PLCs raised nine alarm episodes, all of them false. For SWaT process windows, the GAN evaluation reports 92.00 per cent timestamp recall at a false-positive rate of 5.12 per cent with 24 PCA components. The full input gives the highest recall and five components the lowest false-positive rate, so the preferred representation depends on the relative cost of missed anomalies and false alarms.

Together, these mechanisms close gap G1 at the implementation level. No study shows transfer across time, sites, or adaptive attacks, which motivates Chapters 6 and 7.

Multimodal Android Malware Analysis with Hybroid

4.1 Research Problem and Contribution

This chapter answers RQ2, stated in Section 1.3: To what extent does fusing static code-graph representations with dynamic network-flow features improve Android malware detection and category classification compared with either modality alone? It takes up gap G2, which records that hybrid systems are rarely measured against each modality on its own, so that the gain belonging to the combination cannot be separated from the gain belonging to either part [25, 57]. The question is comparative, and the shape of the chapter follows from that. An accuracy figure for the combined system answers nothing on its own. What answers RQ2 is the difference between the combined system and each of its halves, measured on the same samples under one protocol, and Section 4.11 reports it.

Hybroid observes an application twice. It recovers the program structure from the application package without running it, and it observes the traffic the application produces while running on a physical handset. Both observations are reduced to vectors, the two vectors form one input, and a supervised classifier decides whether the application is malicious and, if it is, which category it belongs to. Figure 4.1 gives the arrangement. The design is described in Sections 4.5 to 4.8 and the evaluation in Sections 4.9 to 4.11.

Three claims are made for the work, and each is stated here in the form the evidence supports rather than in the form the source publication used. The first is the flow pipeline: network traffic captured from a real device is reduced to a fixed feature vector per application, through bidirectional flow records, aggregation, scaling and a selection stage that leaves 13 features. The second is the program-code pipeline: instruction embeddings learned from a large opcode corpus are composed into method-level vectors, which carry an application-level function-call graph that a graph embedding converts into a single vector. The third is the comparison itself, which measures each modality alone and the two together, under one split, for detection and for category classification. The third depends on the first two and is what Section 4.11 rests on.

Two limits on those claims belong here rather than in the limitations section, because they qualify what is being claimed rather than what was found. The source publication names three components, `opcode2vec`, `function2vec` and `graph2vec`, of which the second is never defined anywhere in it and the third names a published method that the work does not use. This chapter therefore describes the three steps and does not name them. And the comparison in Section 4.9 is against four reimplemented systems of unequal weight, which that section states when it reports the numbers.

Hybroid: hybrid analysis of program code and network traffic

Android malware detection and categorization

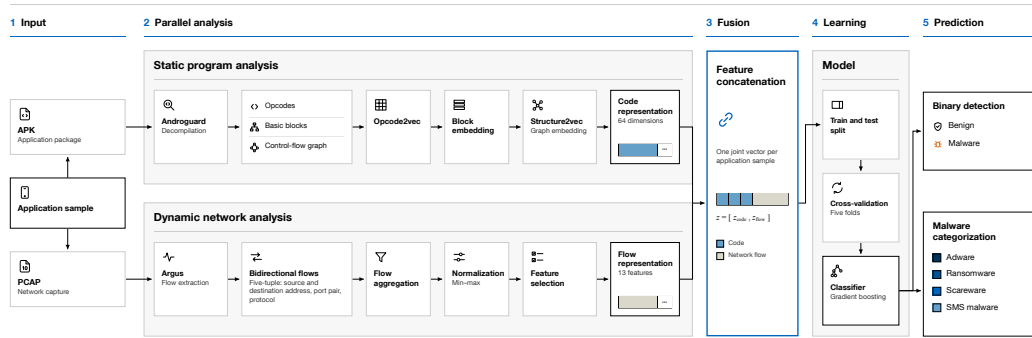


Figure 4.1. The Hybroid architecture. Every application sample provides a pair of inputs: the application package and a capture of the traffic it generates (1). Both are analysed in parallel (2). Androguard decompiles the package into opcodes, basic blocks and a control-flow graph, which are embedded into a 64-dimensional code representation, while Argus turns the capture into bidirectional flows that are aggregated, normalised and reduced to 13 network-flow features. The two representations are concatenated into one joint vector per sample (3), on which a gradient-boosting classifier is trained and cross-validated (4) for binary detection and for categorisation into four malware categories (5). Redrawn by the candidate from the architecture published in [35].

The work was published at ISC 2021 [35], with the candidate first of four authors and the paper recording that the first two share first authorship. The candidate’s own contribution is stated in Section 1.5, which the affidavit covers: the pipeline as a whole and the network-traffic analysis within it are his, as is the framework through which the dataset was analysed, and he carried out the experiments and wrote the paper. His contribution to the treatment of the graph data is partial, and this chapter does not present the program-code pipeline as his own design.

4.2 Related Work

Section 2.1.2 distinguishes static, dynamic and hybrid analysis and reports that roughly one work in ten among surveyed deep-learning defences used both [3]. Section 2.2.1 defines the graph objects. Neither is repeated here. What follows places Hybroid among the systems it was measured against and among the work that has appeared since.

Static detectors read the application package. Features are drawn from the manifest and from the code, and the families in use by 2021 were permissions and intents [129, 130], API calls [131, 132], and structures recovered from the bytecode [64, 133, 134]. DREBIN is the reference point among them and one of the baselines in Section 4.9: it gathers as many such features as it can, from permissions and hardware requests through API calls to network addresses found in the code, and embeds them in one vector space in which a linear model can be inspected to say which features drove a verdict [1]. Its breadth is also where it is exposed, since the manifest entries and the identifiers a static analyser reads can be rewritten without altering what the application does [135], which Section 2.1.3 treats in general terms.

A second static family works below the level of identifiers, on the instruction stream itself. Sequences of opcodes have been used directly as features, with the effectiveness of n -grams over them measured against families of malicious applications [136], and convolutional networks have been trained over raw opcode sequences to learn the relevant patterns rather than to enumerate them in advance [137]. That line is the immediate precursor of the representation in Section 4.5, and the difference is where the sequence stops. A model over a flat opcode sequence sees the program as one string. The representation used here keeps the opcode alphabet and then organises it, first into methods and then into the graph of calls between them.

Graph-based detectors use the structure rather than the identifiers. Adagio hashes neighbourhood subgraphs of the function-call graph into an explicit feature map, which lets a linear classifier be trained over graph structure at low cost and lets a verdict be traced back to the code structures that produced it; it reports detection of 89 per cent of its malware [64]. It is the second static baseline in Section 4.9. apk2vec learns application embeddings from several graph views at once [138]. MaMaDroid takes a different route through the same object, abstracting each call target to its package or family before modelling transitions between them, which buys stability against the appearance of unseen methods at the cost of the callee's identity [2].

All of these build the object Hybroid builds, and what separates them is how a vertex is described. Adagio's vertex labels are hand-built from categories of instruction, and so are the attributed control-flow graphs used for similarity between binaries [66, 139]. Hybroid learns the labels instead [140]. That choice has since acquired support it did not have at the time: a benchmark study of graph-based Android classifiers reports that they exceed 94 per cent accuracy on standard corpora yet lose up to 45 per cent under distribution shift on unseen variants of known families, and it names structure-only function-call graphs as an inherent limitation of those benchmarks, because topology alone misses the semantic patterns that generalisation needs [141]. That study is a preprint under review. For the wider design space of graph representation learning applied to malware, one survey covers the extraction and architecture choices in full [8].

Dynamic detectors observe behaviour. Two branches exist, one reading system-level activity such as system calls and API usage, the other reading what crosses the network. Hybroid belongs to the second, which is cheaper to collect than system-call tracing and which exposes the part of an application's behaviour that a remote operator depends on, since exfiltration and command traffic have to leave the device to be of use [142, 129]. Work in this branch has ranged from rule-based inspection of what an application transmits [142] to learned models over traffic representations [143, 144]. CICAndMal2017 was built to support it, and its authors classify Android malware from flow features alone [145], which makes their system the dynamic baseline in Section 4.9; they later added API calls to the same pipeline [146].

Granularity is the choice that divides this branch, and it runs from raw packets, which deep models have been trained on directly [144], through individual flows to whole sessions [143]. The choice matters more than it appears to, because it fixes what a label can attach to. Section 4.7 returns to it.

Hybrid systems combine the two, and a systematic review of that literature is available and replaces an enumeration here [147]. Two points from the surrounding literature bear directly on this chapter. Measured under one protocol, static analysis has proved at least as effective as dynamic analysis and the combination has measured best [25]; that comparison has rarely been repeated, which is gap G2. And the modality pair used here was taken up again four years later by HGDetecter, which builds a static function-call graph and a network-behaviour graph from the traffic, fuses them through a graph embedding, and evaluates on the same corpus [148]. Its result is conditional, and the condition is the useful part: the gain from fusion is reported as roughly 4 per cent where the traffic features already capture an application's behaviour, and 21 to 26 per cent where they do not. Section 4.11 returns to that.

4.3 Observation and Threat Model

What a detector can see bounds what it can claim, and Chapters 4 and 5 differ in exactly that respect. This section fixes the observation model so that the two are not read as competing designs.

Hybroid observes two things. The first is the Dalvik bytecode recovered from the application package by a static analyser, from which methods, their instructions and the calls between them are taken. The second is the traffic the application produces while running on a physical handset, captured as packets and reduced to bidirectional flow records. Both observations are made once, offline, before any verdict is given.

Several things are not observed. System calls are outside the model, as is code loaded at runtime, and so is native code reached through the Java Native Interface. The contents of encrypted payloads are also outside it, and that limit is measured rather than assumed: across the four malware categories, the share of flows using TLS runs from 0.00 to 10.89 per cent, which Section 4.7 reports and Section 4.12 takes up as a threat to validity. The position in which the system is deployed follows from this. It analyses a sample together with a capture of its traffic, which places it in an analysis pipeline rather than on the handset.

The adversary is non-adaptive. The malware predates the system by several years and was not written against this detector, no attacker knowledge is modelled, and no evasion attempt is made against the trained model. Grey box in the sense fixed in Section 2.5 is therefore not claimed here, since that term denotes a level of attacker knowledge and this evaluation models none. Section 4.12 states what follows from that, and what the attacks published since 2025 imply for a pipeline built over a function-call graph.

4.4 Dataset and Experimental Protocol

The evaluation uses CICAndMal2017 [145], which supplies application packages together with the traffic each one produced. Benign applications were collected from Google Play across 2015 to 2017 and malicious ones from VirusTotal and the Contagio archive. The traffic was captured from three physical handsets rather than from an emulator, with scripted

Table 4.1. CICAndMal2017 source-corpus composition. The category counts are recomputed from the dataset family listing and differ from the published values of 124, 112, 109 and 101, which do not sum to the stated total of 426.

Class	Families	Samples	Share
Benign		1,700	80.0 %
Adware	10	104	4.9 %
Ransomware	10	101	4.8 %
Scareware	11	112	5.3 %
SMS malware	11	109	5.1 %
Malware	42	426	20.0 %
Total		2,126	100.0 %

user behaviour and live accounts on messaging and social platforms, and in three states: immediately after installation, before a reboot, and after it. Real devices are the reason this corpus was chosen, since emulator captures do not reproduce the network behaviour of a handset in use.

Table 4.1 gives the source-corpus composition. The published category rows sum to 446 against the stated 426, and their percentages sum to 101.0. The recount closes on 426 across 42 families. The 47 exclusions are not identified by class. The dataset documentation reports 5,065 benign applications installed, and the 1,700 used here is the number of benign captures the distribution provides.

The source corpus pairs 2,126 applications with their traffic. Both analyses excluded the 47 graph-extraction failures and used the remaining 2,079 applications. Model selection and evaluation use five-fold cross-validation with the results averaged over the folds. Implementation used Python with scikit-learn, TensorFlow and Keras, on a server with 32 cores, 128 GB of memory and a 16 GB accelerator. Accuracy, precision, recall, the F_1 score and the area under the ROC curve are reported, with F_1 preferred under the class imbalance Table 4.1 shows. The averaging convention behind those metrics is not stated in the source, which Section 4.9 returns to.

A second and larger experiment exercised the program-code pipeline alone. It used 45,592 malicious and 90,313 benign applications drawn from AndroZoo, VirusShare and VirusTotal, which is 135,905 samples at a malicious share of 33.5 per cent, split at random into 80 per cent for training and 20 per cent for testing, and it reached an accuracy of 95.0 per cent with an F_1 score of 96.0 per cent. It is reported here as a check that the program-code pipeline scales beyond a corpus of a few thousand samples. It is not a time-aware evaluation, and Section 2.6 sets out why that distinction matters [12].

4.5 Opcode and Method-Level Representation

Dalvik instructions carry an operation and its operands. Only the operation is embedded here, and the reason is that the operands are register references whose values depend on the

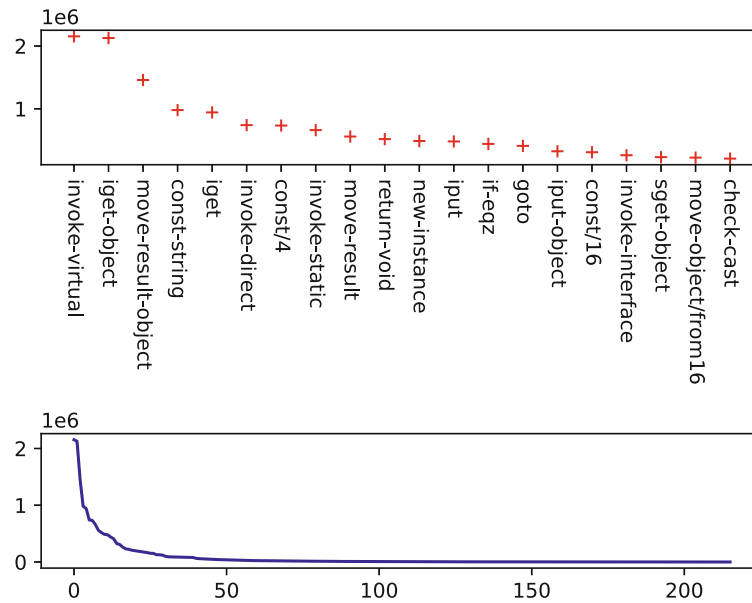


Figure 4.2. Opcode frequency across the pre-training corpus. The upper panel names the twenty most common operations, and the lower panel plots all 216 in rank order, where the shape of the decay is the property the embedding relies on. Reproduced from the candidate’s own publication [35], as is Figure 4.5.

runtime that executes them [45, 46], so they cannot be enumerated across applications in the way a vocabulary must be. The operation, by contrast, is drawn from a fixed alphabet of 216 opcodes, and samples within one malware family that share a malicious pattern share the operations that implement it.

The embedding is learned rather than assigned. Each opcode is mapped to a real-valued vector by the skip-gram formulation of word2vec, in which the model is trained to predict the operations surrounding a given one [149]. Training used a corpus of 18,240,542 opcodes extracted from applications drawn from AndroZoo. Two opcodes that occur in similar positions therefore receive similar vectors, so the representation encodes the company an operation keeps rather than an arbitrary index assigned to it, which is what a one-hot encoding over the same alphabet would give.

Borrowing a technique from language modelling needs a reason beyond convenience, and the distribution of the alphabet supplies it. Counting opcodes across the corpus shows the twenty most frequent following a power law, as Figure 4.2 shows, which is the property that holds for words in natural language, which is the setting the skip-gram objective was built for [149]. The alphabet also behaves better here than in the setting the technique came from. Natural language has an open vocabulary and a model trained on one corpus meets unseen words in another; the Dalvik instruction set is closed at 216 operations for the format the corpus was drawn from, so an embedding trained once covers the applications the analyser reads, and the unseen-token case that troubles language models does not arise in the same way. This is what makes a transductive method acceptable in a detector that must generalise, and Section 4.6 returns to the point.

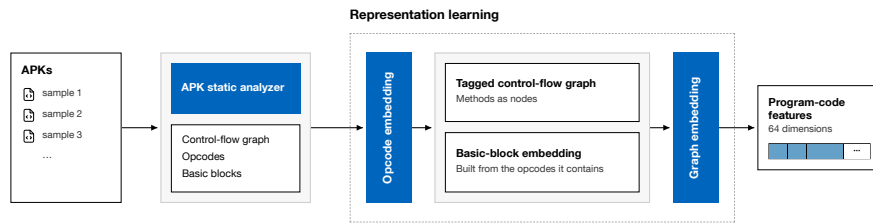


Figure 4.3. The program-code branch. The static analyzer extracts the control-flow graph, opcodes and basic blocks from each APK. Opcodes are embedded first, every method is represented from the tagged control-flow graph of the basic blocks it contains, and a graph embedding reduces the resulting function-call graph to a single 64-dimensional feature vector. Redrawn by the candidate from the arrangement published in [35].

Discarding the operands has a cost, and it is specific rather than general. Every call instruction receives the same vector whatever it invokes, because the target sits in the operand field [150]. This is worth stating plainly at the point where the representation is built, because the object built next is a graph whose edges are exactly those call targets. The vertex features and the edge set are therefore recovered by two passes over the bytecode that share no information, and the vertex features are blind to the structure that the edges record.

Above the instruction, a method is represented by combining the vectors of the instructions it contains. The source publication calls this a basic-block embedding and gives it as a weighted mean, $\tilde{f} = \sum_i w_i x_i / \sum_i w_i$, where x_i is the l -dimensional embedding of the i -th opcode. Two things about that definition are carried forward as they stand rather than tidied. The weights w_i are not specified anywhere in the source, which leaves the mean undefined. And the formula is written over the n opcodes of a function while the prose around it calls the result the embedding of a basic block, so the two do not describe the same object. Section 4.12 records both.

What the vector is used for is stated unambiguously, and that is what Section 4.6 needs. It becomes the feature attached to a vertex of the method’s control-flow graph, whose vertices are basic blocks.

4.6 Program-Graph Representation

Section 2.2.1 defines the control-flow graph, the function-call graph, and the composition in which per-method embeddings become the vertex features of a call graph. Hybroid builds the composition. Control-flow graphs are extracted per method with Androguard [51], each one is reduced to a single vector, and those vectors describe the vertices of the application’s function-call graph, which is the object the classifier consumes. Both terms are needed, and they are not interchangeable.

Figure 4.3 sets out the branch as a whole. Two properties of that object bound what any detector built on it can claim, and both are set out in Section 2.2.1. A call graph recovered

without running the program over-approximates, because the analysis admits edges that no execution takes. It also under-approximates, since calls made through reflection, through code loaded at runtime or through native libraries leave no edge for a static analyser to find, and Section 4.3 records that all three are outside what is observed here. The object is in that sense an artefact of the analyser rather than a property of the program, since which edges are found depends on the toolkit and on how the component lifecycle is modelled. Neither property is peculiar to this work. Both mean that the structure being classified is a static estimate of the program's shape, and Section 4.12 states what follows when an adversary is allowed to influence that estimate.

Reducing the graph to a vector could be done by counting substructures, which is what the kernel methods discussed in Section 4.2 do [64]. The route taken here instead propagates information along the edges and lets the features of a vertex be shaped by its neighbourhood.

The reduction from a graph to a vector uses `structure2vec` [65]. A p -dimensional vector μ_v is attached to each vertex v and initialised at random, then updated for T rounds by a function of the vertex's own features and the sum of its neighbours' current vectors,

$$\mu_v^{(t+1)} = F\left(x_v, \sum_{u \in N(v)} \mu_u^{(t)}\right), \quad \forall v \in V, \quad (4.1)$$

with F realised as

$$F\left(x_v, \sum_{u \in N(v)} \mu_u^{(t)}\right) = \tanh\left(W_1 x_v + \sigma\left(\sum_{u \in N(v)} \mu_u^{(t)}\right)\right), \quad (4.2)$$

where W_1 is trainable and σ is an n -layer fully connected network,

$$\sigma(l) = P_1 \text{ReLU}(P_2 \cdots \text{ReLU}(P_n l)). \quad (4.3)$$

After T rounds the vertex vectors are averaged and transformed by a second trainable matrix, which yields one vector per graph. This is the operator Section 2.3.1 describes in words, instantiated on the graphs built here. The number of rounds fixes how far information travels: after T updates a vertex has been reached by everything within T steps of it, so T sets the size of the neighbourhood any single vertex vector summarises. Averaging at the end rather than summing keeps the result comparable across graphs of different sizes, which matters when applications differ by orders of magnitude in how many methods they contain.

One step in this pipeline is not specified in the source publication, and the chapter states so rather than supplying a plausible reconstruction. Equations 4.1 to 4.3 are applied at the level of a method's control-flow graph, and the averaging inside that procedure pools the basic blocks of one method. The step that combines the resulting per-method vectors into the single 64-dimensional vector standing for the whole application is named as the third step of the pipeline and is specified by no equation. Nothing in the source refers to a caller or a callee in symbols, so the composition is stated there as a design and is reproduced here as one.

A note on transduction closes the description. The opcode embedding is transductive, since `word2vec` needs the whole alphabet at training time. That alphabet is the fixed Dalvik

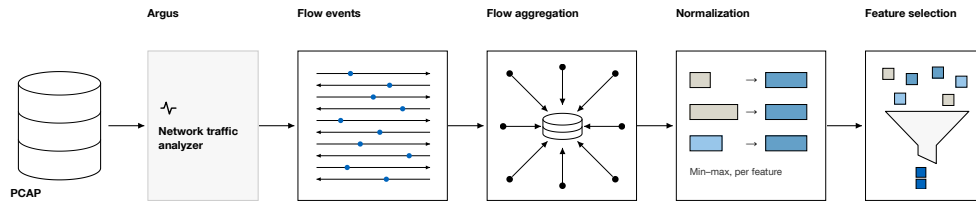


Figure 4.4. The network branch. Argus reduces a raw capture to bidirectional flow events, the events belonging to one application are aggregated into a single record, every attribute is normalised to a common range, and feature selection retains the 13 most informative features. Redrawn by the candidate from the arrangement published in [35].

instruction set, so the requirement is met by construction rather than by assumption about the corpus. The graph embedding is inductive, and graphs in the test partition are unseen when it is trained.

4.7 Network-Flow Representation

Section 2.4.1 sets out the flow abstraction. Figure 4.4 gives the branch. Here the captures are processed with Argus [151], which produces bidirectional records grouped by the usual five-tuple of source and destination address, source and destination port, and protocol, and reports roughly forty statistics for each record. Two of those statistics are categorical, the direction of the transaction and the state flags, and both are one-hot encoded; the rest are numeric.

An application produces many flows, and the representation reduces them to one record. The values of each feature are averaged across all flow events belonging to one capture, which yields a single feature vector per application and matches the granularity of the program-code vector so that the two can be combined. Two consequences follow and neither is stated in the source publication. The label attaches to the application rather than to any individual flow, which avoids the assumption that every flow emitted by a malicious application is itself malicious. Against that, averaging discards the order and the timing of the capture, and a small number of malicious flows is averaged together with whatever background traffic the handset produced during the same window.

The averaged features are scaled to the unit interval by min-max normalisation, on the ground that features whose ranges differ by orders of magnitude would otherwise contribute unequally to any distance computed over them. That rationale applies to the two learners which were tried and dropped rather than to the three reported in Section 4.8, since a tree is unaffected by any monotone rescaling of a feature. The source publication states that standard scaling was considered and rejected.

Selection reduced the roughly forty statistics to the 13 listed in Table 4.2, using Pearson correlation, an extremely randomised trees estimator and a univariate test that keeps the

Table 4.2. The 13 network-flow features retained after selection, grouped by the kind of quantity each one measures.

Kind	Features
Volume and duration	mean record duration, total bytes, source bytes
Transport state	destination TCP window, source loss and retransmission flag, destination out-of-order flag, transaction direction
Path and service	source and destination TTL, source and destination TOS byte
Connection identity	source and destination TCP base sequence number

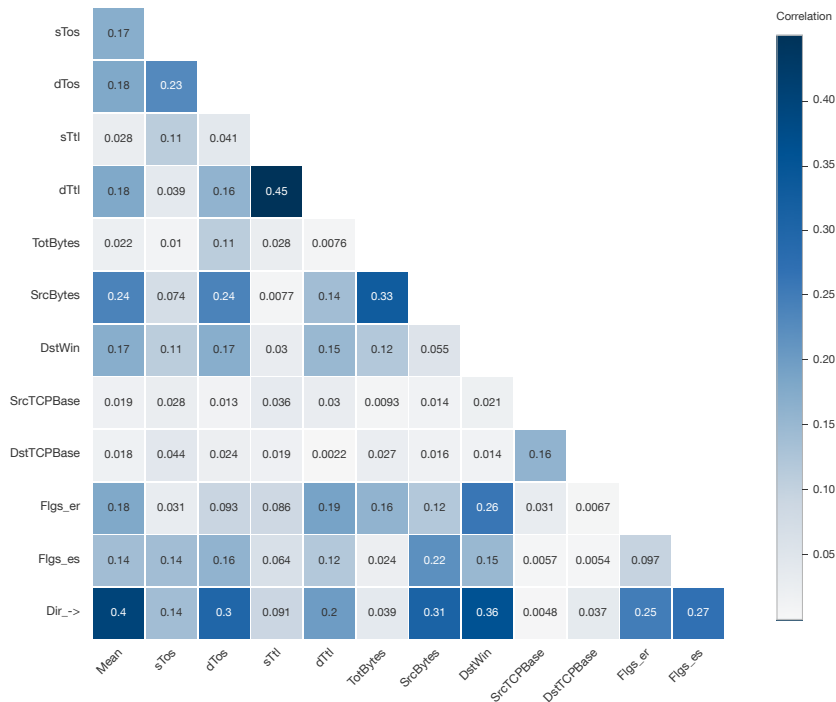


Figure 4.5. Kendall correlation among the retained flow features, used to check that the selected features are not redundant with one another.

highest-scoring features. Redundancy among the survivors was checked with Kendall’s correlation, shown in Figure 4.5. The grouping in the table is this chapter’s addition, and it is there because the four groups do not have equal standing as descriptions of application behaviour. Volume, duration and transport state are produced by what the application does. The remaining two groups are not: the TTL reflects the path and the operating system of the endpoint, the TOS byte is set by few stacks and is commonly zero, and the TCP base sequence numbers are required to be unpredictable per connection and so cannot carry behaviour at all. Section 4.12 takes this up, together with a published audit of the same feature family in a different corpus.

A separate observation concerns what the traffic carried, and Table 4.3 records it. Cleartext HTTP accounts for between 29.22 and 61.38 per cent of flows across the four malware categories, and TLS for between nothing at all and 10.89 per cent. The remainder, which runs from 27.73 to 70.78 per cent depending on the category, is neither named nor characterised in the source publication.

Table 4.3. Share of flows by protocol, per malware category. The two columns do not sum to 100 per cent and the source publication does not say what the remainder is.

Category	HTTP	TLS	Unaccounted
Adware	52.00 %	8.00 %	40.00 %
Ransomware	29.22 %	0.00 %	70.78 %
Scareware	61.38 %	10.89 %	27.73 %
SMS malware	52.20 %	10.28 %	37.52 %

Flow-level features were chosen over inspection of payloads for reasons of privacy and cost, and that choice has become the only available one since: beyond transport encryption, a substantial share of applications encrypt content themselves before sending it, with one measurement of 12,598 applications finding 22.92 per cent doing so and most of those transmitting device-identifying information [152].

4.8 Multimodal Fusion and Classification

Two classifiers appear in this pipeline and they do different work, which is worth separating before either is described, because a reader who meets both without warning will take one of them for a mistake.

The first sits at the end of the program-code branch. A two-layer fully connected network takes the graph vector and is trained to discriminate, with malicious samples labelled +1, benign samples -1, and a decision taken at zero. Its purpose is to fit the parameters of the representation, since the matrices in Equations 4.1 to 4.3 have to be learned against some objective and detection is the objective available. What leaves the branch is the 64-dimensional vector, and the network that produced it plays no further part.

One consequence deserves stating, because it bears on Section 4.10. The objective that fits the program-code representation is binary, and the same representation is then used for the five-class task. A representation tuned to separate malicious from benign need not be equally informative about which kind of malicious an application is, and nothing in the source publication indicates that the embedding was refitted for the second task. Whether the categorisation results would improve under a representation fitted to that objective was not measured.

The second classifier acts on both modalities. The 64-dimensional program-code vector and the 13 flow features are taken together as one input, on which a supervised learner is trained for detection and, separately, for category classification. The source publication describes the two groups as forming a single input vector and does not name the operator that combines them, so this chapter does not name it either. The proportions of that input are however fixed by the two branch designs and they are uneven: 64 of the 77 dimensions come from the program code, so the static half supplies roughly five dimensions in every six. No weighting is applied to correct for this, and none is reported as having been considered.

Table 4.4. Detection compared with four reimplemented systems, read from the published figures. The four systems differ considerably in weight: DREBIN is a full conference paper, Adagio and CIC 2017 are papers of ten and seven pages, and the SVM entry is a two-page poster. The reported performance values come from Figures 6 and 7 of [35].

System	Accuracy	F_1	Recall	Precision	AUC
SVM [153]	0.939	0.860	0.856	0.865	0.876
DREBIN [1]	0.954	0.867	0.873	0.861	0.961
Adagio [64]	0.893	0.932	0.913	0.953	0.867
CIC 2017 [145]	0.876	0.872	0.878	0.871	0.914
Hybroid	0.970	0.969	0.970	0.970	0.996

Five learners were tried. Support vector machines and naive Bayes performed worst and were dropped, leaving a decision tree, a random forest and a gradient boosting classifier, which are the three reported throughout Sections 4.9 to 4.11. No figures are given for the two that were dropped, so the margin by which they were rejected is not on record. The three that remain are tree-based learners of increasing capacity, which is convenient for the comparison in Section 4.11: the decision tree fits axis-aligned splits on a single model, the random forest averages over many such models, and gradient boosting fits them in sequence against the residual of what came before. Reading the three together therefore says something about how much capacity the fused representation needs, and not only about which learner wins.

Combining the modalities at the level of features, rather than training a model on each and combining their decisions, is a design choice, and it was the ordinary one at the time. Its consequences were not visible in 2021. Section 4.11 reports what the measurements here show and sets them beside the evidence that has accumulated since.

4.9 Binary Detection Results

Table 4.4 reports the comparison published for binary detection [35]. Hybroid has the highest point value in each column. Two qualifications accompany the comparison because they affect how the values can be read.

The first concerns the comparison set. The four systems are not of equal weight, and the SVM entry is a two-page poster [153], so the phrase state of the art would carry more than the table supports.

The second qualification concerns the metrics. For a binary problem in which accuracy is computed over the whole test set and precision and recall over the malicious class, the three quantities determine the malicious share p of that set through $\text{acc} = 1 - p[(1 - R) + R(1 - P)/P]$. Solving for each row of Table 4.4 gives 22.0 per cent for the SVM entry, 17.2 for DREBIN, and 81.0, 49.2 and 50.0 per cent for Adagio, CIC 2017 and Hybroid. Depending on the labels of the 47 exclusions, the evaluated malware share lies between 18.2 and 20.5 per cent. The last three values remain incompatible with that interval. The source publication does not state which averaging convention produced its figures, and no raw result survives against

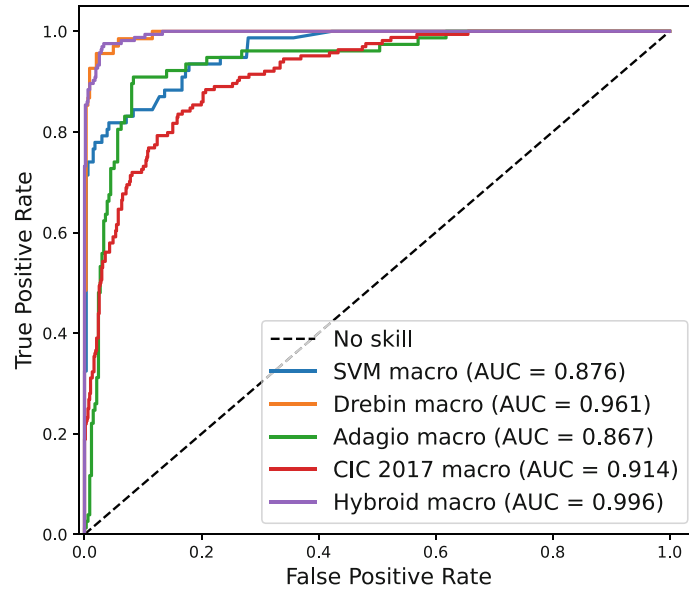


Figure 4.6. Receiver operating characteristic curves for binary malware detection, reproduced from the published vector content of Figure 7 in [35]. The source labels the curves as macro averages and reports the AUC values in the legend. It provides neither a selected operating threshold nor fold-level variation.

which to check. The chapter therefore preserves the published values and does not treat their differences as estimates made under one documented metric convention.

Figure 4.6 shows the corresponding threshold curves. Hybroid has the largest reported AUC at 0.996, followed by DREBIN at 0.961. The dynamic CIC 2017 system has an AUC of 0.914, ahead of the SVM and Adagio entries despite its lower accuracy in Table 4.4. The metric therefore changes the ordering of the baselines. The AUCs summarise ranking across thresholds, while the paper reports no threshold at which the detector would operate. It also supplies no fold-level curves or confidence intervals, so the differences remain descriptive point estimates [35].

Read by modality, three of the four baselines are static and one is dynamic. The comparison still mixes the implementation choices of separate systems with the observation modality, so it cannot isolate a modality effect. Section 4.11 uses the within-system comparison for that purpose.

Figure 4.7 separates the learner comparison from the external baselines. Gradient boosting on the fused representation reaches 0.97 on all three reported metrics. Random forest reaches 0.96, while the decision tree reaches 0.91. Network flow alone has a higher point value than program code alone for each learner. Under program code alone, random forest and gradient boosting both report 0.93 for F_1 , recall and precision. Gradient boosting therefore has the highest fused result, as the paper states, while its program-code result ties with random forest at the published precision [35].

At two-decimal precision, F_1 , recall and precision coincide in eight of the nine configurations. The remaining configuration is gradient boosting over network flow, for which recall is 0.96

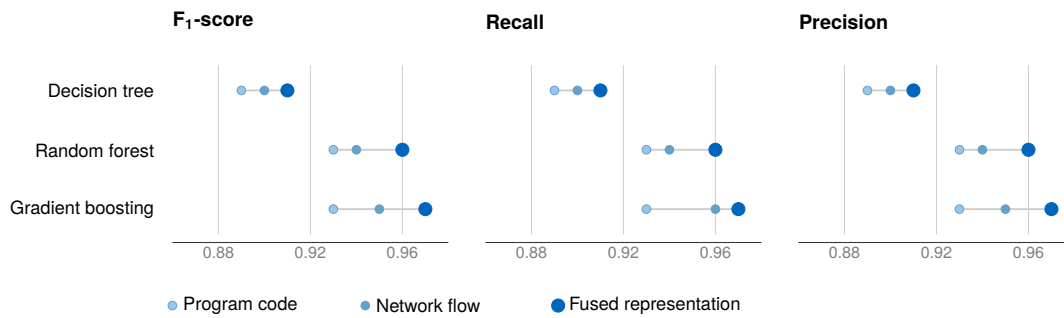


Figure 4.7. Binary detection by learner and observation modality, with each reported metric in its own panel. Every row carries one marker per modality, joined by a rule spanning the values it reports. The values are the five-fold means printed to two decimal places in Figure 8 of [35], checked against the bar geometry of that figure. The score axis is truncated to 0.86–0.98, the interval those values occupy. No fold-level scores are published, so the figure carries no error bars. Redrawn by the candidate.

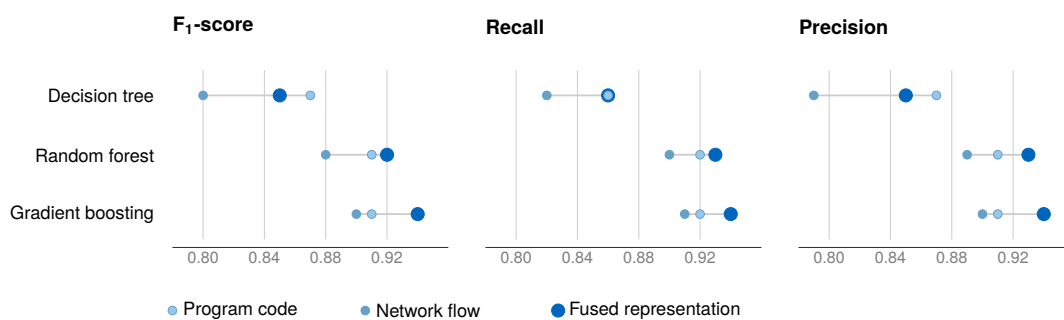


Figure 4.8. Five-class malware-category classification by learner and observation modality, drawn on the same plan as Figure 4.7. The score axis is truncated to 0.78–0.96, because this task occupies a lower interval. The values are the five-fold means printed to two decimal places in Figure 9 of [35], checked against the bar geometry of that figure. Decision-tree recall is 0.86 for program code and for the fused representation, so those two markers are drawn concentrically. No fold-level scores are published, so the figure carries no error bars. Redrawn by the candidate.

and the other two values are 0.95. Micro-averaging would make the three metrics equal by construction, and rounding could also hide small differences. The paper does not state the averaging convention and provides no raw predictions, so neither explanation can be selected from the surviving evidence [35].

4.10 Malware-Category Classification

The second task assigns each sample to one of five mutually exclusive classes, the four malware categories of Table 4.1 together with the benign class. The source publication calls this multi-label classification throughout; it is multi-class classification with a single label per sample, and this chapter uses the latter term [35].

Figure 4.8 gives the category-classification results. Gradient boosting on the fused representation reaches 0.94 on all three metrics. Random forest follows at 0.92 for F_1 and 0.93 for recall and precision. The decision-tree result differs: the fused representation reaches 0.85 for F_1 and precision, below the program-code values of 0.87, while recall is equal at 0.86. The

Table 4.5. Area under the ROC curve per class, gradient boosting on the combined representation, with the pre-exclusion counts of Table 4.1. Values are read from Figure 10 of [35].

Class	Samples	AUC
Benign	1,700	0.995
Ransomware	101	0.991
Scareware	112	0.979
Adware	104	0.963
SMS malware	109	0.946
Macro average		0.976

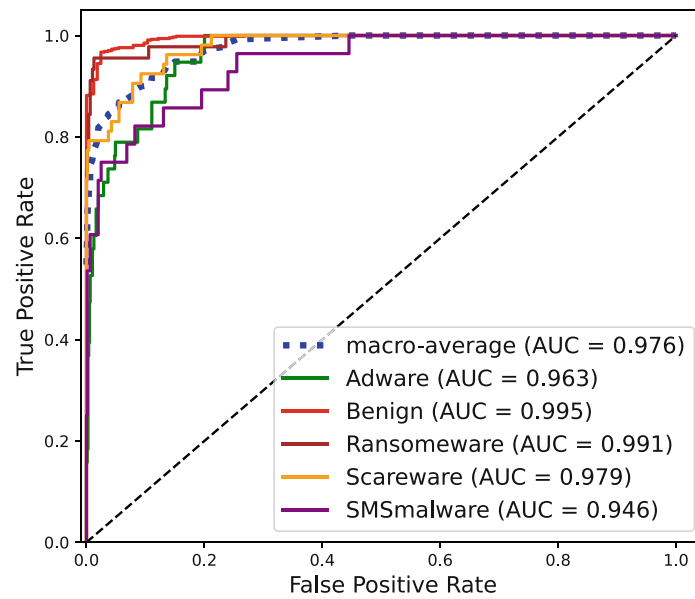


Figure 4.9. Per-class receiver operating characteristic curves for gradient boosting on the fused representation, reproduced from the published vector content of Figure 10 in [35]. The legend reports a macro-average AUC of 0.976 and the AUC of each class. The paper provides no selected operating threshold or fold-level variation.

paper’s general statement that combined features give the best performance therefore has one exception in its own reported figure [35].

Table 4.5 gives the area under the ROC curve for each class, for the gradient boosting classifier on the combined vector. The macro average of 0.976 is reported as a macro accuracy in the source publication, which its own figure legend contradicts; it is an average of areas under the curve [35].

Figure 4.9 preserves the threshold curves from which the AUC values in Table 4.5 were read. Benign and ransomware have the largest reported areas, 0.995 and 0.991. SMS malware has the smallest at 0.946. These values measure class ranking across thresholds. They do not provide the per-class precision and recall at the decision rule used for the five-class result, which the paper does not report [35].

Table 4.6. Difference between the fused score and the better of the two single-modality scores. Positive values favour fusion. The differences are computed from Figures 8 and 9 of [35], whose values are printed to two decimal places.

Task	Learner	ΔF_1	Δ recall	Δ precision
Detection	Decision tree	+0.01	+0.01	+0.01
Detection	Random forest	+0.02	+0.02	+0.02
Detection	Gradient boosting	+0.02	+0.01	+0.02
Categorisation	Decision tree	-0.02	0.00	-0.02
Categorisation	Random forest	+0.01	+0.01	+0.02
Categorisation	Gradient boosting	+0.03	+0.02	+0.03

The averaging convention for the global precision, recall and F_1 values is also unstated. Benign applications remain the majority after the exclusions, so the reported value of 0.94 cannot be interpreted as equal performance across the five classes. Per-class precision and recall, the fold-level values, and any dispersion measure are unavailable in the paper [35].

4.11 Modality Comparison

This section answers RQ2. Figures 4.7 and 4.8 compare the two single modalities with their fusion on the same samples, under one protocol, for the three reported learners [35].

The single-modality ordering changes with the task. Network flow has the higher reported F_1 for detection under all three learners: 0.90 against 0.89, 0.94 against 0.93, and 0.95 against 0.93. Program code has the higher value for categorisation: 0.87 against 0.80, 0.91 against 0.88, and 0.91 against 0.90. This reversal is consistent across the three learners reported in the paper. The experiment does not identify a causal mechanism for it [35].

Table 4.6 isolates the point differences that answer RQ2. Fusion adds one or two points of F_1 in binary detection. For categorisation, it reduces decision-tree F_1 by two points and increases the ensemble results by one and three points. Recall for the decision tree is unchanged. The reported evidence therefore supports a modest fusion improvement for detection and a learner-dependent result for category classification [35].

The paper reports no fold-level values or dispersion measure. The one- and two-point differences cannot be assigned statistical significance, and the two-decimal rounding prevents a finer comparison. The chapter therefore reports their magnitude and direction as published point estimates [35].

4.12 Limitations and Threats to Validity

Encryption, and the age of the corpus. The strongest limit on external validity is measurable from the material in Section 4.7. Between 0.00 and 10.89 per cent of the flows in each malware category use TLS, so the evaluation traffic is overwhelmingly cleartext, and no conclusion

about detection over encrypted flows follows from it. The gap has widened since the capture. Cleartext was permitted by default up to Android 8.1 and has been disabled by default from Android 9 onwards [154], and applications increasingly encrypt content themselves in addition [152]. A detector fitted to 2017 traffic would meet a different protocol mix today, and the flow statistics of Table 4.2 would be observable while the composition of the traffic they summarise would not be the same.

Part of the flow representation may not describe behaviour. Two of the four groups in Table 4.2 are open to this objection. The TCP base sequence numbers are required to be unpredictable per connection and cannot encode anything about an application. The TTL and TOS values describe the path and the endpoint rather than the sender. The concern has a documented precedent. In a widely used intrusion-detection corpus, the time-to-live features inadvertently fingerprint the operating systems of the hosts involved, and an audit of that corpus removed them as unjustifiably performant [101]. That audit covers different corpora, so the finding does not transfer, although the mechanism might. Both classes here were captured from the same three handsets, which leaves little room for the source-side values to separate them, whereas the destination-side values describe remote endpoints, and command servers and application backends are different populations of hosts. Whether the mechanism acted here was never measured, and the feature matrices do not survive, so it stands as an unresolved threat rather than a finding.

The metric convention is unstated. Section 4.9 shows that the values reported for three of the five compared systems cannot be reconciled with a single confusion matrix on an evaluation cohort whose malicious share lies between 18.2 and 20.5 per cent. Section 4.9 also records that precision, recall and F_1 coincide at the reported two-decimal precision in eight of nine detection configurations. An unstated averaging convention and rounding can both produce this pattern. The paper supplies no raw predictions with which to distinguish them [35]. The comparison should be read as indicative rather than as a precise ranking.

The larger experiment is not time-aware. The 135,905-sample experiment of Section 4.4 used a random split. Training and test samples are therefore drawn from the same period, no temporal ordering is enforced, and no area under the time-decay curve is reported. Section 2.6 sets out why an evaluation of this kind overstates what a detector will achieve against samples that appear after it was trained [12]. That experiment is reported here as a check that the pipeline scales to a larger corpus, and nothing about robustness over time is claimed from it.

Obfuscation. Graph features could not be extracted from 47 obfuscated applications, which is 2.2 per cent of the source corpus. The source publication states in two places that the graph structure is unaffected by obfuscation, which those 47 failures contradict. The defensible version of the claim is narrower: identifiers and manifest entries can be renamed without changing the call structure, so a representation built on structure is insensitive to the transformations that defeat name-based features, and it remains sensitive to packers and to transformations that rewrite the structure itself.

Two steps are unspecified. The weights of the method-level mean in Section 4.5 are not given, which leaves that mean undefined as published. The step that composes the per-method

vectors into the application-level vector in Section 4.6 is not described by any equation. Both are recorded as gaps in the source rather than reconstructed here.

Fusion is not uniformly beneficial. Section 4.11 reports the decision tree on the category task, where the combined representation scores below the program code alone. The paper states that combined features give the best performance, while its category figure reports F_1 and precision of 0.85 for the combined decision tree against 0.87 for program code alone [35].

The threat model is non-adaptive, and the exposure is structural. No adversary aware of this detector is modelled, and no evasion attempt was made against it. That limit has a specific shape here rather than a generic one. The representation is built over the application's function-call graph, and attacks published since operate on exactly that object at the level of the code: semantics-preserving rewriting that injects methods and call edges reaches the input of the pipeline before any feature is computed, reported at 87.9 per cent attack success against graph-summary detectors under white-box access [112], and a related attack reports an average of 85.17 per cent across seven detectors whose set includes graph neural networks, under a perturbation budget of five per cent [155]. Neither was run against Hybroid, so nothing here shows it robust and nothing shows it broken. One asymmetry is worth stating as a hypothesis rather than as a result: the call edges those operators inject are made unreachable by construction, so they generate no traffic, and the flow half of the representation would be unchanged by them. Whether that translates into graceful degradation is untested.

4.13 Chapter Summary

This chapter presented Hybroid, which detects and categorises Android malware from two observations of the same application: the structure of its code, recovered without running it and represented as a function-call graph whose vertices carry embeddings learned from the instructions of each method and from the control flow between its basic blocks, and the traffic it produces on a physical handset, reduced to 13 features of its bidirectional flows. On CICAndMal2017 the combined representation reaches an accuracy of 97.0 per cent for detection, and 0.94 on precision, recall and F_1 for category classification, in both cases with a gradient boosting classifier [35].

The within-system contrasts in Section 4.11 answer RQ2. Fusion adds one or two points of F_1 for detection over the better single modality. For category classification the difference ranges from minus two points for the decision tree to plus three for gradient boosting; random forest gains one point. Network flow leads program code in detection, while program code leads network flow in category classification for each reported learner. These are point differences rounded to two decimals, and the paper reports no fold-level dispersion from which to test their statistical significance [35].

Chapter 5 keeps the question of context and changes where the context is sought. Here it was sought across two sources of evidence, one static and one dynamic, at the cost of requiring a traffic capture for every sample. There it is sought within the program alone, by representing

relations that involve more than two functions at once, for a detector that needs only the application package.

Higher-Order Android Malware Analysis with HGANN-Mal

5.1 Research Problem and Relation to Hybrid

This chapter answers RQ3, stated in Section 1.3: To what extent do attention-weighted hypergraph representations of static Android program structure improve binary malware detection and malware classification over pairwise graph models and non-attentive hypergraph models? It takes up gap G3. What that gap records is an asymmetry in the evidence. The cost of projecting a group relation onto pairwise edges has been measured [70, 71], and the conditions under which the projection loses nothing are known [68, 69], yet the benefit of avoiding it in Android malware analysis rests on a single published study [156]. The weighting usually applied alongside the higher-order encoding carries its own unsettled evidence [77].

HGANN-Mal represents an application as a hypergraph whose vertices are the methods of its function-call graph. Three families of hyperedge are laid over those vertices. The first records proximity in the call graph, the second groups methods whose learned representations are close, and the third follows a backward program slice from a call site that reaches a security-sensitive API. The three are merged into one incidence structure, a hypergraph operator with learned attention over hyperedges reduces the application to a single vector, and a classifier with two heads decides malicious against benign and, separately, the class within a labelled taxonomy. The source publication numbers the pipeline in eleven stages [36], and the chapter keeps those numbers. Stages 0 to 2 catalogue and unpack the packages and recover the call graph and the slices, and Stage 3 computes the node features. Stages 4 to 6 build and assemble the hypergraph, Stage 7 forms mini-batches, Stage 8 is the attention operator, and Stages 9 and 10 are the read-out and the two heads. The construction is described in Sections 5.4 to 5.8 and the evaluation in Sections 5.9 to 5.13.

Three claims are made for the work, and each is stated here in the form the evidence supports. The first is the construction: a method-level function-call graph is lifted to a hypergraph by three rules of different kinds, one topological, one semantic and one derived from a program slice, with a weight on the third that rises with the number of data-transmission calls the slice contains. The second is the operator: membership in a hyperedge is weighted by a learned score rather than by a binary indicator, so the aggregation is trained rather than fixed by the incidence structure. The third is the comparison, which measures the resulting system against two pairwise graph networks and two hypergraph operators without attention, on two corpora and three tasks, under one protocol.

Two limits belong here rather than in the limitations section, because they qualify what is being claimed rather than what was found. The comparison changes the hyperedge vocabulary

and the aggregation together, so a difference in the totals cannot be assigned to either change on its own; Section 5.13 states what experiment would separate them and Section 5.14 carries the consequence. And the field this work enters is narrow. A sweep of the published record finds one other hypergraph-based Android malware detector [156], so the comparison against hypergraph baselines is a comparison against operators taken from other domains rather than against competing Android systems.

Relation to Hybroid

Chapter 4 and this chapter answer one question about context in two ways, and they are not stages of a single development. Hybroid widens the evidence base: it observes the application twice, once as a program and once as a source of network traffic, and asks what the combination adds. HGANN-Mal keeps to one source of evidence and widens the relation instead: it asks what is gained when a group of methods can be recorded as a group rather than as a set of pairs. The first buys context by paying for a second observation, with the acquisition cost that Chapter 4 reports. The second buys context inside an observation already made. Chapter 7 takes up that trade directly, and it is the reason the two chapters sit next to each other.

Their results are not comparable, and no comparison between them is offered anywhere in this dissertation. The corpora differ, the class definitions differ, and the protocols differ. Where the source publication of this chapter cites Hybroid as third-party prior work on hybrid analysis, it is naming the candidate's own earlier work, and this chapter states the relationship rather than repeating that framing.

The work was published at ISC 2025 [36], the candidate first of two authors with his supervisor as the second. The method, its implementation, the experiments and the manuscript are the candidate's own. Section 1.5 states the same in the form the affidavit covers.

5.2 Related Work

Section 2.2 defined the representations and Section 2.3 the operators. What this section adds is the systems: which of them have been built for Android, what they represent, and where the work of this chapter sits among them.

Graph representations of Android applications

Detection over program graphs predates graph neural networks. Adagio embedded call graphs with a neighbourhood hash kernel and classified the result [64], and MaMaDroid abstracted call sequences to package or family level before modelling transitions between them [2]. Their learned successors keep the graph and train the representation over it. The survey that organises this line covers control-flow graphs, call graphs, dependency graphs, heterogeneous application-and-API graphs, system-call graphs and network-flow graphs, and no hypergraph operator appears among them [8].

One of those successors is close enough to this chapter to be worth stating in detail. MsDroid gives up the whole program and works on local snippets around security-sensitive APIs, one subgraph per snippet, classifying the subgraphs and combining their verdicts; it locates malicious snippets under application-level labels alone, and it returns an explanation naming the control flows responsible [157]. The localisation is the same instinct that produces the slice-aware hyperedges of Section 5.7, reached through a different construct. MsDroid cuts subgraphs out of the call graph, while this chapter keeps one graph and lays a hyperedge over the methods a slice touches.

What every member of the line shares is the restriction Section 2.2 describes. An edge relates two methods. A behaviour realised by several methods acting together is representable only as a set of pairs, and the grouping is not recoverable from that set.

Hypergraph learning

Learning over hypergraphs supplies the missing construct. HGNN defined a spectral convolution over the incidence structure [27], its successor generalised the propagation with a random-walk transition matrix and is the operator this chapter calls HGNN+ and the source publication calls HGNNP [28], and attention was carried across by making membership itself real-valued and learned [80]. The line has since moved again: a stability-oriented successor from the same group supersedes the operator used as a baseline here [82], which is where a comparison run today would start.

Hypergraphs for Android malware, and how narrow that field is

A sweep of the published record finds exactly one prior application of hypergraph learning to Android malware, and it is worth describing precisely, both because it is the only true competitor and because the source publication of this chapter misattributes it once in its own text [156]. That work builds a function-call graph at method granularity, describes each vertex by six centrality measures together with a one-hot encoding of what the function does, and forms hyperedges by two rules: the K -order call neighbours of a function, and the set of functions sharing a permission. It evaluates HGNN, HGNN+, HyperGCN and UniGNN against GCN, GIN and GraphSAGE, on Malnet-Tiny and on Drebin, and it reports its best Drebin result as 97.1 per cent accuracy with HGNN+. Its hyperedge weights are fixed in advance and no learned weighting is evaluated anywhere in it, so every member of a hyperedge contributes equally to the aggregation.

HGANN-Mal differs from that work in three respects. It adds a third construction rule that neither topology nor permissions supply, taking a hyperedge from the backward slice of a security-sensitive call site, so that a group of methods jointly implicated in reaching a sensitive API becomes one object. It weights that family by the transmission and access calls the slice contains, which is the only place in the construction where a hyperedge carries a weight other than one. And it replaces fixed membership with a learned score, which is the change RQ3 asks about. Being the second entrant in a two-member field is itself worth stating plainly:

the hypergraph baselines in Section 5.10 are general operators applied to this chapter's own representation, because there is almost nothing else to compare against.

The comparison that the published record does not support

A reader will want the two Android hypergraph systems placed side by side, and the published figures do not allow it. Zhang et al. report 97.1 per cent accuracy for HGNN+ on the twenty-family Drebin task over 4,664 samples under an 80/20 split [156], while Section 5.11 reports 95.0 per cent for HGANN-Mal and 93.3 per cent for the same operator on the same task over the same samples under a 70/20/10 split. Setting those numbers against each other would compare implementations, feature sets, splits and training regimes at once.

The clearest evidence that such a comparison misleads is contained in the pair itself. One operator, HGNN+, on one corpus and one task, differs by 3.8 points of accuracy between the two papers, and that difference is larger than three of the four margins this chapter reports over its own baselines. The problem is not peculiar to these two publications. A recent study that re-implemented twelve Android detectors under one toolchain names unfair comparison as a field-wide defect, on the ground that published approaches are evaluated on datasets of different sizes, different ratios of benign to malicious samples and different training and test proportions [158], which is the same objection in general form [13, 122]. This chapter therefore reports what its own protocol measures and makes no claim against the published figures of the other work.

What the evaluation literature requires

Three further lines bear on the chapter and are taken up where they apply rather than here. Methodological work on machine learning in security names the pitfalls that this chapter's evaluation has to be checked against [13], and studies of detector behaviour under obfuscation, drift and adversarial perturbation give the conditions under which published accuracy figures fail to hold [61, 159, 158]. Attacks aimed specifically at graph-based Android detection show what an adversary can do to the representation itself [160, 112, 155]. Section 5.14 states what each of them implies for the results reported here.

5.3 Observation and Threat Model

Everything this chapter measures follows from one decision about what is observed. HGANN-Mal reads the application package and does not run it. The manifest and the Dalvik bytecode are the whole of the evidence, which fixes both the cost of the analysis and the limits on what it can see.

The cost is low and it scales. No device, no emulator and no scripted interaction is needed, and the analysis of one application is independent of every other, so the pipeline parallelises across a corpus without coordination. The second modality of Chapter 4 depends on traffic

that the CICAndMal2017 authors captured on physical handsets, and Section 7.3 compares the two observation requirements directly.

The limits are the standard limits of static analysis on Android, stated in Section 2.1.2, and three of them bear on the representation this chapter builds. Code loaded at runtime through the class loader is absent from the recovered bytecode, so no vertex exists for it. Reflective invocation resolves a target from a string at runtime, so the call edge is missing from a statically recovered call graph unless the analyser models the reflection explicitly, which the pipeline described here does not. Packing and native payloads move the interesting behaviour outside the DEX files that are parsed at all. Each of the three removes vertices or edges from the object the model consumes, and a representation that is built on relations between methods is affected by a missing method more than a representation built on a bag of permissions.

The adversary assumed by the evaluation is not adaptive. The corpora are historical collections, their samples were written without knowledge of this detector, and no sample is modified in response to what the detector learns. That assumption is standard for the reported task and it is also the assumption Chapter 6 argues should be stated rather than left implicit, which is why it is stated here. It has a consequence the chapter does not evade: an adversary who rewrites the application changes the call graph the extraction recovers, and everything downstream of that graph inherits the change. Section 5.14 works that exposure out against the published attacks on function-call-graph detectors.

5.4 APK Processing and Function-Call Graph Extraction

Stage 0 builds a catalogue. Every application package under a root directory is hashed with SHA-256, and the hash is joined against the label list distributed with the corpus, so each sample carries a benign label or a family label before any analysis runs. Working from the hash rather than from the file name matters for a corpus assembled from several sources, since it makes duplicate files visible; Section 5.9 returns to what was and was not done with that information.

In Stage 1 each package is decompressed with `apktool` and the Python `zipfile` module, which yields the manifest and every `classes*.dex` file the package contains. Applications built with multi-dex are therefore covered, and the bytecode of all of them enters the next stage. The reported runs used `apktool`, `MobSF`, `Androguard` with its DAD decompiler and a ready-made `code2vec` model.

`Androguard` [51], which the source runs beside `MobSF` in Stage 2a, disassembles the DEX files and produces the object the rest of the chapter is built on: a function-call graph $G = (V, E)$ at method granularity, in which a vertex is a method identified by its class, its name and its descriptor, and a directed edge records that one method may invoke another. The same pass emits a flat list of pairs that records, for each method, which framework APIs it calls. That list is what allows a flagged API to be located at its call sites rather than merely counted, and Section 5.5 uses it for exactly that.

Two properties of this object were fixed in Section 2.2.1 and are not restated here beyond their consequences. A statically recovered call graph over-approximates, so an edge in G records that an invocation is possible and not that it occurs. And the recovered graph depends on the analyser: a different toolkit, or a different treatment of the component lifecycle and of callbacks registered with the framework, yields a different vertex and edge set for the same application. Both properties travel through every construction in Section 5.7, because each of the three hyperedge rules is defined over G or over an artefact derived from it.

5.5 Security-Sensitive API Identification and Program Slicing

Stage 2 decides which parts of an application are worth isolating. In Stage 2a two tools run over each package. MobSF parses the manifest, disassembles the bytecode and reports the permissions an application requests together with the framework APIs it uses. Of the permissions reported, only those whose status is *dangerous* in the platform’s own classification are retained, which narrows attention to the capabilities that guard user data and device functions [48].

The APIs reached from those capabilities are then sorted into two classes by a hand-written rule. An *access* API reads something the user would consider private, and the examples the source publication names are contacts, location, short messages and storage. A *transmission* API moves data off the device, with HTTP clients, raw sockets and `sendTextMessage` as its examples. The pairing is deliberate. Data exfiltration is a sequence in which something is read and then sent, and a rule that marks both ends of that sequence gives the construction in Section 5.7 something to join into one group. The source publication describes the classification as a heuristic.

Stage 2b then computes a backward slice for every call site of a marked API. The variables that reach the call site are identified first, and the control-flow graph of the enclosing code is walked backwards, keeping only the blocks and statements that influence those variables. What the stage stores is a compact control-flow graph per slice. The purpose is localisation rather than verification: the slice does not establish that data flows from a source to a sink, in the sense that a taint analysis would [53], and it is not offered as evidence of exfiltration. It marks a neighbourhood of code that participates in reaching a sensitive call, and that neighbourhood becomes one hyperedge.

5.6 Node Features

In Stage 3 each vertex of the call graph receives a feature vector assembled from four groups, listed in Table 5.1 and drawn in Figure 5.1. The groups are concatenated and the result is standardised to zero mean and unit variance across the corpus, giving the feature matrix X with one row per method. The figure labels the blocks h_{op} , s_{struct} , a_{api} and c_{2v} , and the last of

Table 5.1. The four groups of node feature. The width of the first group is the size of the opcode vocabulary.

Group	Width	What it records
Opcode histogram	vocabulary	the distribution of Dalvik instructions in the method body
Structural metrics	4	in-degree, out-degree, PageRank and betweenness centrality of the method in G
API indicators	4	whether the method reaches an access API and whether it reaches a transmission API, with the counts of each within the slices attached to it
Semantic embedding	320	a code2vec representation of the decompiled method body

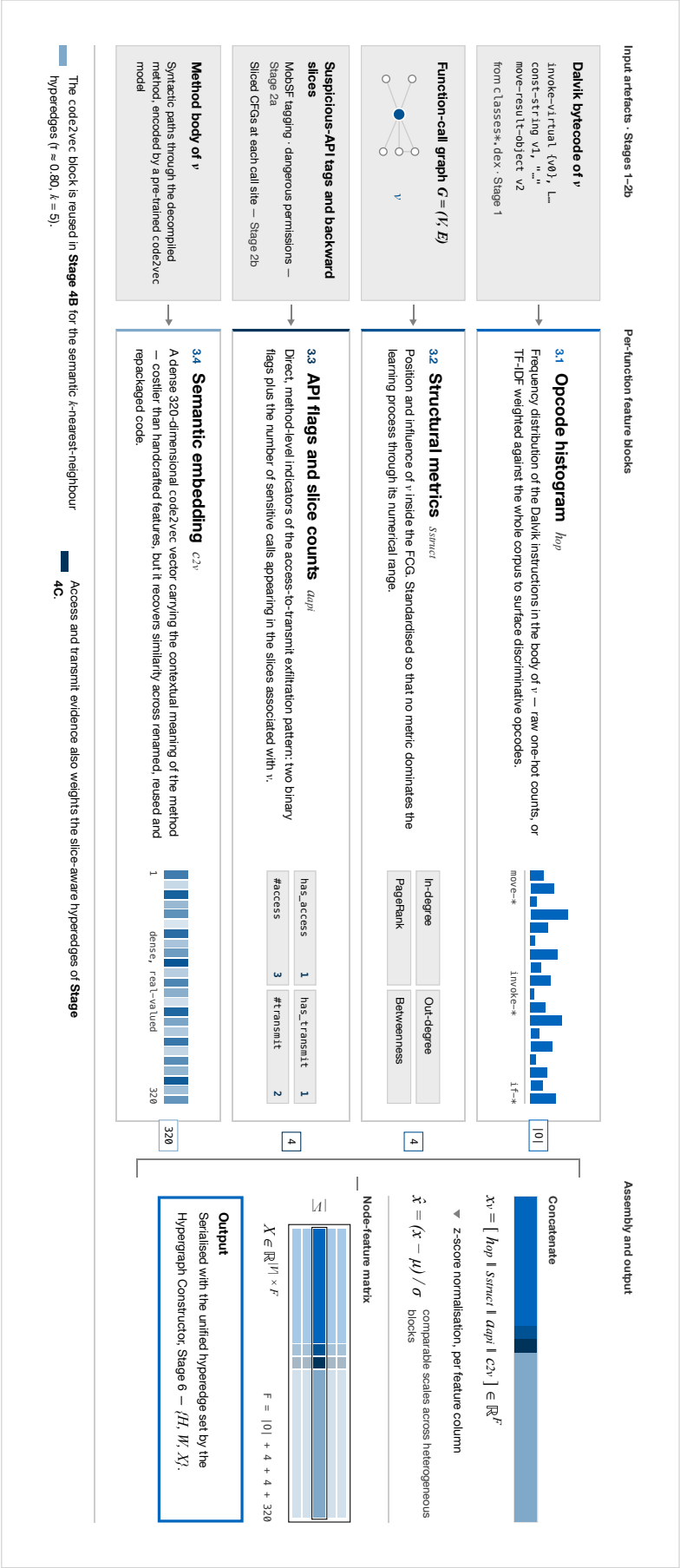
them is the embedding written c_v in Section 5.7. With O the opcode vocabulary, the width is $F = |O| + 328$.

The first group describes what a method does at instruction level, and it is the feature the program-code branch of Chapter 4 also rests on, there as a learned embedding of the opcode sequence rather than as a histogram. Operand values are not used, which removes the constants and register numbers that vary between builds of the same code. The histogram can be taken as raw counts or weighted by inverse document frequency across the corpus.

The second group places the method in the call graph rather than describing its body, and it is the only group that changes when the graph changes. The third connects the vertex to the analysis of Section 5.5, so that a method reaching a sensitive API is distinguishable before any hyperedge is built. Both are small, and together they contribute eight of the dimensions.

The fourth group is the largest. code2vec represents a method as a bag of paths through its abstract syntax tree and learns a fixed-width vector from that bag [67]. The published method consumes Java source, and the released model was trained on a corpus of Java source, whereas the input here is Dalvik bytecode recovered from a distributed package. The candidate confirms that the two were bridged by decompilation: each method was decompiled to Java with DAD, the decompiler built into Androguard, and embedded with a ready-made, pre-trained code2vec model. The source publication records neither step, and the width it reports differs from the released model's.

The embedding of a method therefore depends on how completely the decompiler recovers its source, as well as on the model. That dependence matters for the construction too. The semantic hyperedges of Section 5.7 are built from cosine similarities between these embeddings, so the decompiler and the model shape one of the three construction rules as well.



The code2vec block is reused in **Stage 4B** for the semantic k -nearest-neighbour hyperedges ($r = 0.80, k = 5$)

Access and transmit evidence also weights the slice-aware hyperedges of **Stage 4C**

Figure 5.1.

Construction of the node feature vector in Stage 3. For each method v of the function-call graph, an opcode histogram, four structural centrality metrics, four suspicious-API flags and counts, and a 320-dimensional code2vec embedding are computed from the artefacts of Stages 1 to 2b. The four blocks are concatenated into x_v and z-score normalised, which yields the node-feature matrix $X \in \mathbb{R}^{|V| \times F}$ consumed by the hypergraph constructor in Stage 6.

5.7 Hyperedge Construction

The lifting from a graph to a hypergraph is where the representation is decided. Stages 4A to 4C apply three rules of different kinds to the same vertex set, and Stages 5 and 6 merge and assemble what they produce. Write $\mathcal{H} = (\mathcal{V}, \mathcal{E}, w)$ for the result, with $\mathcal{V} = V$ the methods of the call graph. Figure 5.2 draws one hyperedge of each kind together with the parameters of its rule. Its panels keep the names H_K , H_{SEM} and H_{API} that the source gives the three lists, which Equation 5.5 writes as the candidate sets $\mathcal{C}_{\text{topo}}$, $\mathcal{C}_{\text{sem}}$ and $\mathcal{C}_{\text{sus}}$.

Topological hyperedges. The source collects, for each method v , its callers and callees up to order K . Let $\vec{d}_G(v, u)$ be the number of call edges on a shortest directed path from v to u , with $\vec{d}_G(v, u) = \infty$ when there is none, and let $\text{dist}_G(v, u) = \min\{\vec{d}_G(v, u), \vec{d}_G(u, v)\}$. Then

$$e_{\text{topo}}(v) = \{u \in V : \text{dist}_G(v, u) \leq K\}, \quad K = 3, \quad (5.1)$$

holds v together with its callers and callees up to K steps away. This convention follows the wording of the source. A distance measured with call directions ignored would also admit the other callees of a caller and would give larger hyperedges, and the source does not record which convention the implementation used. This is the rule the earlier hypergraph work on Android uses, there in combination with a permission-based rule [156]. It records a neighbourhood as one object rather than as a set of pairs, which is the whole of the higher-order claim at this point: a vertex and its K -hop neighbourhood enter the operator together.

Semantic hyperedges. A nearest-neighbour search over the code2vec embeddings, computed with the FAISS library, groups methods whose representations are close. Write $c_v \in \mathbb{R}^{320}$ for the embedding of method v , the fourth group of Table 5.1; the full feature vector x_v also carries the other three groups, and the search uses the embedding alone. The candidate confirms that the search compares cosine similarities, $\text{sim}(a, b) = a^\top b / (\|a\| \|b\|)$, and that it runs within one application. With $\mathcal{N}_k(v)$ the k methods of the same application, other than v , that score highest under sim ,

$$e_{\text{sem}}(v) = \{v\} \cup \{u \in \mathcal{N}_k(v) : \text{sim}(c_v, c_u) \geq \tau\}, \quad k = 5, \quad \tau \approx 0.80. \quad (5.2)$$

The original algorithm, whose test panel 4B of Figure 5.2 reproduces, names the search output D , a letter that suggests a distance, and keeps a neighbour when $D \geq \tau$. The test agrees with the cosine similarity, since for a distance the inequality would run the other way. The query method is included and is not counted among its k neighbours, so $|e_{\text{sem}}(v)| \leq k + 1$; the source does not say whether its search counted the query itself. Nothing in this rule refers to the call graph, which is the point of including it: two methods that never call one another, and that sit in different parts of the application, are joined when they do the same kind of work. The source names polymorphic malware and code reuse as the intended cases. Because the search stays inside one application, a semantic hyperedge joins methods of that application only, and code shared across a family can make the hyperedges of different applications alike without joining them.

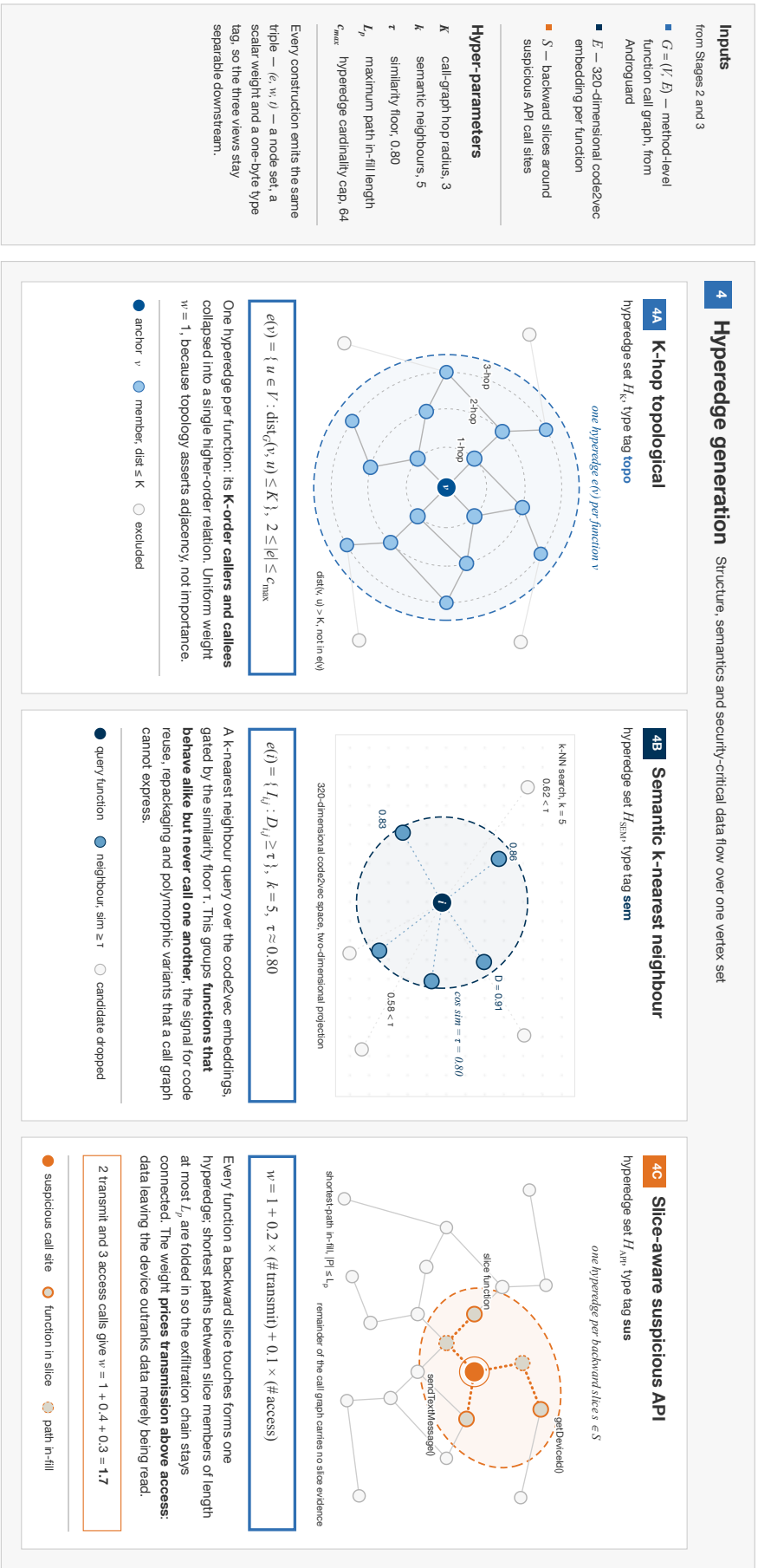


Figure 5.2.

Hyperedge generation in Stage 4: three complementary constructions over one vertex set of methods. (4A) Topological hyperedges take the K -hop ball around every method in the call graph, with $K = 3$ and uniform weight $w = 1$. (4B) Semantic hyperedges are k -nearest-neighbour groups in the 320-dimensional code2vec space, with $k = 5$, admitted only above the similarity floor $\tau \approx 0.80$. (4C) Slice-aware hyperedges are built from the backward slices of Stage 2b, extended with shortest paths of length at most L_p between slice members, and weighted $w = 1 + 0.2 \times \# \text{transmit} + 0.1 \times \# \text{access}$. A candidate with fewer than 2 or more than 64 members is rejected. Node colour marks hyperedge membership only.

Slice-aware hyperedges. Each backward slice s from Section 5.5 yields one hyperedge over the methods the slice contains, extended by the methods on short call paths between them. Let F_s be that member set, fixed before any path is added. For a directed path P in the call graph, let $V(P)$ be its methods and $\ell(P)$ its number of call edges, and let P_{uv} be a shortest path from u to v . Then

$$e_{\text{sus}}(s) = F_s \cup \bigcup_{\substack{u,v \in F_s, u \neq v \\ P_{uv} \text{ exists, } \ell(P_{uv}) \leq L_p}} V(P_{uv}). \quad (5.3)$$

Only pairs of original members are joined, so a method that one path adds does not start further paths. The original algorithm loops over the pairs of the set it is enlarging, and freezing F_s is the convention adopted here to make the result independent of the loop order. A pair with no directed path contributes nothing. Where several shortest paths exist, the one chosen decides which methods join. This family carries the only non-unit weight in the hypergraph:

$$w(e_{\text{sus}}(s)) = 1 + 0.2 \# \text{transmit}(s) + 0.1 \# \text{access}(s), \quad (5.4)$$

so a slice that both reads private data and sends it weighs more than one that only reads, and the transmission end weighs twice the access end. The weight is a construction heuristic, so it cannot be read as a probability of exfiltration.

Merging. Each rule proposes candidate triples (e, w, t) of a member set, a weight and a type tag. Collected as a multiset and passed through the post-processing of the source, they give

$$\mathcal{C} = \mathcal{C}_{\text{topo}} \uplus \mathcal{C}_{\text{sem}} \uplus \mathcal{C}_{\text{sus}}, \quad \mathcal{E} = \text{Post}_\eta(\{(e, w, t) \in \mathcal{C} : 2 \leq |e| \leq c_{\text{max}}\}), \quad c_{\text{max}} = 64. \quad (5.5)$$

The lower bound removes singletons, which carry no relation, and with them every slice that has a single member, since such a slice has no pair to join. The upper bound is described in the source publication as necessary for stable message passing. Its prose says that hyperedges are capped, whereas its algorithm rejects a candidate outside the interval. This chapter follows the algorithm: truncation would need a rule for choosing which members to keep, and none is given. Rejection has a consequence of its own. A method whose K -step neighbourhood holds more than 64 methods loses its topological hyperedge entirely, so the topological family omits exactly the methods with the largest neighbourhoods.

The operator Post_η removes duplicates. Two candidates with the same members can carry different weights or tags, as a slice hyperedge and a topological one over the same methods would, and the policy η that decides which weight and which tag survive is not recorded; this chapter does not supply one. Filtering can also leave a method in no hyperedge, for instance an isolated method that belongs to no slice and has no semantic neighbour above τ . Its degree is then zero and the normalisation of Section 5.8 is undefined for it, and the source does not say whether such methods were dropped or handled in another way.

Each surviving hyperedge is tagged with its family in one byte, and the incidence matrix H and the diagonal weight matrix W are assembled from the result and serialised per application. Algorithm 5.1 restores the construction procedure of the source publication with the conventions above written into it. It also adds the post-processing and the sparse assembly, which the output line of the original announces and its body omits. The type tag records

Algorithm 5.1 Construction and assembly of the HGANN-Mal hypergraph. Stages 4A to 4C restore the algorithm of the source publication [36] with the conventions of Equations 5.1 to 5.3 written in; stages 5 and 6 add the post-processing and assembly that its prose describes.

Input: call graph $G = (V, E)$; embeddings c_v ; slices $s \in \mathcal{S}$ with member sets F_s ; parameters $K, k, \tau, L_p, c_{\max}$; duplicate policy η

Output: sparse incidence matrix H , weight matrix W , type tags t

Stage 4A: topological hyperedges

```

1:  $\mathcal{C} \leftarrow \emptyset$  ▷ multiset of candidates
2: for  $v \in V$  do
3:    $e \leftarrow \{u \in V : \text{dist}_G(v, u) \leq K\}$ 
4:   if  $2 \leq |e| \leq c_{\max}$  then
5:      $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, 1, \text{topo})\}$ 

Stage 4B: semantic ( $k$ -NN) hyperedges
6:  $(I, S) \leftarrow \text{FAISS}(\{c_v\}_{v \in V}, k)$  ▷  $k$  others per method;  $S$  holds cosine similarities
7: for  $v \in V$  do
8:    $e \leftarrow \{v\} \cup \{I_{v,j} : S_{v,j} \geq \tau\}$ 
9:   if  $2 \leq |e| \leq c_{\max}$  then
10:     $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, 1, \text{sem})\}$ 

Stage 4C: slice-aware hyperedges
11: for  $s \in \mathcal{S}$  do
12:    $e \leftarrow F_s$ ;  $w \leftarrow 1 + 0.2 \# \text{transmit}(s) + 0.1 \# \text{access}(s)$ 
13:   for  $(u, v) \in F_s \times F_s$  with  $u \neq v$  do ▷  $F_s$  is not enlarged
14:     if  $P_{uv}$  exists and  $\ell(P_{uv}) \leq L_p$  then
15:        $e \leftarrow e \cup V(P_{uv})$ 
16:   if  $2 \leq |e| \leq c_{\max}$  then
17:      $\mathcal{C} \leftarrow \mathcal{C} \uplus \{(e, w, \text{sus})\}$ 

Stage 5: post-processing
18:  $\mathcal{E} \leftarrow \text{Post}_\eta(\mathcal{C})$  ▷ policy  $\eta$  not recorded

Stage 6: sparse assembly
19: for  $j = 1, \dots, |\mathcal{E}|$ , with  $(e_j, w_j, t_j)$  the  $j$ -th element of  $\mathcal{E}$  do
20:    $H_{vj} \leftarrow 1$  for every  $v \in e_j$ ;  $W_{jj} \leftarrow w_j$ ;  $t(j) \leftarrow t_j$ 
21: return  $H, W, t$ 

```

something the operator never reads. The layer described in Section 5.8 consumes H, W and X , so a hyperedge's family influences the model only through its weight, and since two of the three families carry weight one, the topological and semantic hyperedges are indistinguishable to the operator once they are built. That is a design gap rather than an error, and Section 5.13 names it as the cleanest available extension.

5.8 Hypergraph Attention Model

The operator is defined over the structure fixed in Section 2.2.2, whose notation is used here without restatement: the incidence matrix H , the weighted vertex degree $d(v)$ and the hyperedge degree $\delta(e)$ with their diagonal matrices D_v and D_e , and the diagonal weight matrix W . Write $n = |\mathcal{V}|$, and assume for now that $d(v) > 0$ for every method; Section 5.7 records that filtering can violate this. The hypergraph Laplacian is

$$L = I - P_0, \quad P_0 = D_v^{-1/2} H W D_e^{-1} H^\top D_v^{-1/2}, \quad (5.6)$$

and for a signal $f \in \mathbb{R}^n$ over the methods it has the energy

$$f^\top L f = \frac{1}{2} \sum_{e \in \mathcal{E}} \frac{w(e)}{\delta(e)} \sum_{u,v \in e} \left(\frac{f_u}{\sqrt{d(u)}} - \frac{f_v}{\sqrt{d(v)}} \right)^2, \quad (5.7)$$

where the inner sum runs over ordered pairs [26]. The energy is small when the degree-normalised signal varies little inside the constructed groups, and each hyperedge counts in proportion to its weight and in inverse proportion to its size. It describes the operator and is not a term of the training loss. One layer of hypergraph convolution propagates through P_0 :

$$X^{(l+1)} = \sigma \left(D_v^{-1/2} H W D_e^{-1} H^\top D_v^{-1/2} X^{(l)} \Theta^{(l)} \right), \quad (5.8)$$

with $\Theta^{(l)}$ the learned projection of layer l and σ a non-linearity. Read from right to left, the operator gathers vertex states into the hyperedges that contain them, averages within each hyperedge through D_e^{-1} , scales by the hyperedge weight, and returns the result to the member vertices with the symmetric normalisation of $D_v^{-1/2}$. One layer therefore couples every pair of methods that share a hyperedge. The matrix P_0 is positive semidefinite and similar to the row-stochastic matrix $D_v^{-1} H W D_e^{-1} H^\top$, so its eigenvalues lie in $[0, 1]$. Equation 5.8 thus multiplies each eigencomponent of L with eigenvalue λ by $1 - \lambda$, and the components with the highest energy are damped most. This is the smoothing behind the low-pass bias recorded in Section 2.3.3.

Attention over hyperedges

Equation 5.8 weights a vertex's hyperedges by W , which is fixed when the hypergraph is built. Attention replaces the binary membership indicator by a learned real value, so that the strength with which a vertex participates in a hyperedge is trained [80]. The layer index is dropped from here on. Let $X \in \mathbb{R}^{n \times F}$ hold the input states as rows, let $\Theta \in \mathbb{R}^{F \times F'}$ be the projection, and set $Y = X\Theta$, with $y_v \in \mathbb{R}^{F'}$ the row of method v read as a column vector. A hyperedge is first represented by the mean of the projected states of its members,

$$z_e = \frac{1}{\delta(e)} \sum_{u \in e} y_u, \quad (5.9)$$

and the coefficient joining a vertex to one of its hyperedges is the normalised score

$$\alpha(v, e) = \frac{\exp(\sigma(a^\top [y_v \parallel z_e]))}{\sum_{e' \ni v} \exp(\sigma(a^\top [y_v \parallel z_{e'}]))}, \quad (5.10)$$

with $a \in \mathbb{R}^{2F'}$ a learned vector and $\parallel$ concatenation. Collecting the coefficients into H_{att} , whose entry at (v, e) is $\alpha(v, e)$ where H has a one and zero elsewhere, gives the layer actually used:

$$X' = \sigma \left(D_v^{-1/2} H_{\text{att}} W D_e^{-1} H_{\text{att}}^\top D_v^{-1/2} X \Theta \right). \quad (5.11)$$

The degree matrices are those of the binary H , as in the source publication; degrees recomputed from H_{att} would define a different layer. Gradients reach H_{att} as well as X and Θ , so the membership weights are learned. The support is not. $H_{ve} = 0$ implies $\alpha(v, e) = 0$, and a finite score gives every present membership a positive coefficient, so training reweights the

memberships the construction produced and cannot add a method to a hyperedge or remove one.

Equations 5.9 and 5.10 state the additive form of the score, which the source publication leaves as an uninstantiated similarity function and which the candidate confirms is what was implemented. The published formulation also writes the score between two vertices while indexing it by a hyperedge; the form above resolves that by scoring a vertex against a representation of the hyperedge, which is what the implementation computes.

The messages between methods

Expanding Equation 5.11 shows what one method receives from another. The layer runs in two stages. Each hyperedge first forms a message, and each method then combines the messages of its hyperedges:

$$\mu_e = \frac{1}{\delta(e)} \sum_{u \in e} \frac{\alpha(u, e)}{\sqrt{d(u)}} y_u, \quad x'_v = \sigma \left(\frac{1}{\sqrt{d(v)}} \sum_{e \ni v} w(e) \alpha(v, e) \mu_e \right). \quad (5.12)$$

Substituting the first expression into the second gives one matrix of couplings between methods,

$$X' = \sigma(P_{\text{att}} X \Theta), \quad P_{\text{att}}(v, u) = \frac{1}{\sqrt{d(v)} d(u)} \sum_{e \ni v, u} \frac{w(e)}{\delta(e)} \alpha(v, e) \alpha(u, e). \quad (5.13)$$

Two methods interact only through the hyperedges they share, and each shared hyperedge adds one term. In that term the coefficient of the sender and the coefficient of the receiver appear as a product. The construction weight $w(e)$ is a separate factor, so the slice heuristic of Equation 5.4 and the learned coefficients act on a message independently. The coefficients depend on the layer input through Equations 5.9 and 5.10, which makes P_{att} a function of X ; for a fixed input it is a symmetric matrix.

The key z_e and the message μ_e are different aggregations over the same members. The key is an unweighted mean, whereas the message weights each member u by $\alpha(u, e)/\sqrt{d(u)}$, so attention changes the average inside a hyperedge as well as the weighting across the hyperedges of a method. It does not change the sign of any term. Every member enters μ_e with a positive coefficient, so before the non-linearity each output is the projection of a non-negative combination of input states, and one layer cannot form a difference between two members of a hyperedge. The high-frequency component that standard hypergraph convolution discards has been characterised [81]. Attention changes the proportions of that average and keeps every coefficient positive, so the propagation of one layer remains an averaging of member states.

Grouping the sum in Equation 5.13 by hyperedge gives a structural statement:

$$P_{\text{att}} = \sum_{e \in \mathcal{E}} \frac{w(e)}{\delta(e)} \psi_e \psi_e^\top, \quad (\psi_e)_v = \frac{\alpha(v, e)}{\sqrt{d(v)}}, \quad (5.14)$$

with $\alpha(v, e) = 0$ for $v \notin e$. Each hyperedge adds a positive semidefinite matrix of rank one, and because every weight is positive, P_{att} is positive semidefinite for every input. The statement concerns the propagation matrix at a fixed input. It carries no guarantee about the convergence of training.

Because $\alpha(v, e)$ depends on both the method and its hyperedge, two different hyperedge groupings can produce the same fixed matrix P_0 yet different attentive propagation matrices. The reported experiments do not determine whether the learned model used this capacity or whether it caused the measured accuracy differences.

What the mechanism can and cannot be claimed to do

Two further properties bound what the chapter claims for the operator.

The first concerns the ranking that the score induces. Splitting $a = (a_1, a_2)$ across the concatenation writes the argument of σ in Equation 5.10 as $q_v + b_e$, with a query term and a hyperedge key

$$q_v = a_1^\top y_v, \quad b_e = a_2^\top z_e. \quad (5.15)$$

If σ is strictly increasing, then for a fixed layer input and any two hyperedges e_1 and e_2 that contain v ,

$$\alpha(v, e_1) > \alpha(v, e_2) \iff b_{e_1} > b_{e_2}. \quad (5.16)$$

The proof is short. Because σ and $\exp$ are strictly increasing, the numerators of Equation 5.10 are ordered as the keys are, and the two coefficients share one positive denominator. Any two methods therefore rank the hyperedges they share in the same order, and the keys alone fix that order. Methods can still attend to different hyperedges, since each normalises over its own. A non-decreasing activation such as ReLU can merge two hyperedges into a tie but cannot reverse their order between methods. The argument is the one Brody et al. give for the additive score of pairwise graph attention [29], applied to the score of Equation 5.10. It holds for each attention head separately and says nothing about hypergraph attention with other scoring functions. Under either kind of activation, the restriction on wording that the chapter keeps is thus a result for this operator: within one layer, HGANN-Mal can learn which hyperedges matter, and it cannot reverse their order from one method to another.

The query term does not cancel, however. With σ a leaky ReLU of slope 0.2 and keys -1 and 1 , the coefficients are approximately 0.23 and 0.77 for $q_v = 0$, and 0.12 and 0.88 for $q_v = 2$. The order agrees and the values differ. Cancellation is exact when σ is affine on the arguments of one method, as a leaky ReLU is when they share a sign.

The second concerns normalisation. Uniform coefficients do not reproduce Equation 5.8, because the degrees in Equation 5.11 are those of the binary H . Let r_v be the number of hyperedges containing v and $R = \text{diag}(r_v)$. Equal scores give $H_{\text{att}} = R^{-1}H$ and hence

$$P_{\text{att}} = R^{-1}P_0R^{-1}, \quad (5.17)$$

which reduces to P_0/r^2 when every method lies in r hyperedges, and to $P_0/4$ in the example above. An ablation that replaced the learned coefficients by uniform ones would therefore

change the scale of propagation as well as its weighting. A comparison with the non-attentive operator needs a stated normalisation convention before a difference can be read as an effect of attention.

Read-out and classification

Applications differ in size, so the vertex states are collapsed into one vector per application. Two variants are named in the source publication, an unweighted mean over the vertices and a learned attention pooling.

Training and inference run on batches. The source assembles a batch of B applications by placing their incidence and weight matrices on a block diagonal and stacking their feature matrices,

$$H_{\mathcal{B}} = \text{diag}(H_1, \dots, H_B), \quad W_{\mathcal{B}} = \text{diag}(W_1, \dots, W_B), \quad X_{\mathcal{B}} = \begin{bmatrix} X_1 \\ \vdots \\ X_B \end{bmatrix}, \quad (5.18)$$

together with an index that maps each row of $X_{\mathcal{B}}$ to its application. No hyperedge spans two blocks, so degrees, keys, coefficients and messages are all computed inside one application. The output for an application therefore does not depend on the other applications in its batch, and the read-out uses the index to pool each block on its own.

The layer is also equivariant under relabelling. Let Π reorder the methods of an application and Q its hyperedges, and write $\Phi(X, H, W)$ for the output of Equation 5.11. Then

$$\Phi(\Pi X, \Pi H Q^{\top}, Q W Q^{\top}) = \Pi \Phi(X, H, W), \quad g(\Pi X, \Pi H Q^{\top}, Q W Q^{\top}) = g(X, H, W), \quad (5.19)$$

where g is the application vector. The first identity holds because the degree matrices, the keys and the coefficients all permute with H , and the second because both read-outs are symmetric in the vertex states. The prediction therefore does not depend on the order in which method rows or hyperedge columns are stored. This concerns the hypergraph once it is built. It gives no resistance to obfuscation, and order-dependent choices earlier in the pipeline, such as tie-breaking among nearest neighbours or among shortest paths, can change H itself.

That vector passes through a shared multi-layer perceptron with two hidden layers, rectified linear activations and dropout, and then into two heads. The first emits a single logit whose sigmoid is the probability that the application is malicious. The second emits one logit per class of the labelled taxonomy, normalised by a softmax. Training minimises the sum of the two objectives,

$$\mathcal{L} = \lambda_{\text{bin}} \text{BCE}(z_{\text{bin}}, y_{\text{bin}}) + \lambda_{\text{fam}} \text{CE}(z_{\text{fam}}, y_{\text{fam}}), \quad (5.20)$$

weighted by two constants.

Equation 5.20 applies both terms unconditionally. On the corpus of Section 5.12, benign applications form one of the labelled classes and the second term is defined for them. On the corpus of Section 5.11 the classes are twenty malware families. A benign application has no

family among them, and neither has a malicious application from one of the smaller families, so the family term has no target for either. One well-defined specification averages each term over the training samples that carry its label,

$$\mathcal{L} = \frac{\lambda_{\text{bin}}}{|\mathcal{D}_{\text{bin}}|} \sum_{i \in \mathcal{D}_{\text{bin}}} \text{BCE}(z_{\text{bin},i}, y_{\text{bin},i}) + \frac{\lambda_{\text{fam}}}{|\mathcal{D}_{\text{fam}}|} \sum_{i \in \mathcal{D}_{\text{fam}}} \text{CE}(z_{\text{fam},i}, y_{\text{fam},i}), \quad (5.21)$$

with $\mathcal{D}_{\text{bin}}$ and $\mathcal{D}_{\text{fam}}$ the non-empty sets of samples that have a binary label and a class label.

5.9 Datasets, Tasks, and Split Policy

Two corpora are used, and they answer to different needs. Drebin is the reference collection of the field, which makes results on it comparable with a long line of published work, and its family labels support a classification task that a category taxonomy cannot. CICMalDroid is newer, it carries benign applications alongside malicious ones, and its labels are categories of behaviour rather than families. Using both guards against a conclusion that holds only for one collection, which is the reason the source publication gives for the second corpus.

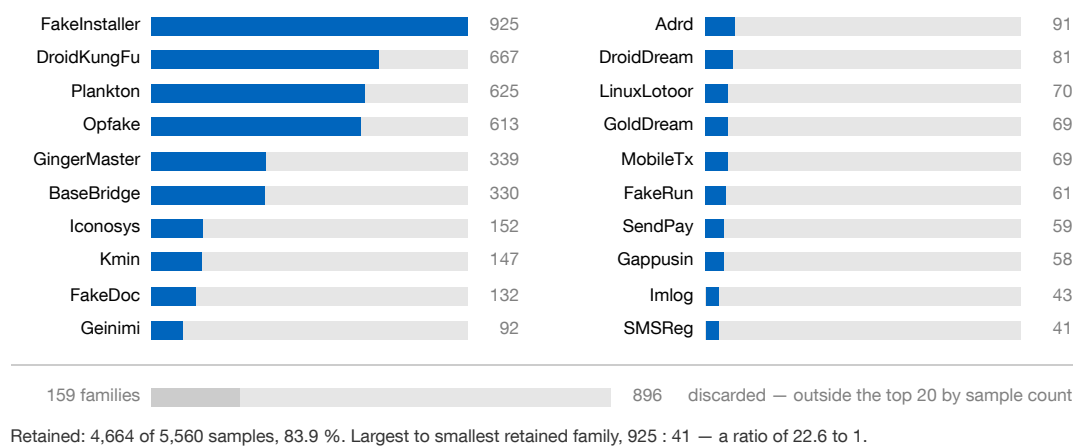
Drebin

Drebin is the malicious part of a larger collection gathered between August 2010 and October 2012 from the Google Play Store, from Chinese and Russian alternative markets, from Android websites and malware forums, and from the whole of the Malgenome project [1]. Every sample in that collection was submitted to VirusTotal and judged by ten antivirus scanners; an application was counted as malicious when at least two of the ten flagged it, applications labelled adware were then removed, and 5,560 malicious samples remained. The family names are taken from the labels of one of the ten scanners. The source publication of this chapter counts 179 families in the set [36], a figure the dataset paper does not state. Family sizes are heavily skewed, so the classification task uses the twenty largest, which cover 4,664 of the samples and are drawn with their sizes in Figure 5.3(a). The 159 smaller families hold the other 896 samples, 16.1 per cent of the set, and they are left out of the family task only, since binary detection uses all 5,560 malicious samples.

Three properties of that construction bear on the results. The malicious label is a vote of two scanners out of ten, which is a lower bar than a majority and admits samples the other eight consider harmless. The ground truth for malware-family classification is one vendor's naming, which is a convention rather than a taxonomy of behaviour. And the 123,453 benign packages the original DREBIN work evaluated against are not part of what is distributed, which is why a negative class had to be found elsewhere.

Binary detection needs a negative class, and Drebin supplies none, so the benign side was drawn at random from AndroZoo [162]. AndroZoo collects Android applications by crawling Google Play, the Chinese markets Anzhi and AppChina, and several smaller repositories; the crawl began in late 2011 and has continued since, and the collection held more than three million applications when it was first described. Every application in it has been submitted to

(a) Drebin — the twenty retained families



(b) CICMalDroid2020 — five categories

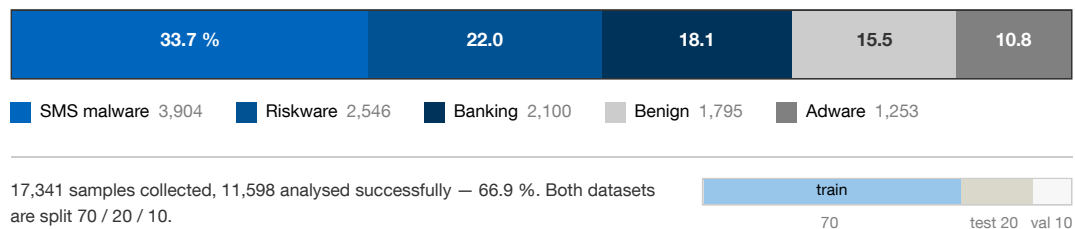


Figure 5.3. Class distribution of the two corpora. (a) The twenty largest Drebin families, the classes of the malware-family classification task. The 159 smaller families, with 896 samples, take no part in that task and remain in binary detection. (b) The five CICMalDroid 2020 categories of the published distribution [161]; Benign is the negative class of binary detection on this corpus. The long tail of Drebin is the reason the source publication reports an F1 score, which it labels Macro-F1, alongside accuracy.

VirusTotal, and what the collection stores per application is the number of antivirus products that flagged it, together with the compilation date of its bytecode, its size and the market it came from. About 9,470 benign applications were used against the full set of 5,560 malicious ones, giving a corpus of roughly 15,030 in which malware is 37.0 per cent.

Turning that stored count into a benign label requires a threshold, and the threshold is a decision rather than a property of the data. The AndroZoo authors report the flagged share at one antivirus product and again at ten, and state that antivirus results have to be treated with care; the two thresholds give very different populations. Section 5.14 states what follows from assembling the two classes from different collections.

CICMalDroid

CICMalDroid 2020 was collected between December 2017 and December 2018 and holds five categories, four malicious and one benign. Its benign class is not a separate acquisition: applications that fell into none of the four malicious categories were taken as benign and then checked against VirusTotal. Its published distribution, drawn in Figure 5.3(b), holds 11,598 applications, which is not the number collected: 17,341 applications were gathered, 13,077 executed successfully in the dataset authors' sandbox, and the labelled distribution retains 11,598 after the analysis records of the remainder failed to parse [161]. This chapter uses the published distribution, so it inherits a selection made by dynamic analysis, and Section 5.14 records what that implies for a static detector. The categories are far more even than the Drebin families, since the largest holds 3.1 times as many samples as the smallest.

Three tasks, and they are not interchangeable

Binary detection separates malicious from benign, and it is reported on both corpora. On Drebin the negative class is the AndroZoo sample described above; on CICMalDroid it is the 1,795 benign applications of Figure 5.3(b) against the remaining 9,803.

The malware-family classification task assigns a malicious application to one of the twenty Drebin families. No benign application takes part, and the task presupposes that the sample is malicious.

The malware-category classification task assigns an application to one of the five CICMalDroid categories, one of which is Benign. It therefore includes the detection decision within a wider one.

The distinction is kept throughout this chapter and throughout the dissertation, because the three tasks have different priors, different error costs and different trivial baselines, and reporting them under one name would make the numbers look comparable when they are not.

The two corpora do not even agree on what counts as malicious. Drebin removed every application its scanners labelled adware, while CICMalDroid keeps Adware as one of its four

Table 5.2. Binary detection, in per cent, as published. The source labels the first column Macro-F1 and states no averaging convention. Section 5.14 shows that the HGANN-Mal values on Drebin are positive-class values for the malicious class, and that several rows cannot follow one convention. Best value in each column in bold.

Method	Drebin and AndroZoo				CICMalDroid			
	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.	Acc.
GCN	92.2	92.2	91.5	92.8	90.6	90.6	90.6	90.9
GraphSAGE	92.1	92.1	92.5	93.3	91.0	91.1	91.1	91.5
HGNN	96.2	95.8	96.3	96.8	94.2	94.8	94.3	94.6
HGNN+	96.1	95.9	96.1	97.1	95.9	96.1	96.5	97.7
HGANN-Mal	97.8	97.8	97.6	98.3	96.2	96.2	97.6	97.6
Majority class	63.0				84.5			

malicious categories. A sample of the kind Drebin discarded is therefore a positive case on one corpus and absent from the other, which is one more reason the figures in Sections 5.11 and 5.12 are not read against each other.

Split policy

Each corpus is divided at random into 70 per cent for training, 20 per cent for testing and 10 per cent for validation, and the same division is used for every method in the comparison, so the baselines and HGANN-Mal see the same samples. At that test fraction the smallest Drebin family, with 41 samples, places about eight samples in the test partition in expectation, so a per-family score for the smallest classes would rest on a handful of decisions. Two properties of that policy are stated here and their consequences are worked out in Section 5.14: the division is random rather than ordered by time, so training and test samples come from the same period, and no deduplication or hash-aware grouping was applied, although Drebin is known to contain repackaged near-duplicates.

5.10 Binary Detection Results

Binary detection asks one question of an application: malicious or not. It is reported on both corpora, and the two experiments differ in a way that matters for reading the numbers. On CICMalDroid the benign applications belong to the corpus, and the task uses the 1,795 benign samples against the 9,803 malicious ones. On Drebin they do not, because Drebin holds malware only, so the benign side was assembled separately as Section 5.9 describes. Table 5.2 reports both. Figures 5.4 and 5.5 plot its F1 and accuracy columns together with those of the multiclass table in Section 5.11, for all four task and corpus combinations.

Three readings follow, and they are of unequal strength.

The clearest is the gap between the pairwise and the higher-order representations. On Drebin every hypergraph operator sits above every pairwise one, by between 3.5 and 5.5 points of

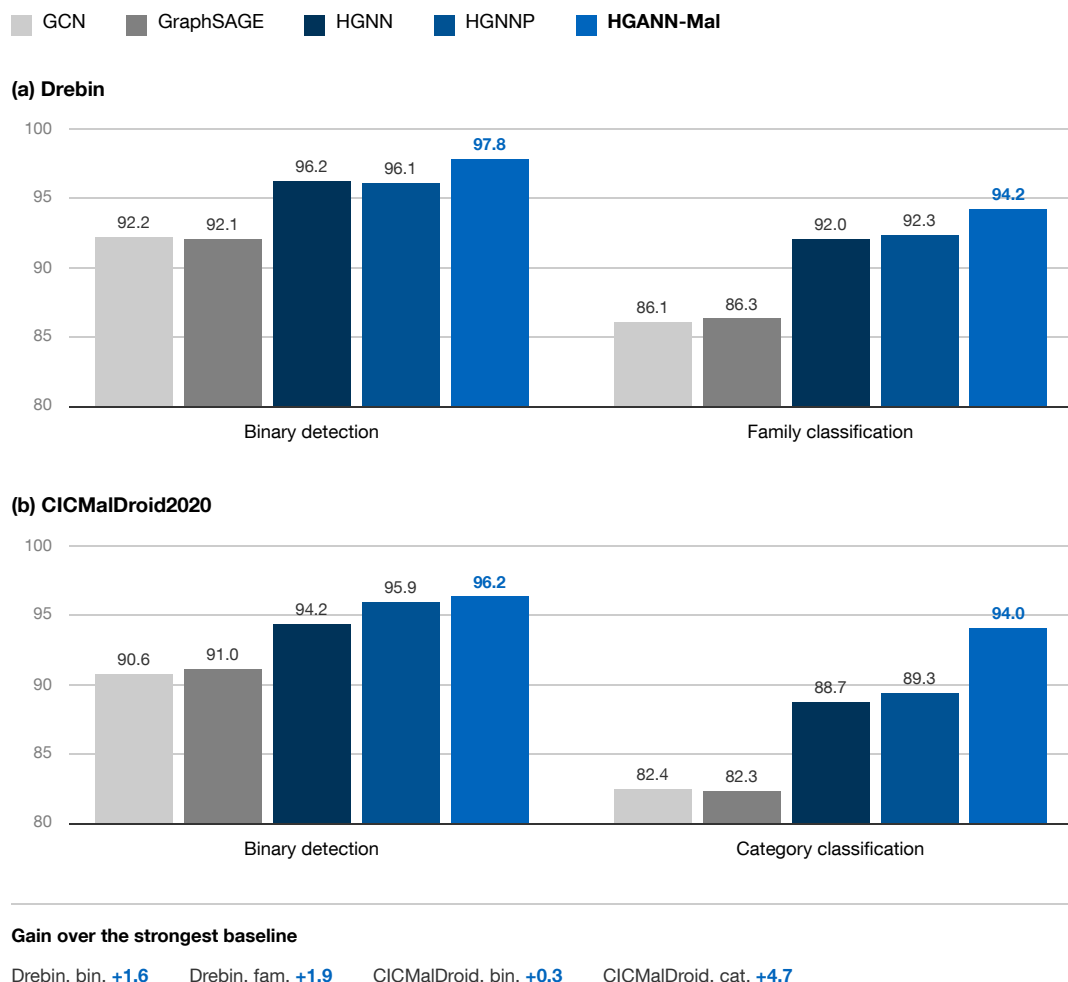


Figure 5.4. F1 on both corpora and both tasks, as published in Tables 5.2 and 5.3. The source labels these values Macro-F1, and Section 5.14 shows that several rows cannot follow one averaging convention. The vertical axis is truncated at 80 so that differences among the hypergraph methods remain visible. HGNNP is the source publication’s name for HGNN+.

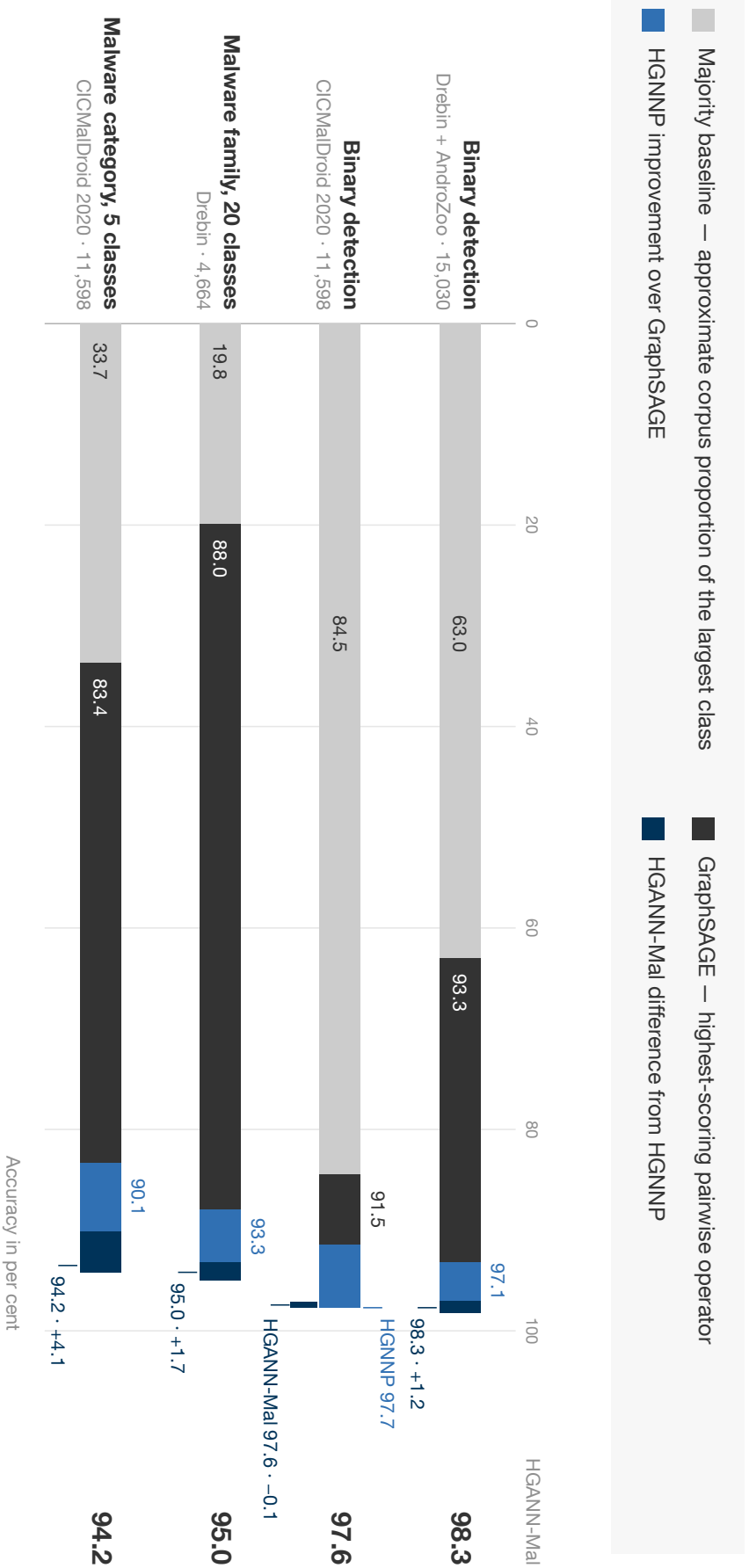


Figure 5.5. Accuracy reported in Tables 5.2 and 5.3, divided at the majority baseline, at GraphSAGE and at HGNNP, which the figure labels HGANN-Mal after the source publication. Variability across runs and the isolated contribution of attention are not established by these reported results.

accuracy and between 3.9 and 5.7 points of F1, and the ordering holds on CICMalDroid as well. Figure 5.5 shows the same step on all four combinations: HGNN+ exceeds the best pairwise operator, GraphSAGE in every case, by 3.8 to 6.7 points of accuracy, and in each combination this step is larger than the difference between HGANN-Mal and HGNN+. All four baselines were re-implemented on this pipeline and consumed the same call graph and the same node features, so the difference belongs to the representation rather than to the input. That is the comparison RQ3 asks for first, and it is the one the evidence supports best.

The second is the gap between HGANN-Mal and the two hypergraph operators without attention. On Drebin it is 1.2 points of accuracy against the stronger of them, and on CICMalDroid it is not there at all: HGNN+ holds the best accuracy at 97.7 against 97.6, while HGANN-Mal holds the better recall at 97.6 against 96.5 and the better F1 at 96.2 against 95.9. A system that leads on three columns of four, and trails by one tenth of a point on the fourth, is level with its baseline on that corpus rather than ahead of it. The source publication reads the same pattern as a precision-recall trade-off, which is one explanation among several; single runs without dispersion cannot separate a trade-off from noise, and Section 5.14 returns to that.

The third concerns the base rate, and it is the reason the last row of the table is there. A constant predictor scores 63.0 per cent on the Drebin corpus, where benign applications are the majority, and 84.5 per cent on CICMalDroid, where they are 15.5 per cent of the samples. Reported accuracy stands 35.3 points above the trivial baseline in the first case and 13.1 points above it in the second. Neither figure is diminished by this, but the second is much less impressive than 97.6 per cent sounds, and the source publication reports no such baseline.

Drebin confusion matrix and class balance

The Drebin binary experiment evaluated HGANN-Mal on 1,112 malicious and 1,894 benign applications. The classifier identified 1,085 malware samples and missed 27. It raised 24 false alarms among the benign applications and classified 1,870 as benign. These 51 errors across 3,006 decisions give a false-positive rate of 1.3 per cent.

The experiment used a corpus with 37.0 per cent malware, whereas deployments may contain a lower share. A separate sensitivity calculation holds recall and the false-positive rate fixed while reducing the malware share to 10 per cent. Under this assumption, precision falls from 97.8 to about 89.5 per cent because benign applications outnumber malicious applications nine to one. This calculation illustrates the effect of class balance under fixed rates, as discussed in Section 2.6.

5.11 Malware-Family Classification on Drebin

The second task assigns a malicious application to the family it belongs to. It runs on the 4,664 Drebin samples of the twenty largest families, shown in Figure 5.3(a), and no benign application takes part. Results are in the left panel of Table 5.3.

Table 5.3. Multiclass classification, in per cent, as published. The Drebin task assigns a malicious application to one of twenty families; the CICMalDroid task assigns an application to one of five categories, of which Benign is one. The two tasks are not comparable with each other. Section 5.14 identifies two rows whose three columns cannot follow one averaging convention.

Method	Drebin, 20 families				CICMalDroid, 5 categories			
	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.	Acc.
GCN	86.1	85.9	86.2	87.8	82.4	82.0	82.4	82.7
GraphSAGE	86.3	86.7	86.2	88.0	82.3	82.2	82.6	83.4
HGNN	92.0	92.1	92.4	92.0	88.7	88.8	88.8	89.0
HGNN+	92.3	92.5	92.5	93.3	89.3	89.5	89.8	90.1
HGANN-Mal	94.2	94.1	93.8	95.0	94.0	94.1	94.0	94.2
Majority class	19.8				33.7			

The ordering repeats: pairwise operators lowest, non-attentive hypergraph operators four to five and a half points of accuracy above them, and HGANN-Mal 1.7 points above the better of those. Against the trivial baseline the margin is wide here, since the largest family covers 19.8 per cent of the samples.

One property of the task deserves more caution than the totals invite. The twenty families range from 925 samples down to 41, a ratio above twenty to one. Read as a macro average, which is how the source labels it, an F1 of 94.2 per cent would require the smallest families to be classified nearly as well as the largest, since every class contributes equally to that average. Section 5.14 shows that the printed precision and recall of the same row exclude any reading in which the three columns share one convention. The classes are also one vendor’s labels rather than an agreed taxonomy, as Section 5.9 records, so a disagreement between the model and the label is not always an error of the model. The families are further related to one another: several of the twenty are known to contain repackaged applications built from shared code. Shared code gives the applications of one family similar semantic hyperedges, each built inside its own application under Section 5.7, and it is also the condition under which duplication inflates a measured score. Section 5.14 returns to that threat.

5.12 Malware-Category Classification on CICMalDroid

The third task assigns an application to one of the five CICMalDroid categories. Benign is one of them, so this task subsumes the detection question rather than sitting beside it, and an application misclassified between two malicious categories is a different kind of error from one misclassified as benign. Results are in the right panel of Table 5.3.

The margin over the non-attentive operators is 4.7 points of F1 and 4.1 of accuracy, the widest of the four task and corpus combinations reported anywhere in this chapter. The narrowest, on the same corpus, is binary detection, where Section 5.10 found no margin at all. The ranking of the four combinations does not depend on the measure. Over the strongest baseline, the F1 margins in Figure 5.4 are 4.7 points here, 1.9 and 1.6 on the Drebin family and binary tasks

and 0.3 on CICMalDroid binary detection, which is also the order of the accuracy margins. That pattern is worth stating plainly because it constrains what may be concluded: whatever the model adds, it adds most where the decision is between several classes and least where it is between two. A representation that groups methods by shared behaviour would plausibly help most in exactly that setting, since categories such as adware and riskware are separated by what an application does with what it collects rather than by whether it collects anything. The observation is offered as a reading of the pattern and not as a finding, since one run per configuration cannot support more.

5.13 Ablation, Efficiency, and Interpretability Analysis

The ablation that was not run

HGANN-Mal differs from the hypergraph baselines in two ways at once. It builds a different set of hyperedges, fusing a topological, a semantic and a slice-aware family where the baselines take k -hop and permission-based ones [156], and it weights membership by a learned score instead of a fixed indicator. Both changes are present in every number in Tables 5.2 and 5.3, and no experiment in this chapter varies one while holding the other. The measured differences are therefore differences between two systems, and they do not establish that attention is what produces them.

Four experiments would settle it, and they are recorded here so that the gap is specified rather than merely admitted. First, the baseline operator without attention, run over the identical fused hyperedge set, which isolates the aggregation. Second, HGANN-Mal run over topological hyperedges alone, which isolates the vocabulary. Third, a leave-one-out over the three hyperedge families, which apportions the vocabulary’s contribution among them. Fourth, a sweep over K , k , τ , $c_{\max}$, L_p and the two weight constants, none of which is varied anywhere in the source publication. The first two are the ones that bear on RQ3; the second two would tell a reader how sensitive the construction is to choices made once and never revisited. Equation 5.17 adds a condition to the first: uniform coefficients also rescale propagation, so that comparison has to fix a normalisation convention before its difference can be attributed to attention.

The question is not idle. On pairwise graphs, over node-property benchmarks and against non-attentive baselines, no unmodified attentive model has been shown to outperform all of those baselines on any dataset or task tested [77], and the scoring functions of both the original and the revised formulations realise only finitely many neighbour rankings at bounded width [29, 77]. Neither result was stated for hyperedges, as Section 2.3.2 records. The static-ranking argument carries over to the score used here, as Equation 5.16 shows, while the finite-ranking bound has not been examined for hyperedge scores. What the two results establish is that the attribution cannot be assumed, and the evidence needed to make it was not gathered.

Efficiency

The stated procedure admits bounds. Let n be the number of methods, m the number of retained hyperedges, $|\mathcal{S}|$ the number of slices and $\nu = \text{nnz}(H) = \sum_{e \in \mathcal{E}} |e|$ the number of stored memberships. Algorithm 5.1 proposes at most one topological and one semantic candidate per method and one candidate per slice, and post-processing only removes candidates. Semantic hyperedges hold at most $k + 1$ methods under the convention of Equation 5.2, and the others at most $c_{\max}$. Hence

$$m \leq 2n + |\mathcal{S}|, \quad \nu \leq (c_{\max} + k + 1)n + c_{\max}|\mathcal{S}| = 70n + 64|\mathcal{S}|, \quad (5.22)$$

and the representation grows linearly in the numbers of methods and slices. For one attention head with input width F and output width F' , a sparse evaluation of Equation 5.11 computes the projection, the keys, the scores and the two incidence products in

$$T_{\text{layer}} = O(nFF' + \nu F'), \quad M_{\text{layer}} = O(nF + (n + m)F' + \nu + FF') \quad (5.23)$$

time and memory, where $m \leq \nu/2$ because every retained hyperedge has at least two members. The bound follows the two stages of Equation 5.12 and never forms P_{att} . That matrix can be far denser than H , since two methods are coupled whenever they share a hyperedge: it can hold up to $\sum_e |e|^2 \leq c_{\max}\nu$ non-zero entries, and the cap bounds this density. Further heads and layers add their own terms.

Construction costs more. With $|E|$ call edges, breadth-first searches from every method build the topological hyperedges in $O(n(n + |E|))$. An exhaustive search within one application evaluates the similarity of $O(n^2)$ pairs of 320-dimensional embeddings, and since the FAISS index is unspecified, no sub-quadratic bound can be claimed for it. The slice paths need one breadth-first search from each member of each slice, $O(\sum_s |F_s|(n + |E|))$ in total. The cap is applied after a candidate is built, so it does not limit the cost of candidates that are then rejected, and centrality computation and the semantic embedding add costs outside these bounds. None of these quantities is a measurement. The claim in the source publication that the approach is inexpensive because it is static is true of the analysis. For the representation, the bounds above are the only evidence, and those for construction are quadratic in the number of methods.

Interpretability

The slice-aware hyperedges are the part of the construction that could carry an explanation. Each one is anchored at a call site of a security-sensitive API and covers the methods a backward slice passes through, so an attention weight on such a hyperedge points at a concrete region of code and at the API that made it interesting. Reporting the highest-weighted hyperedges for a sample, with the call sites they contain, would turn a score into something an analyst can check. Equation 5.16 makes that report compact: within one layer and head every method ranks its hyperedges by the same keys b_e , so one ranked list of keys per application summarises what the layer attends to.

That was not done. The source publication states interpretability as future work, and no attention weight is inspected, visualised or reported anywhere in it. A single retrained model on one corpus would be enough: the weights of one application, ranked, with the API call at the anchor of each hyperedge, would turn this section from a description of what the design permits into a result about what the model attends to. It is named here because it is the cheapest substantial improvement available to the design, and because the construction was built in a way that makes it reachable.

One design gap, and it is the cleanest extension available

Section 5.7 records that every hyperedge is tagged with the family that produced it, and that the operator never reads the tag. A hyperedge reaches the layer as a column of H and an entry of W , so a topological hyperedge and a semantic one over the same methods are the same object to the model, and only the slice-aware family is distinguished at all, through a weight it happens to carry. The type is available, it is stored, and it is discarded.

Two ways of spending it suggest themselves. The weight matrix could carry a learned scalar per family, so that the training decides how much a topological grouping is worth against a semantic one, which costs three parameters. Or the attention score of Equation 5.10 could take the type as an input. Entered additively, the type would only shift the keys b_e , and Equation 5.16 shows that every method would still rank its hyperedges in one shared order; a weighting that depends on the family and the vertex together needs a score that is not additive, such as the revised form of Brody et al. [29]. Neither was tried here, and the second is the more interesting because it is the version of the mechanism that the confound in Section 5.14 leaves unexamined.

5.14 Limitations and Threats to Validity

The results above are measurements on two historical corpora under one protocol. This section states the limits of what they show and closes with a brief account of threats to validity.

The gain cannot be attributed to attention. HGANN-Mal changes the hyperedge vocabulary and the aggregation together, and no experiment separates the two (Section 5.13). The evidence supports a claim about the complete system: it performs better than four baselines on three of the four task and corpus combinations. Whether learned weighting of hyperedges produces that gain is an open question.

Every figure is a single run. No configuration was repeated, and the tables therefore carry no standard deviation, confidence interval or significance test. Small margins are affected most. A tenth of a point separates HGANN-Mal from HGNN+ on CICMalDroid accuracy, and that difference cannot be interpreted. Even the 1.2 points on Drebin would need repeated runs before they could be called reliable.

The evaluation covers favourable conditions only. Both splits are random, so training and test samples come from the same collection period, and the figures do not describe

performance on applications released later [12]. Neither corpus was obfuscated, and no adversary was present. Under obfuscation, concept drift and adversarial examples, none of twelve representative Android detectors kept the performance it reached in the ideal setting, and one call-graph-based detector scored 0.912 of F1 on one year's test set and 0.463 on another's [61]. Sections 5.10 to 5.12 thus report the conditions that a comparison between representations needs. A deployment would meet harder ones.

Some behaviour lies outside the representation. Code fetched at runtime and targets resolved by reflection are absent from the recovered call graph, and so are payloads held in packed archives or native libraries (Section 5.3). A representation built from relations among methods loses more from a missing method than one built from a bag of permissions.

The baselines are general operators. GCN, GraphSAGE, HGNN and HGNN+ were re-implemented on this pipeline's own representation. They test whether the higher-order encoding and attention help within one design, but they do not place HGANN-Mal against the Android detectors that the field uses as reference points. The hypergraph baseline has since been superseded by a stability-oriented successor from the same group [82], so the comparison reflects that line of work as it stood in 2022.

The threat model is non-adaptive. No adversary in this evaluation knows that the detector exists, although published attacks act on the object this pipeline consumes. The first structural attack on graph-based Android detection reports over 90 per cent success in feature space and up to 100 per cent in problem space against function-call-graph detectors under white-box access [160]. FCGHunter rewrites an application with semantics-preserving operators and reaches a white-box success rate of 87.9 per cent against forty detectors built from graph-summary features [112]. A4M reports an average of 85.17 per cent against seven detectors built with MaMaDroid, APIGraph and a graph neural network, under a perturbation budget of five per cent [155]. None of these attacks was run against HGANN-Mal. Every hyperedge of Section 5.7 is derived from the call graph or from quantities computed over it, so methods and edges injected into that graph also change the hyperedges. Attention operates downstream of an extraction that the adversary controls. HGANN-Mal therefore inherits the exposure of its representation, and in line with Chapter 6 this chapter makes no robustness claim.

Threats to validity. The result tables head their first column Macro-F1 without stating a convention. For the HGANN-Mal Drebin binary row a macro reading is impossible: over 1,112 malicious and 1,894 benign test applications, macro precision 97.8 and macro recall 97.6 at an accuracy that rounds to 98.3 would require at least 1,897 true negatives. That row is read as positive-class values. Under positive-class, macro and support-weighted averaging alike, $F_1 \leq (P + R)/2$, and four rows exceed that mean by more than the 0.1 point that rounding allows: GCN and HGNN in the Drebin binary panel, HGANN-Mal in the malware-family classification panel, and GCN in the malware-category classification panel. No single convention fits these rows, so the multiclass columns are not read as macro averages; accuracy is unaffected. On the data side, the benign Drebin applications come from AndroZoo, which began crawling in late 2011, while Drebin covers August 2010 to October 2012, so a classifier may separate the two collections by build tools or API levels that have nothing to do with malice [13, 162]. The source publication records that about 49 per cent of Drebin samples are repackaged applications, and none was removed. Near-duplicates across the partitions may

inflate the family scores of Section 5.11 more than the binary ones [15, 14]. CICMalDroid, for its part, keeps only applications that ran in its authors' sandbox [161], which may remove evasive samples that a static detector most needs to catch.

5.15 Chapter Summary

This chapter answered RQ3 by lifting a method-level function-call graph to a hypergraph, joining its vertices by three rules of different kinds, and learning how strongly each method participates in each group rather than fixing that strength when the structure was built. The system was measured against two pairwise graph operators and two hypergraph operators without attention, on Drebin and on CICMalDroid, for binary detection and for classification into families and into categories.

Two findings hold. The higher-order representation is better than the pairwise one on every task and corpus reported, by between three and eleven and a half points of accuracy depending on the task and on which pair is compared, with the input to both held identical. The attention-weighted system is ahead of the non-attentive hypergraph operators on three of four combinations and level on the fourth, and its widest margin appears on the multiclass task rather than on detection. The analysis of Section 5.8 sharpens what the second finding can mean without adding evidence for it. Attention reweights the memberships that the construction fixed, and within one layer it ranks the hyperedges of every method in one shared order. A constructed example shows that it can still separate structures that the fixed operator identifies, although nothing shows that training exploits this.

One thing does not hold, and the chapter states it rather than working around it. Because the hyperedge vocabulary and the weighting were changed together, the improvement over the hypergraph baselines cannot be attributed to attention, and the experiment that would separate them was not run. Chapter 7 takes the representational question up against Chapter 4, and Chapter 8 records the gap as the first item of future work.

Evaluating Adversarial Robustness: Benchmark Design and Metric Analysis

6.1 Research Problem and Contribution

This chapter addresses RQ4: How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness? The question concerns gap G4, which links adversary assumptions to the validity of the resulting measurements.

Adversarial machine learning studies how deliberate changes to training data or inference inputs can cause a learning system to fail, together with methods for resisting such changes [18]. A poisoning attack intervenes during training, whereas an evasion attack modifies an input presented to an already trained model. Defences may expose the model to adversarial examples during training, transform its inputs or internal representations, or detect and respond to suspected adversarial inputs. This chapter concerns evasion attacks and the evaluation of these three defence strategies. The broader attack taxonomy and terminology are defined in Section 2.5.

An empirical robustness estimate depends on the permitted perturbations and the attacks used to search for them [104, 21]. An unsuccessful search may reflect resistance to the tested attack, or a failure of the attack to exploit the model. For example, Athalye et al. identified obstructed gradients in seven of nine non-certified defences accepted at ICLR 2018 and circumvented six completely and one partially [19]. Evaluators therefore need to examine the measurement procedure alongside the defended classifier.

The SPARTA SAFAIR work produced two complementary evaluation mechanisms: an adversarial machine-learning competition and a reusable benchmark framework [163, 39]. The competition evaluated submitted defences on hidden test samples. The benchmark provided common interfaces for evaluating trained models against reference attack implementations, with task-specific components for different prediction problems. Its versioned source code is publicly available on GitHub [164].

Using the project records, this chapter reconstructs both evaluation mechanisms and assesses whether the reported measurements are consistent with the documented attack objectives, attacker access and perturbation constraints. The empirical evidence comprises the contest submissions and the application of the benchmark to partner defences on histopathology

images. The chapter then derives properties of the accuracy-loss score and the binary detector response policy. These analytical results establish what the published aggregates support and what additional observations would be required to distinguish competing explanations. A short comparison with the project's PDF-malware study examines how feature constraints alter the evaluation problem.

SPARTA was funded under Horizon 2020 grant agreement No. 830892. Its seventh work package, SAFAIR, concerned the security of artificial-intelligence systems. The candidate wrote the whole of D7.6, which reports the contest and benchmark, and Chapter 5 of D7.5 [39, 38]. Section 6.10 distinguishes this contribution from the partner methods and experiments discussed below.

6.2 Threat Analysis and Evaluation Scope

SPARTA's threat analysis links attack techniques to affected ML assets and the adversary's knowledge [37, Sections 3.3.2–3.3.6]. Applied to the evaluations in this chapter, these distinctions locate the attack within the ML life cycle. Poisoning concerns training data or the learning process; evasion concerns inputs supplied to a trained model.

The evaluations examined here concern evasion. The contest specified attacks that induce a chosen identity or errors in selected facial attributes [163, Section 3.3]. The histopathology benchmark reports classification accuracy on clean and perturbed images [39, Section 3.1.3]. The affected asset is therefore the inference input, and the measured outcome is prediction correctness under attack. Sections 6.3 and 6.4 specify attacker access and permitted perturbations, which determine the conditions under which these measurements can be interpreted.

6.3 Evaluation Requirements and Threat Models

The threat model specifies the attack objective, the information available to the attacker and the operations permitted on the input. Section 2.5 defines the white-box, black-box and grey-box terminology used throughout the dissertation. Under its grey-box convention, some model information is known and the training process cannot be influenced. Knowledge of training data and the ability to alter those data are separate assumptions.

The contest granted knowledge of the training corpus, withheld the defence technique and model architecture, and allowed gradient-based updates against the deployed model [163, 165]. Access to gradients is an oracle capability. The term grey box alone therefore leaves the experimental interface underspecified. The conditions stated in Section 6.4, including the query interface and input budget, define the contest adversary used here. This is an academic evaluation setting; a survey of 271 practitioners found that academic knowledge assumptions often grant more information than attackers have in deployment [107].

D7.3 adopted the evaluation checklist of Carlini et al. It called for attacks adapted to the complete defence, explicit attack hyperparameters and comparisons with transfer or gradient-

free attacks. It also requested perturbation-budget curves and ablations [163, 21]. These requirements provide a basis for assessing the delivered results. Participation by independent attack teams can broaden the search, but the scoring rule and the validity of each generated input require separate checks.

6.4 Tasks and Evaluation Protocol

6.4.1 Prediction Tasks and Permitted Perturbations

The contest defined two tasks, each with an attack track and a defence track: targeted face re-identification and attribute alteration. In targeted face re-identification, the attacker modifies a face image so that the classifier assigns the image to a specified target identity. In attribute alteration, the attacker modifies an image so that the classifier misclassifies a nominated subset of its forty binary facial attributes. The corresponding defence tracks evaluated model robustness against the same perturbations [163]. A participant could attack one task and defend the other, but could not enter both sides of the same task.

Let f_θ denote the model scores and h the resulting class decision. For an image x in a specified input coordinate system, define the feasible perturbations by

$$\mathcal{S}_\varepsilon(x) = \{x' \in \mathcal{X}_{\text{valid}} : \|x' - x\|_\infty \leq \varepsilon\}. \quad (6.1)$$

Here $\mathcal{X}_{\text{valid}}$ includes the permitted pixel range and quantisation. For pixel values scaled to $[0, 1]$, the contest budget was $\varepsilon = 8/255$, corresponding to a maximum change of eight intensity levels per channel in the discrete 0–255 encoding. The rules required discrete output images, deterministic and stateless submitted models, and an upper limit of four hours for generation of 1,000 perturbed examples [39]. The report identifies two 12 GB NVIDIA GPUs, a Titan X Pascal and a GeForce GTX, with a 32-core AMD CPU and 128 GB RAM. Container execution used Ubuntu 18.04 and CUDA 10.1. These are reported resource limits, not measured per-example runtimes.

The budget must be interpreted before any model-specific normalisation. For example, a linear rescaling from $[0, 1]$ to $[-1, 1]$ doubles the numerical distance. A reproducible evaluation therefore records both the input coordinate system and preprocessing.

6.4.2 SAFAIR Contest Scoring and Evaluation Protocol

SAFAIR Contest ranked submissions by the change between clean and perturbed accuracy,

$$\Delta = A_{\text{initial}} - A_{\text{final}}. \quad (6.2)$$

Attacks sought the largest Δ and defences the smallest. Unless stated otherwise, accuracies and Δ are proportions in this chapter; 100Δ is a difference in percentage points. D7.5 specifies clean accuracy and runtime as tie-breakers [38]. These do not prevent a low-utility model with a smaller Δ from outranking a more accurate model.

Table 6.1. Planned and reported SAFAIR evaluation procedures.

	Plan [163]	Reported delivery [38, 39]
Participants	attack and defence tracks for both tasks	defence submissions only
Attack selection	organiser baselines, then five selected participant attacks	four named organiser baselines
Defence score	weighted opening Δ ; mean final Δ over opponents	aggregate best and worst perturbed accuracies; Δ from the best value
Final comparison	five attacks against five defences	no participant-attack round
Reproduction	hidden evaluation set and submitted containers	sampling code and tool descriptions; incomplete run-level records

The planned opening round evaluated defences against FGSM, BIM and PGD, with respective weights 0.2, 0.4 and 0.4 [163]. FGSM makes one update in the sign of the input-loss gradient [114]. BIM repeats smaller FGSM-style updates and clips each result to the permitted input range and perturbation neighbourhood [166]. PGD also applies iterative gradient updates, projects each result onto the feasible perturbation set, and can use multiple starting points [115]. Carlini–Wagner instead formulates adversarial-example generation as an optimisation problem that balances the required classification outcome against perturbation size [167]. The five selected attacks would then face the five selected defences, with the final score averaged over opponents [163]. No attack submissions arrived. The organisers used fixed baseline attacks instead; D7.5 lists Carlini–Wagner in addition to FGSM, BIM and PGD [38]. Table 6.1 records the change in protocol.

The lack of independent attacks reduced the diversity of the evaluation. It does not explain all weaknesses of the resulting measurement. As derived in Section 6.8.4, a constant classifier has $\Delta = 0$ against any set of label-preserving attacks, including the opponents in the planned final round. The scoring defect and the attack-coverage defect are therefore distinct.

The contest drew inspiration from the NIPS 2017 competition, whose defence score was the proportion of correctly classified attacked images averaged across attacks [168]. That score differs from Equation 6.2. The competitions also differed in task and participation, so their results do not isolate the effect of independent contestants. Details of the schedule are retained in the cited project records; the scientific comparison here concerns the scoring and attack interfaces.

6.5 Data and Evaluation Measures

6.5.1 Corpora and Hidden Evaluation Samples

The contest used CelebA for model training, a corpus described by Liu et al. as approximately 200,000 images of 10,000 identities, with forty binary attributes per image [169]. For hidden attribute evaluation the organisers used LFWA, the forty-attribute annotation of Labeled Faces in the Wild, which contains 13,233 images of 5,749 identities [170, 169]. A separate corpus

Table 6.2. Evaluation units and data provenance. Stated sample sizes describe the printed procedures, not recovered per-example result files.

Evaluation	Input and prediction	Source and sampling
Contest identity	one image; one identity label	augmented CelebA; 1,000 selected test records [39]
Contest attributes	one image; forty binary labels	LFWA; 1,000 selected images [39]
Partner benchmark	one 50×50 histopathology patch; IDC/non-IDC	training, validation and test subsets described; evaluated subset size and patient separation unreported [171]
PDF comparison	21 tag counts; benign/malicious	20,000 files; 45/45/10 per cent defender/proxy/crafting split [38]

reduces direct image reuse, but does not establish equality of image distributions or attribute prevalences.

Re-identification required identity labels compatible with the trained classifier. The organisers therefore used CelebA images under a random choice of colour jitter, greyscale conversion, horizontal reflection or rotation of at most five degrees [39]. The printed procedure applies a face-detection check and retries failed transformations before excluding an image. The resulting selection may differ from an unfiltered sample. The report recognises that participants could have used similar augmentations during training. Augmentation alone does not demonstrate independence from training images.

6.5.2 Accuracy and Task-Specific Attack Success

For N examples with identity labels y_i , the decision is $h(x_i) = \arg \max_k f_{\theta,k}(x_i)$. Attribute decisions are $h_j(x_i) = \mathbf{1}[f_{\theta,j}(x_i) \geq 0]$ when the outputs are logits, with equivalent threshold 0.5 for sigmoid probabilities. The corresponding accuracies are

$$A_{\text{id}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[h(x_i) = y_i], \quad A_{\text{attr}} = \frac{1}{40N} \sum_{i=1}^N \sum_{j=1}^{40} \mathbf{1}[h_j(x_i) = y_{ij}]. \quad (6.3)$$

These conventions follow the task-specific scoring code printed in D7.6. Attribute accuracy counts individual attribute decisions; it does not require all forty predictions for an image to be correct [39].

Accuracy loss does not identify success against a specified target. For identity targets $t_i \neq y_i$, let $I_t = \{i : h(x_i) = y_i, t_i \neq y_i\}$ contain eligible, initially correct examples. Given valid attacked inputs x'_i , conditional targeted success is

$$\text{ASR}_{\text{target}} = \frac{1}{|I_t|} \sum_{i \in I_t} \mathbf{1}[h(x'_i) = t_i]. \quad (6.4)$$

A prediction of a third identity reduces accuracy without satisfying the targeted objective. An empty eligible set makes this rate undefined and its size must be reported.

For a nonempty nominated attribute subset J_i , define $I_J = \{i : h_j(x_i) = y_{ij} \text{ for all } j \in J_i\}$. One explicit success criterion is

$$\text{ASR}_{\text{subset}} = \frac{1}{|I_J|} \sum_{i \in I_J} \prod_{j \in J_i} \mathbf{1}[h_j(x'_i) \neq y_{ij}]. \quad (6.5)$$

This requires errors on every nominated attribute of an initially correct subset. Reporting $|I_J|$ separates induced errors from those already present; the rate is undefined if $|I_J| = 0$. Equations 6.4 and 6.5 specify measurements needed to evaluate the stated objectives; they are not additional scores recovered from the contest. If an attack flips all k initially correct bits in the nominated subset and leaves all other decisions unchanged, the per-image contribution to attribute accuracy falls by only $k/40$. For $k = 1$ this is 2.5 percentage points, although the specified attack succeeds.

6.5.3 Aggregation over Attacks

An attack suite can fail on different examples under each method. Selecting the lowest aggregate accuracy does not account for that complementarity. Following per-example evaluation guidance [21], let $\mathcal{A}_\varepsilon$ be a finite set of attacks whose outputs satisfy Equation 6.1. For a single-label task, define

$$\hat{A}_{\mathcal{A}}(\varepsilon) = \frac{1}{N} \sum_{i=1}^N \min_{a \in \mathcal{A}_\varepsilon \cup \{\text{id}\}} \mathbf{1}[h(a(x_i)) = y_i]. \quad (6.6)$$

The identity transformation includes clean errors in the score. The suite must respect the same threat model for each defence. Because it searches only a subset of valid perturbations, its surviving accuracy is an upper bound on worst-case accuracy on these examples, not a certificate of robustness. Moreover,

$$\hat{A}_{\mathcal{A}}(\varepsilon) \leq \min_{a \in \mathcal{A}_\varepsilon \cup \{\text{id}\}} \frac{1}{N} \sum_i \mathbf{1}[h(a(x_i)) = y_i], \quad (6.7)$$

since each per-example minimum is no greater than its value for any fixed attack. The aggregate tables do not identify which examples fail under which attack, so this suite score cannot be calculated retrospectively from them.

6.5.4 Biometric Data and Data Protection

The contest processed photographs of identifiable people and predicted identities or facial attributes.

Attribute imbalance also affects the scientific interpretation. The SD account describes a constant attribute predictor whose accuracy remains high because some attributes are rare [39]. A complete evaluation would therefore report attribute prevalences and per-attribute outcomes alongside the aggregate score. Demographic performance differences cannot be inferred from the reported aggregates.

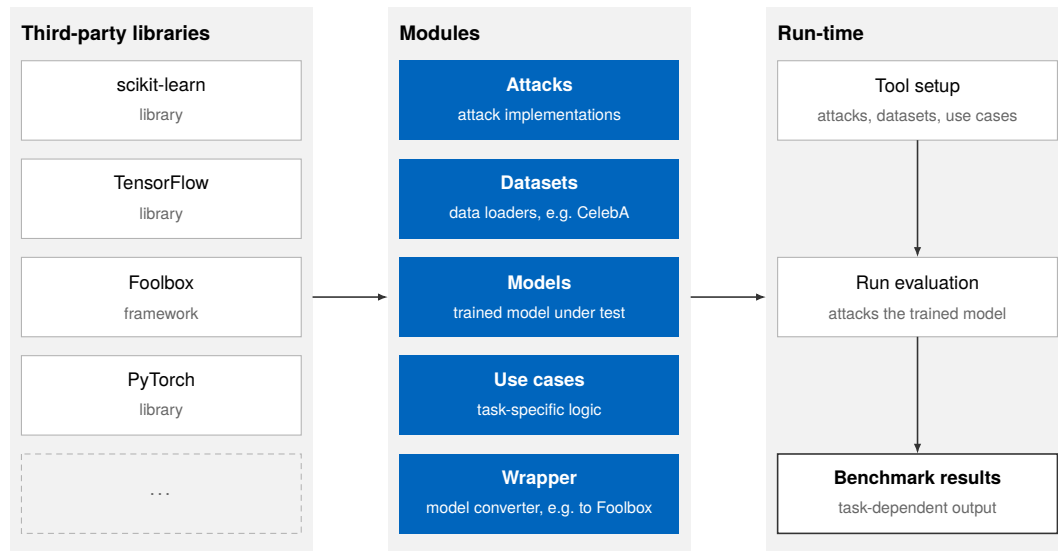


Figure 6.1. Benchmark architecture, redrawn after D7.6 Figure 2, p. 22 [39]. Box labels follow the original. The five modules separate task definitions and data loading from attack execution.

6.6 Benchmark Architecture and Reproducibility

The benchmark implements a common software procedure for adversarial evaluation. It separates attack execution from the task-specific model and data interfaces. D7.3 proposed five modules, which D7.6 reports as implemented: attacks, datasets and models, together with use cases and a wrapper [163, 39]. Figure 6.1 redraws the published architecture.

Within this framework, standardisation concerns the component interfaces and execution procedure. A run loads a trained checkpoint and evaluation data, converts the model to the attack-library interface, measures performance before and after the selected attacks, and emits the accuracies in one JSON-formatted result. Researchers can extend the procedure through new attack classes and task-specific components. The implementation of this reusable workflow is available in the versioned GitHub repository [164].

The wrapper exposes a model through Foolbox, whose reference implementations supply the attacks [172]. PyTorch is the native model interface; the report also describes conversion of TensorFlow or Keras models. Use-case classes select the prediction criterion and loss for the task. A new attack extends a base class and is discovered by the loader; a configuration list excludes named attacks. These interfaces permit reuse across models. Compatibility still depends on the output semantics: the report’s multi-label wrapper handles fixed-budget attacks, while some minimum-distance interfaces are omitted [39].

D7.6 also contains a description of splitting data and training classifiers, alongside repeated statements that weights must already exist. The documented invocations require checkpoints, and the inspected public implementation loads trained models [164]. Accordingly, the benchmark is described here as an evaluation tool. Its public source is separate from the contest development toolkit [40].

Perturbation validation. The public revision identified by commit 2fac62b947ed pins Foolbox 3.3.1. Its attack call returns a raw candidate and a budget-clipped candidate, with a success indicator. However, lines 134–135 of `main.py` pass the raw candidate to the accuracy function [164]. In Foolbox’s minimisation interface the clipped candidate is obtained by restricting the raw perturbation to the requested distance budget; the raw output need not satisfy that budget [173].

A further distinction concerns targeted runs. The same implementation replaces the reference labels with the nominated target before computing its fields labelled accuracy. Those fields then measure target-label agreement, with a different interpretation from true-label accuracy [164]. Each output therefore needs an explicit task, reference label and denominator.

6.7 Defence Mechanisms and Integration

The evaluated configurations instantiate the three defence strategies introduced in Section 6.1: adversarial training, transformations of inputs or internal representations, and adversarial-input detection.

D7.6 evaluates five partner configurations described in D7.2. The underlying histopathology classifier combines a pretrained VGG16 feature extractor with a dense prediction network [171, 39]. Write its scores as $f_\theta(x) = c_\psi(\phi_\omega(x))$, where ϕ_ω is the convolutional feature extractor and c_ψ the prediction head. The following notation makes the intervention of each defence explicit without supplying missing layer configurations.

6.7.1 Training and Representation Transformations

The adversarial-training configuration retrains the classifier with clean and perturbed examples. D7.6 identifies FGSM, BIM and PGD as the attacks used to generate its adversarial training examples [39]. This method exposes the model to those attacks during training; resistance to other attacks remains an empirical question.

For the dimensionality-reduction variants, let r_η denote an autoencoder. The two configurations have score functions

$$f_{\text{top}}(x) = c_\psi(\phi_\omega(r_{\eta_x}(x))), \quad f_{\text{mid}}(x) = c_\psi(r_{\eta_z}(\phi_\omega(x))). \quad (6.8)$$

The first transforms pixels and the second transforms extracted features. D7.2 states that the original convolutional network and prediction head retain their weights in these autoencoder variants. Its separate encoder-only variant trains a new prediction head, but that variant is not a column of the D7.6 benchmark table [171]. Figure 6.2 redraws the two published placement diagrams in this notation.

A smaller reconstruction error alone does not establish robustness of either composition. The relevant attack must optimise against the complete transformed classifier, including its

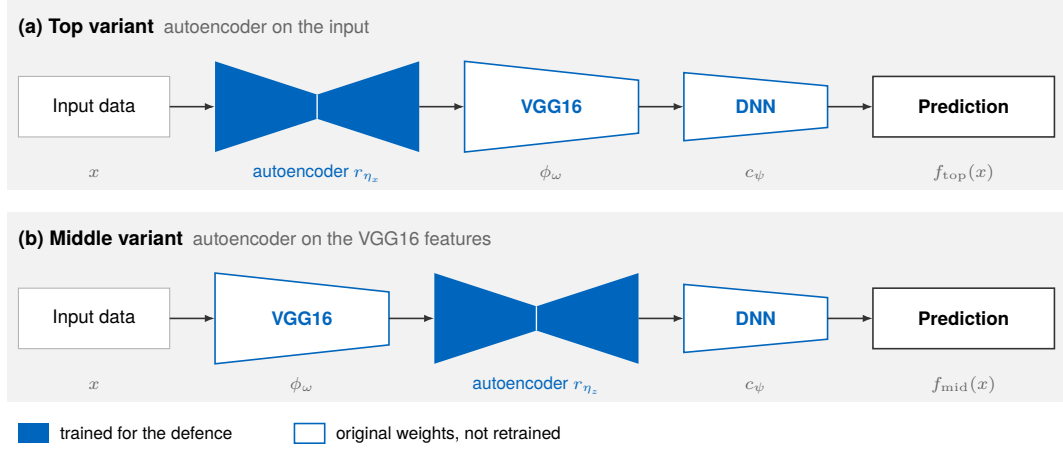


Figure 6.2. Autoencoder placement in the partner classifier, redrawn after D7.2 Figures 13 and 11, p. 14 [171]. In the top variant (a), the autoencoder r_{η_x} transforms the input before feature extraction. In the middle variant (b), r_{η_x} acts between the feature extractor ϕ_ω and the prediction head c_ψ , as in Equation 6.8. The legend follows D7.2, which keeps the original weights of VGG16 and the dense network in both variants. The diagrams specify placement, not a matched ablation of all training choices.

preprocessing. Reusing examples generated against the original classifier tests a different condition from attacking the defended composition [21].

6.7.2 History and Activation Detectors

Figure 6.3 shows the two partner detectors and their response paths.

Prediction similarity adds a history of queries and their predictions. The partner study records user information with image similarity and alarm statistics, and reports SSIM as the similarity measure [171]. Its alarm can be written $d_t = d(x_t, H_{t-1})$, where H_{t-1} contains the preceding queries associated with the monitored interaction. A sequence of related optimisation queries may then trigger an alarm. A transferred adversarial example submitted once presents a different observation to this stateful detector.

For reproducibility, a history-based evaluation must specify the sequence order, user association and reset policy. The contest’s stateless-submission rule does not describe this benchmark configuration. The reports do not establish a common history protocol across the attacks in the result table, so their differences cannot be attributed solely to whether an attack uses gradients.

The activation detector instead characterises the response of the underlying classifier to an input. For each of nineteen hidden layers it calculates nine statistics: mean and median; skewness and kurtosis; standard deviation and interquartile range; minimum and maximum; and the number of nonzero activations [171]. With layer activations $a_\ell(x)$ and the statistic vector s , the representation is

$$z(x) = \text{concat}_{\ell=1}^{19} s(a_\ell(x)) \in \mathbb{R}^{171}. \quad (6.9)$$

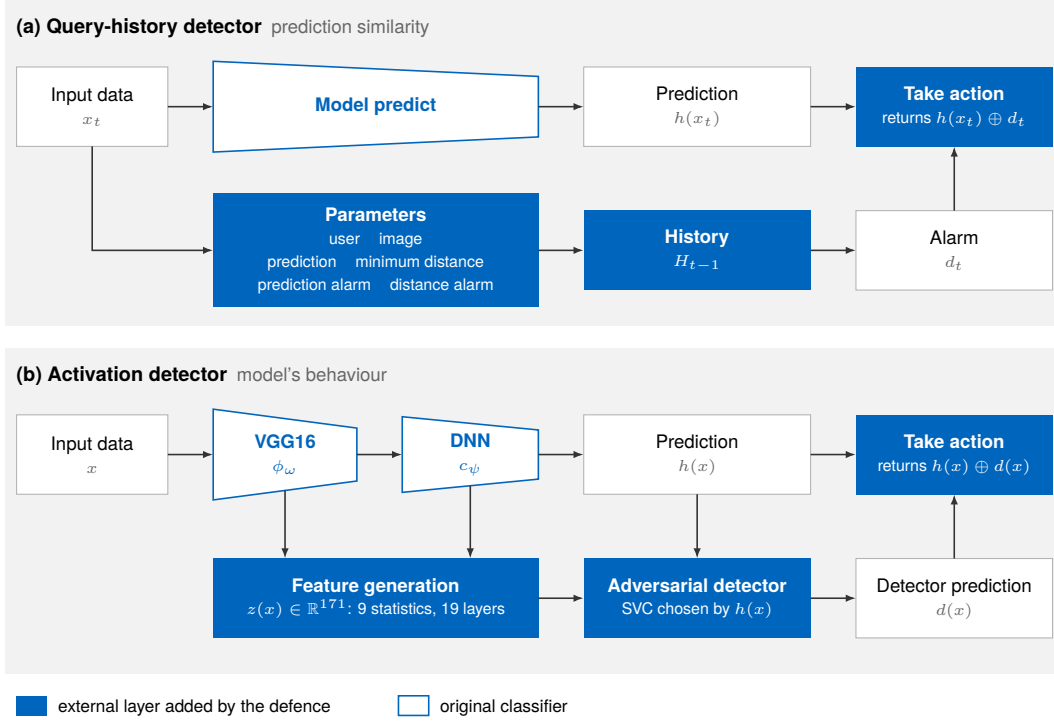


Figure 6.3. Partner detection mechanisms, redrawn after D7.2 Figures 15 and 16, p. 16 [171]. Panel (a) uses preceding queries, whereas panel (b) uses internal activations of the classifier. The recorded parameters, the activation features of Equation 6.9 and the per-class detector follow D7.2. The response is the label reversal of Equation 6.10, which D7.6 applies in the benchmark [39]. The final action is part of the defended system and must be included when task accuracy is measured.

After standardisation and feature selection, two support-vector classifiers are trained, one for each class predicted by the original model. The selected detector therefore depends on that prediction. The 94% detector accuracy reported in D7.2 concerns detection of adversarial inputs; it is not the defended task accuracy of the D7.6 table.

6.7.3 From an Alarm to a Classification

To place the detectors in an accuracy table, D7.6 uses the binary response policy of returning the opposite class whenever the detector raises an alarm [39]. Let $h(x) \in \{0, 1\}$ denote the base prediction, $d(x) \in \{0, 1\}$ the alarm and $g(x)$ the returned class. For a fixed history, where applicable,

$$g(x) = h(x) \oplus d(x), \quad (6.10)$$

where $\oplus$ denotes exclusive OR. This rule corrects an erroneous binary prediction when the detector raises an alarm, but turns a correct prediction into an error under the same alarm. Consequently, on a common labelled evaluation set,

$$A(g) - A(h) = \Pr(d = 1, h \neq y) - \Pr(d = 1, h = y). \quad (6.11)$$

The probabilities denote empirical fractions on that set. To obtain the identity, partition the observations by the alarm and the correctness of h : unflagged predictions are unchanged,

Table 6.3. Reported contest scores, including the worst-perturbed column, from D7.6 Table 1, p. 16 [39]. Task names follow that table. The SD task label conflicts with its detailed attribute-model description; best and worst values are not identified as paired outcomes for one checkpoint under a common attack suite.

Rank	Team	Task	Clean	Best	Worst	Δ
1	SD	re-identification	0.87	0.85	0.67	0.02
2	Vicomtech	re-identification	0.85	0.72	0.72	0.13
	Vicomtech	attribute alteration	0.80	0.65	0.65	0.15
3	BPI	re-identification	0.84	0.68	0.68	0.16

flagged errors are corrected, and flagged correct predictions are reversed. If $d(x) = 1$ for every observation, then $A(g) = 1 - A(h)$, while g can still depend on the input.

Detection accuracy alone does not determine either term in Equation 6.11. A useful evaluation records alarms jointly with base and returned predictions, and measures false alarms on clean inputs. Abstention would be a different response policy, for which coverage and error among accepted predictions must be reported [21]. The historical benchmark reports the reversal policy; its aggregate accuracies do not identify the two joint fractions.

6.8 Results and Metric Analysis

6.8.1 Contest Evidence

Table 6.3 preserves the numerical columns of D7.6 Table 1. The source reports one row per ranked team and task, rather than one row per submitted checkpoint. Its surrounding text describes best and worst scores across submissions, but does not link each value to an attack and checkpoint [39].

The SD row reports 0.85 as best perturbed accuracy and 0.67 as worst, a spread of eighteen percentage points. The printed $\Delta = 0.02$ uses the best value. Subtracting 0.67 from 0.87 would give 0.20, but it would not establish a worst-case loss for the same model without the missing checkpoint association. The report’s prose also substitutes 87% for attacked accuracy and gives 61% and 88% values that do not match its table. The numerical record used here is the table, with those inconsistencies retained as limits on interpretation.

The counts of submissions also disagree. D7.5 states seven defence submissions, six for re-identification and one for attributes, whereas D7.6 states six submissions in total [38, 39]. An unscored ITTI account appears in D7.6, but the reports do not state that this explains the difference. Similarly, D7.5 gives SD a zero Δ while D7.6 Table 1 gives 0.02. The development accounts describe models compatible with different scores, but do not supply the identifiers needed to reconcile the two reports.

The four rows do not establish a general relation between clean performance and robustness. They combine different prediction tasks and selected submissions from three teams. No regression or significance test is appropriate for this table. D7.6 mentions ten-fold cross-

validation in the Vicomtech and BPI accounts, but does not provide fold assignments or fold-level outcomes. Those statements cannot supply uncertainty estimates for the printed scores.

6.8.2 SD Development Configurations

D7.6 describes three SD models. Its introductory text assigns them to re-identification, while the model details specify forty attribute outputs. The high accuracy of a constant attribute vector cannot be interpreted as a demonstrated identity-classification result. The following analysis therefore follows the attribute-model description and leaves its mapping to Table 6.3 unresolved [39].

The first configuration was a convolutional network trained on PGD examples with $\varepsilon = 0.3$, approximately 9.6 times the contest's 8/255 budget if both use $[0, 1]$ pixels. The team reports that training produced the same output for every input, with approximately 80% clean attribute accuracy and zero accuracy loss. It attributes that accuracy to the prevalence of absent attributes and recognises a weakness in the dataset and contest rules. This is the source's direct account of prediction collapse, although neither the checkpoint nor individual predictions accompany the reported result.

The second configuration combines six EfficientNet and three pruned ECA-ResNet models. Their averaged outputs are thresholded to obscure gradients. For a nonzero scalar attribute logit u , write the transformation as

$$q(u) = \begin{cases} 10, & u > 0, \\ -10, & u < 0. \end{cases} \quad \mathbf{1}[q(u) \geq 0] = \mathbf{1}[u \geq 0], \quad q'(u) = 0 \quad (u \neq 0). \quad (6.12)$$

The source does not specify the exact-zero tie. Away from that tie, the operation preserves the binary decision and suppresses its direct derivative. It can obstruct a gradient-based attack without changing the classifier's decision regions. Attacks adapted to the non-differentiable operation or transferred from another model remain relevant [19, 20].

The second configuration achieved approximately 92% clean accuracy and 79% under transferred attacks in the team's development account. The third used one pruned ECA-ResNet, added adversarial training, and retained the threshold operation. Its reported clean accuracy was approximately 87%, with 84–85% under transferred attacks [39]. Figure 6.4 shows these values separately from the contest table.

The third configuration trades five percentage points of reported clean accuracy for five to six points of transferred-attack accuracy relative to the second. This is a descriptive comparison between systems: ensemble size and training changed together, and the attacks' individual settings are not given. It does not isolate the effect of adversarial training. The transfer results nevertheless show why resistance to a direct gradient attack is an incomplete measure of these constructions.

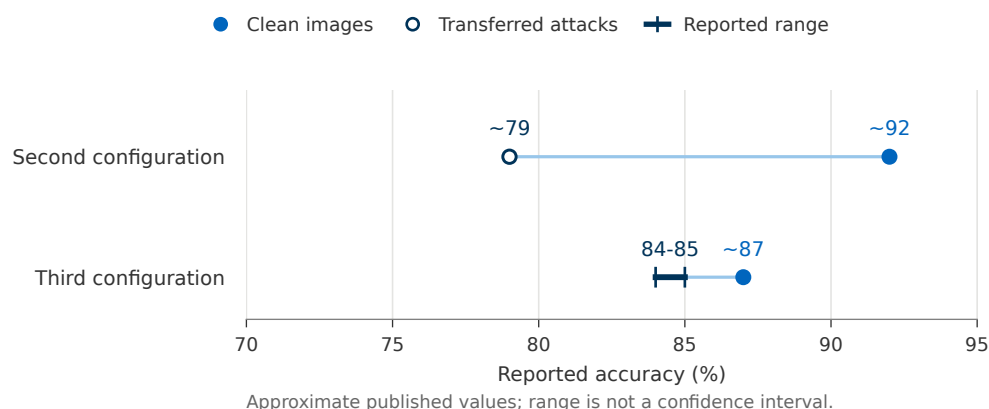


Figure 6.4. SD development configurations from D7.6 Sections 2.8.1.2–2.8.1.3, pp. 18–19 [39]. The interval 84–85% is a reported approximate range, not a confidence interval. The source describes attribute outputs despite its re-identification heading. These values are not additional official contest rows.

Table 6.4. Published accuracies for the five partner defence configurations, from D7.6 Table 2, pp. 28–29 [39]. Attack hyperparameters and the evaluated subset are unspecified. Each entry is a reported aggregate, with no dispersion estimate.

Attack	Adv. training	Top AE	Middle AE	Pred. similarity	Activations
none (clean)	0.844	0.726	0.817	0.844	0.708
FGSM	0.837	0.716	0.812	0.837	0.708
L_2 basic iterative	0.844	0.725	0.817	0.842	0.708
BIM	0.837	0.716	0.812	0.837	0.708
DeepFool	0.003	0	0.001	0.015	0.708
additive noise	0.844	0.725	0.817	0.844	0.708
NewtonFool	0.464	0	0	0.460	0.708
PGD	0.837	0.717	0.813	0.837	0.708

6.8.3 Histopathology Benchmark

D7.6 reports that Vicomtech used the benchmark tool to evaluate five defence configurations on a histopathology classification task [39, Section 3.1.3]. This application documents use of the framework for a prediction problem distinct from the competition’s facial image tasks. Table 6.4 reproduces the reported accuracies for binary IDC classification. For the two detectors, these include the response policy in Equation 6.10. The clean row records the prediction cost associated with each configuration.

The middle autoencoder exceeds the top autoencoder by 9.1 percentage points on clean data and 9.6 points under the reported PGD condition. Its PGD loss is 0.4 points, compared with 0.9 for the top variant. These values favour the middle configuration within the reported evaluation. They do not establish that placement alone caused the difference, because the complete training configurations and repeated-run outcomes are unavailable.

Adversarial training and prediction similarity have equal reported clean accuracy and equal FGSM, BIM and PGD values. Prediction similarity is higher under DeepFool by 1.2 percentage points, but lower under NewtonFool by 0.4 points. It therefore does not dominate adversarial training across the table. The source’s explanation that similarity detects gradient-based

construction is insufficient to explain the results, since DeepFool and NewtonFool also use gradient information. Query sequences and detector alarms would be needed to test that mechanism.

The activation column remains at 0.708, including on clean inputs. This establishes equality of the printed aggregates to their reported precision. It does not establish identical predictions, an always-active detector or insensitivity to inputs. Equation 6.11 also shows that always reversing predictions would not, by itself, yield the class prior. The mechanism behind the column remains unidentified.

The near-zero DeepFool results for four configurations require a different qualification. Minimum-distortion attacks search for an adversarial input and should be compared by the distortion required, or by success after imposing a common valid budget [174, 21]. Neither the norm and stopping settings nor achieved distortions are reported for these benchmark runs. The table therefore cannot establish that its attack rows represent equally constrained adversaries. The code finding in Section 6.6 reinforces the need to validate distances, but does not resolve the historical configuration. The additive-noise row is a useful control condition; without its distribution, budget and number of trials, its small changes do not establish a comprehensive random-search check.

6.8.4 Sensitivity to the Scoring Rule

Figure 6.5 subtracts each attacked accuracy from that configuration’s clean accuracy. It makes the score dependence visible without changing the original results. Under PGD, the activation detector has the smallest Δ , zero, but its attacked accuracy is 12.9 percentage points below adversarial training. Similarly, the middle autoencoder has a smaller loss than adversarial training while its attacked accuracy is 2.4 points lower. A smaller loss alone therefore does not identify the more useful defended classifier.

For paired clean and attacked examples, let C denote an initially correct prediction and C' a correct prediction after attack. Expanding the two accuracy terms yields

$$\Delta = \Pr(C \cap \neg C') - \Pr(\neg C \cap C'). \quad (6.13)$$

Both accuracies contain the correctly classified observations in $C \cap C'$, which cancel. Thus, Δ measures the net change in correctness. It is not the conditional fraction of initially correct examples made incorrect. Equal numbers of harmful and corrective changes can leave $\Delta = 0$ despite changes in individual outcomes. The aggregate table cannot determine those two counts.

Constant-prediction result. For any constant classifier $h_c(x) = c$ and label-preserving attack a , $h_c(a(x_i)) = h_c(x_i)$ for each observation. Therefore,

$$A_{\text{clean}}(h_c) = A_{\text{attack}}(h_c) = \hat{p}_c, \quad \Delta(h_c) = 0, \quad (6.14)$$

where $\hat{p}_c$ is the evaluated class fraction. The statement holds for each attack and any weighted average over attacks. Independent opponents cannot eliminate this property of the score. The

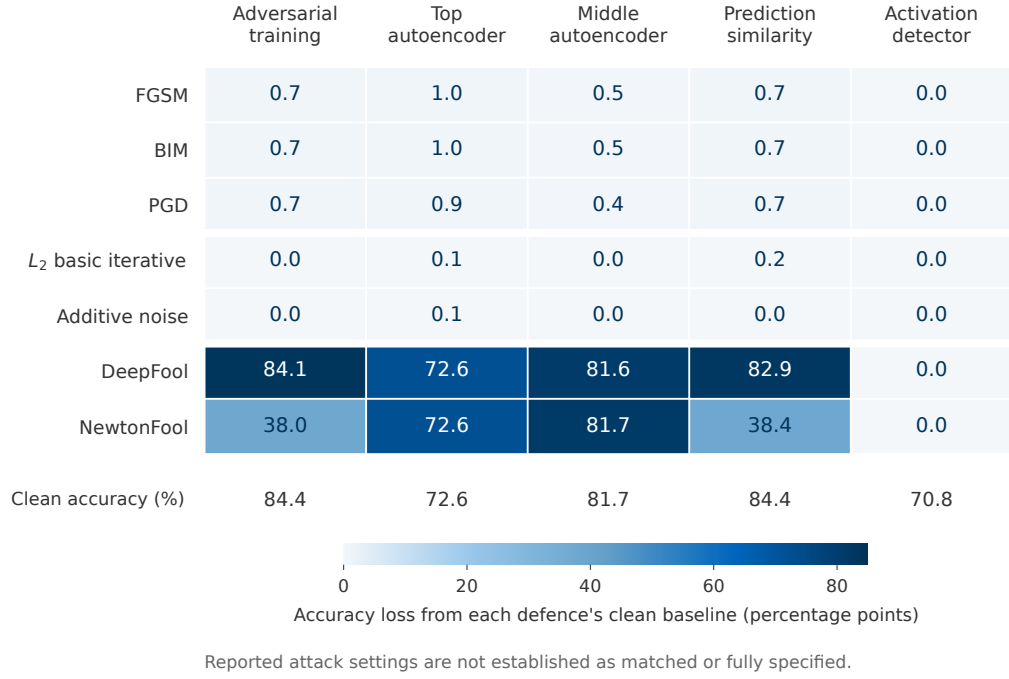


Figure 6.5. Accuracy losses computed from D7.6 Table 2 [39]. Each cell is $100(A_{\text{clean}} - A_{\text{attack}})$ in percentage points; the original accuracies remain in Table 6.4. The separation of attack rows does not imply matched budgets. In particular, zero loss for the activation detector describes its aggregate score, not a demonstrated mechanism or robustness certificate.

result establishes an indifference to prediction utility when ranking only by loss; it does not imply that the constant classifier correctly classifies the other classes.

For attributes, choose a fixed vector m using training-set majority labels. Its held-out accuracy is

$$A_{\text{attr}}(m) = \frac{1}{40} \sum_{j=1}^{40} [m_j \hat{p}_j + (1 - m_j)(1 - \hat{p}_j)], \quad \Delta(m) = 0, \quad (6.15)$$

where $\hat{p}_j$ is the prevalence of attribute j in the evaluation set. When the training and evaluation majorities agree, each summand is $\max(\hat{p}_j, 1 - \hat{p}_j)$. This explains how a fixed attribute vector can have high average accuracy on an imbalanced corpus. The baseline labels must be fixed before evaluation; selecting them from hidden test labels would use unavailable information.

These identities support reporting clean utility together with attacked performance and a matched constant baseline. Prediction distributions and paired error transitions provide additional checks. None of these quantities can be reconstructed uniquely from an unchanged accuracy column, so the activation result remains an observation rather than a second demonstration of the SD collapse mechanism.

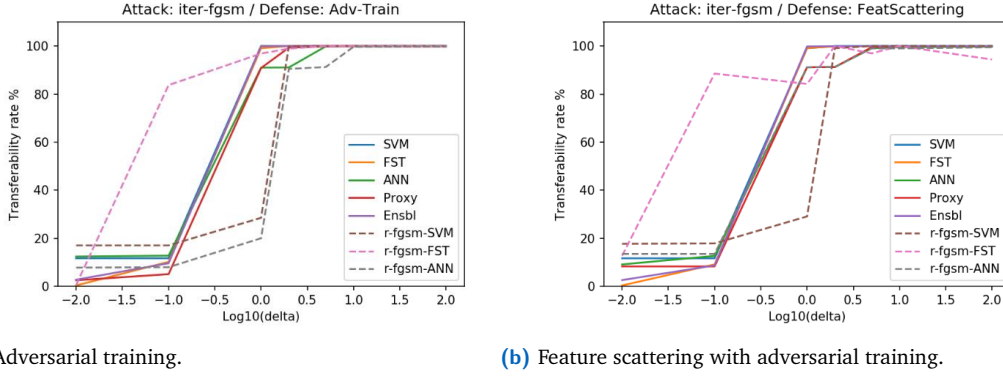


Figure 6.6. Original iterative-FGSM transfer curves from the partner PDF-malware study, D7.5 Figure 15, p. 23 [38]. Both panels retain the published legends and values. The horizontal axis is the source’s $\log_{10}(\delta)$ perturbation budget, distinct from the accuracy loss Δ used in this chapter. These are feature-space results, with no uncertainty bands or demonstrated construction of modified PDFs with preserved malicious functionality.

6.8.5 Comparison with Constrained PDF-Malware Features

The public D7.5 report contains a partner study of evasion against PDF-malware classifiers [38]. It provides a comparison with the continuous image setting, rather than an additional experiment on the Android systems in this dissertation. The study uses 10,000 benign and 10,000 malicious Contagio files. PDFiD extracts 21 tag counts; the data are partitioned into 45% for defender training, 45% for proxy training and 10% for adversarial crafting. A neural proxy generates examples transferred to neural, SVM and random-forest classifiers, without queries to the target during generation.

The allowed changes increase tag counts and preserve integer values after inverse standardisation and rounding. If $v \in \mathbb{N}_0^{21}$ is the original count vector, the reported feature constraints require

$$v' \in \mathbb{N}_0^{21}, \quad v'_j \geq v_j \quad (j = 1, \dots, 21). \quad (6.16)$$

A perturbation bound also applies in the attack representation. Its interpretation depends on standardisation and the subsequent rounding. These constraints restrict the feature search, but do not establish that a corresponding modified PDF was constructed or that its malicious functionality was preserved. The distinction is the feature-space versus problem-space distinction of Section 2.5.

The retained legend names the SVM, random forest (FST), neural network (ANN) and their ensemble (Ensbl); the prefix r-fgsm denotes adversarial retraining with FGSM examples. Proxy measures success against the attack’s source model, rather than cross-model transfer. In the feature-scattering condition the SVM and random forest receive adversarial augmentation, so that panel represents a hybrid defence comparison [38].

In Figure 6.6, transfer success for many reported models rises sharply as the budget increases, and several approach 100% at larger budgets. Other curves are non-monotone. Such curves record the success of the specified search at each setting; they do not establish the monotone

worst-case envelope over nested feasible sets. The two panels also show dependence on the target model and defence configuration. They support reporting a budget-dependent curve with explicit constraints, while the limited settings and absent repeated-run information prevent a general ranking of defence families. The same methodological requirement applies to Android: an evaluation must state how permitted artefact transformations map to the representation used by the detector.

6.9 Threats to Validity

Measurement validity. The contest combines task-dependent accuracies with a net-change score. The identity and attribute objectives require different success criteria, and the targeted software output uses target-label agreement. Aggregate accuracy loss cannot identify induced errors, useful classification or detector alarms. The analytical results in Sections 6.7 and 6.8.4 identify these limits; they do not recover unreported individual outcomes.

Attack coverage and budgets. No independent attack submissions entered the contest. Its fixed gradient-based baselines did not constitute a documented adaptive assessment of every defence. The SD development account includes transfer attacks, but the official table provides no per-attack breakdown. The benchmark additionally mixes attack families without the full configurations needed for comparison. A later evaluation would require valid perturbations in common coordinates and attacks adapted to the complete defended system, with convergence checks [21, 20].

Sampling and uncertainty. The exact hidden samples are unavailable, and the histopathology subset size, class proportions and patient-level partitioning are unreported. Patch counts cannot be treated as independent patient counts, and forty attributes on the same image are dependent observations. The available rounded aggregates and statements about cross-validation do not support confidence intervals, paired significance tests or a fitted clean-robustness relation.

Conflicting records and implementation provenance. The SD task names and the submission totals disagree across the reports. No checkpoint mapping resolves the zero and 0.02 deltas or the best and worst score associations. The inspected code exposes a raw-versus-clipped scoring issue, but its connection to the published experiment is unknown. The source repository and dependency versions therefore support implementation inspection, while experiment reproduction still requires the corresponding data and model artefacts. These distinctions remain explicit rather than being resolved by a presumed run history.

Generalisation and attribution. The four participating teams belonged to SPARTA, so the contest did not test participation beyond the consortium. The evaluated image tasks also differ from Android malware analysis in their inputs and validity constraints. The public PDF-malware study narrows this conceptual difference through constrained count features, but supplies no Android result. Its experiments and the partner defence algorithms retain their attribution. D7.4 is a confidential deliverable; the figures and empirical comparisons in this chapter use public reports.

A focused subsequent study could compare an ordinary classifier with a training-selected constant baseline and the same classifier under output thresholding. For detectors, paired base predictions and alarms would test Equation 6.11. Budget curves and per-example attack aggregation would then measure how the conclusions change under validated perturbations. Those experiments remain separate from the historical evidence presented here.

6.10 Individual Contribution and Project Attribution

The candidate's contribution is stated at section level, following his account of the collaborative project. He wrote D7.1 Sections 2.2.2 and 2.3, and D7.2 Sections 2.1 and 2.2. For D7.2 Sections 2.4 and 2.5 he contributed the concept and definitions. He edited and refined partner text in D7.3, with no authored content claimed for that plan. He wrote Sections 3.1 and 3.2 of D7.4, Chapter 5 of D7.5, and the whole of D7.6 [37, 171, 163, 38, 39]. His D7.4 contribution is recorded as part of this account, without using the confidential report as an empirical source.

Within SAFAIR, the candidate contributed to organising the competition and to developing the reusable benchmark framework. The contest toolkit and benchmark source code are published on GitHub under the candidate's name [40, 164]. The evaluated defensive algorithms remain partner methods. In particular, the histopathology configurations originate in the Vicomtech study of D7.2 Section 2.3.2; the PDF-malware curves belong to the partner study in D7.5 Section 2.8. Authorship of the reporting deliverables does not transfer those experiments or algorithms to the candidate. The mathematical analysis and source comparison in this chapter examine their implications for the evaluation framework.

6.11 Chapter Summary

RQ4 concerns the translation of an adversary model into a reproducible benchmark and the relation between clean performance and adversarial robustness. The SAFAIR work delivered a competition with hidden evaluation samples and a reusable framework for evaluating trained models against reference attacks. The framework established common component interfaces and an execution procedure with adaptations for different prediction tasks. Its reported use by Vicomtech on histopathology images documents an application separate from the competition. The subsequent analysis identifies the experimental records and measurement checks required to interpret the resulting scores.

The evidence supports several bounded conclusions. The histopathology table gives different comparisons under accuracy loss and absolute attacked accuracy. The SD development account reports improved transfer-attack performance in a configuration with lower clean accuracy, but changes in ensemble size and training prevent causal attribution. The constant-prediction result proves that the historical delta score can assign zero loss to a classifier with low utility. Logit thresholding provides a separate mechanism: the attribute decisions remain unchanged while direct gradients are obstructed. The invariant activation-detector accuracy does not identify either mechanism.

These findings support an evaluation that reports clean and attacked utility together, tests attacks against the full defence, and records individual predictions with valid perturbation budgets. For detector-based systems, the response to an alarm is part of the method being evaluated. The constrained PDF example further shows why the validity of an input must be specified for its domain. The chapter establishes these methodological consequences from the documented systems and their measurements, while leaving a general clean–robustness relation and experiment-level reproduction unresolved.

Cross-Contribution Discussion

The contribution chapters examine different forms of context under separate experimental protocols. This discussion relates the observations each representation requires to the conclusions its evaluation supports. Comparisons between Hybroid and HGANN-Mal are conceptual; their scores do not provide a direct comparison because the corpora and tasks differ. The central question is when additional context provides useful information and how the evidence for that benefit is bounded.

7.1 Communication Context and Program Context

In the network studies of Chapter 3, context comprises the communication partners of a device or service and the exchanges observed between them. In HGANN-Mal, context consists of relations among methods within an application. Hybroid combines program structure with network observations. Each representation depends on its observation process: the service graph contains relations encountered during learning, whereas the call graph contains invocations recovered by static analysis. Neither is a complete record of system behaviour. This comparison concerns their methodological properties; it does not imply a developmental link between the IoT and Android studies.

Legitimate changes can also produce anomalous observations. Each of the nine false alarm episodes on the industrial benign monitoring day had a benign cause. Three followed engineering workstation activity and two followed planned device restarts. The service-graph design permits an operator to approve new relations. An application update that changes its call graph presents a related problem, but the Android evaluations did not examine it. The industrial observations therefore motivate a question about adaptation to legitimate change, without demonstrating its effect on either Android detector.

7.2 Multimodal and Higher-Order Representations

Both Android systems derive method-level call graphs with Androguard and use opcode information with operands discarded. Hybroid adds traffic captured from the application. HGANN-Mal constructs hyperedges over information derived from its static representation. The additions address different observational limits. Flow averaging removes the order and detailed temporal organisation of exchanges, while retaining aggregate features such as mean record duration. Hypergraph construction remains dependent on the calls and methods recovered by the analyser.

Within the Hybroid evaluation, fusion increased detection F_1 by one or two percentage points over the better single modality. For category classification, its effect ranged from a two-point decrease to a three-point increase. Network features led code features for detection and trailed them for category classification under each reported learner. Thus, the observed benefit of an additional modality depends on the task and classifier.

The hypergraph systems in Chapter 5 exceeded the pairwise baselines under the reported protocols. HGANN-Mal's advantage over the non-attentive systems was metric-dependent: on CICMalDroid binary detection, it trailed HGNN+ by 0.1 accuracy points but led by 0.3 F_1 points. Hyperedge construction and attention changed together, so these differences cannot be attributed to attention alone. The results support comparisons between complete systems; the missing ablation limits conclusions about the mechanism responsible.

7.3 Observation Requirements and Data-Acquisition Cost

The fused Hybroid model requires a traffic capture for each application. Its evaluation used CICAndMal2017 captures recorded by the dataset authors on three physical handsets under scripted use. Both branches excluded 47 graph-extraction failures from the 2,126 matched samples, leaving 2,079 for evaluation. The static branch was also evaluated on 135,905 applications without corresponding traffic records. This demonstrates its use on a larger corpus, although it does not quantify a runtime advantage over the fused model. HGANN-Mal likewise requires application packages without traffic acquisition, but reports no extraction-time or memory measurements.

The industrial study required access to the plant and manual labelling of 25 attack episodes. The Raspberry Pi 3 experiment measured extractor throughput separately, including a moderate load of about 2,500 input frames per second. Collection effort and extraction throughput are therefore different costs. The available records do not isolate how either determined corpus size.

These requirements affect which applications and operating conditions can be evaluated. The S7-300 test partition contains one episode per attack class; each Hybroid malware category contains between 101 and 112 applications in the corpus. Additional traffic may provide evidence absent from static code, but its benefit must justify the required observation. Chapter 4's hypothesis that unreachable injected calls could leave traffic unchanged remains untested against Hybroid.

7.4 Predictive Evaluation and Adversarial Evaluation

The predictive results of Chapters 3–5 concern samples and attacks that were not adapted to the evaluated detector. Chapter 6 adds an explicit adversary and shows why attack performance must be interpreted alongside useful classification. A constant classifier has zero accuracy loss under every label-preserving attack (Equation 6.14). One SD configuration reportedly reached constant attribute predictions at approximately 80 per cent accuracy. This makes a matched trivial baseline relevant to interpretation, even when an aggregate score appears high.

For Android malware, the attacker must also preserve a functioning application and its malicious behaviour. Published attacks can satisfy such constraints while changing recovered call graphs [112]. Both Android systems derive their program representations from those graphs; constructing hyperedges does not remove the upstream influence of code modification. An evaluation would need to specify valid package transformations and attack the complete pipeline, including the fusion stage where applicable. Clean and attacked performance would then be reported under the same conditions [21]. Neither Android system underwent that evaluation, so their predictive results establish neither adversarial robustness nor a demonstrated exploitable weakness.

7.5 Dataset Limitations and Extraction Failures

Evaluation depends on which samples reach the classifier. Hybroid excluded the same 47 graph-extraction failures from both branches. HGANN-Mal does not report its extraction-failure count or how failures were handled. Their extraction coverage therefore cannot be compared. CICMalDroid retains 11,598 of 17,341 collected applications after its analysis filters, which may exclude some evasive behaviour. The industrial extractor requires observed traffic, while the hidden contest procedure excludes images that fail its face-detection check. Each result is conditional on the samples retained by its pipeline.

Application provenance and traffic acquisition also require separate treatment. CICAnd-Mal2017 collected benign applications from Google Play and malware from VirusTotal and Contagio, although their traffic was captured using common handsets and procedures. Shared capture conditions control part of the measurement process; they do not eliminate differences in application origin. The Drebin experiment combines malware with benign applications drawn from AndroZoo, with no documented collection-window restriction on that draw. Such differences can provide predictive cues unrelated to malicious behaviour, as demonstrated in the Android case study of Arp et al. [13].

The contribution chapters identify inconsistencies in their source records and recompute the quantities supported by the available counts. Remaining unknowns limit interpretation even where a reported score can be reproduced arithmetically.

Table 7.1. Evaluation conditions and reported binary class shares. “Unreported” denotes missing information, rather than a measured absence. Baseline descriptions refer to the evidence presented in the contribution chapters.

Study	Reported split	Positive class and share	Population represented	Trivial baseline
S7-300	Whole capture sessions	Attack, 6.2%	Confirmed test records	Stated in the text
SWaT GAN	Chronological; later 393,679 timestamps for testing	Anomalous, 13.1%	Later part of one SWaT attack recording	Stated in the text
Hybroid	Five-fold cross-validation	Malware, 20.0% in source corpus	2,079 of 2,126 samples	Not reported
HGANN-Mal: Drebin/AndroZoo	Random 70/20/10	Malware, approximately 37.0%	Assembled binary corpus	Corpus majority
HGANN-Mal: CICMalDroid	Random 70/20/10	Malware, 84.5%	Full study corpus	Corpus majority
SAFAIR histopathology benchmark	Evaluated subset unreported	IDC, unreported	Test population unidentified	Analytical discussion only

7.6 Scope of Generalisation

Table 7.1 distinguishes evaluation protocols and the populations behind the reported class shares. These percentages describe binary detection; they do not characterise the multiclass label distributions. Corpus proportions need not equal an undocumented test partition.

The SWaT protocol evaluates the later 393,679 timestamps of the same attack-period recording; the earlier 56,240 timestamps support training and validation. This split prevents reuse of test observations during model development, but it does not measure transfer to another campaign or installation. Session separation and random partitioning address different questions from a time-ordered evaluation [12]. The reported folds also do not provide the independent repeated-run outcomes needed to quantify variability. In the industrial studies, the S7-300 test attacks belong to classes present during training, and the SWaT detector is developed with attack observations in training and supervised selection.

Within-study contrasts provide the most direct evidence for performance differences under their reported conditions. Their interpretation still depends on the controls: Hybroid does not specify whether preprocessing and embeddings were fitted within each fold, and HGANN-Mal does not isolate attention from hyperedge construction. These limitations concern internal validity. Transfer to another population is a separate question, as illustrated by performance deterioration under challenging conditions in the comparative Android study of Gao et al. [61]. Chapter 6’s analytical identities apply independently of a particular corpus under their stated assumptions; its empirical measurements retain their sampling and implementation limits.

7.7 Implications for Android Malware Analysis

The results support choosing context according to the prediction task and the observations available. Traffic improved detection relative to code features in the Hybroid experiment, but fusion did not improve every category-classification configuration. Higher-order systems performed well in the Chapter 5 comparisons, although the contribution of attention remains unidentified. Additional context therefore requires evidence of benefit within the intended task, together with controls that identify what produced the difference.

Observation limits remain after a representation is enriched. Static analysis avoids per-application traffic acquisition but cannot recover behaviour absent from its analysis. Traffic depends on the exercised behaviour and capture environment; the protocol mix in the Hybroid corpus further limits conclusions about encrypted communication. A deployment claim would require evaluation of these conditions as well as prediction quality. The dissertation establishes bounded benefits of contextual representations and a method for examining the validity of their evaluation. Its remaining empirical questions concern the missing component ablations and performance under valid application transformations or later data.

Conclusion and Future Work

8.1 Answers to the Research Questions

RQ1. *How can behaviour in networked and industrial systems be represented to support anomaly detection?* Behaviour can be represented at the unit that a deployment exposes, but the representation must also admit legitimate change. On a benign industrial day, all nine false alarm episodes had benign causes, five of them engineering work or planned device restarts. Chapter 3 closes gap G1 at the implementation level.

RQ1.1. *How can service relationships be represented as communication graphs to identify unexpected interactions?* Each node learns the relations observed while a service starts and reports any relation absent from its graph, which the site owner may then legitimate. Detection added about 6 ms to a 24 ms round trip, and this cost did not grow between ten and forty services.

RQ1.2. *How can a modular learning framework analyse network and process features for anomaly detection in industrial control systems?* NADICS fixes the interfaces around the learning stage, so that estimators can be exchanged by name. Under a session-disjoint split of the S7-300 corpus from IUNO, a random forest reached an attack F_1 of 94.50 per cent at a record false-positive rate of 0.484 per cent. On 393,679 later SWaT timestamps withheld from model development, a GAN-based detector added at the same interface flags 92.00 per cent of anomalous timestamps at a false-positive rate of 5.12 per cent.

RQ2. *To what extent does fusing static code-graph representations with dynamic network-flow features improve Android malware detection and category classification compared with either modality alone?* On CICAndMal2017, fusion added one or two points of detection F_1 over the better single modality, and between minus two and plus three points in category classification. Network flow led program code in detection and trailed it in category classification. The combination therefore helps detection slightly and category classification only with some learners, which closes gap G2 for this corpus and protocol.

RQ3. *To what extent do attention-weighted hypergraph representations of static Android program structure improve binary malware detection and malware classification over pairwise graph models and non-attentive hypergraph models?* With identical inputs, every hypergraph operator exceeded both pairwise operators in all four task and corpus combinations, and HGNN+ led GraphSAGE by 3.8 to 6.7 points of accuracy. HGANN-Mal led the non-attentive operators by 1.2 to 4.1 points in three combinations and was level in the fourth. Its attention can separate hypergraphs that share a clique expansion, although within one layer it ranks the hyperedges of every method in one shared order. These comparisons of complete systems close gap G3.

RQ4. *How can an explicit adversary model be translated into a reproducible benchmark for adversarial attacks and defences, and what does such a benchmark reveal about the relation between clean predictive performance and adversarial robustness?* The SAFAIR work translated the adversary model into an evaluation protocol, realised as a contest and then as a modular benchmark that combines attacks and defences by configuration. Reproducible measurement also requires a fixed prediction objective and a valid perturbation set. The accuracy-loss score used for ranking is indifferent to utility: any constant classifier has zero loss under label-preserving attacks, and under PGD the defence with zero loss was 12.9 points less accurate than adversarial training. Gap G4 is closed for building and scoring such a benchmark; the four contest rows show no general relation between clean performance and robustness.

8.2 Summary of Contributions

For IoT services, the candidate contributed to the concept and the measurement design of a communication-graph model that each node keeps current as services change, at a measured cost in memory and latency. For industrial control systems, he designed and implemented NADICS with its S7 and PROFINET extractor, whose 63-field schema reaches from flow statistics to S7 function codes, and he produced the labelled S7-300 corpus of 25 attack episodes. The GAN-based detector for SWaT process data is also his work.

Hybroid contributes a flow pipeline that reduces the traffic of an application to 13 features, together with a comparison of each modality with their fusion on the same samples. HGANN-Mal lifts the function-call graph to a hypergraph under three construction rules and learns membership weights by attention. The third rule builds a hyperedge from the backward slice of each security-sensitive call site and weights it by the transmission and access calls in that slice. Chapter 5 restores the construction algorithm and analyses the attention operator, and it shows that the hypergraph grows linearly with the numbers of methods and slices. The SAFAIR contest toolkit and benchmark, both published in the candidate's name, form the evaluation contribution. Chapter 6 derives how the accuracy-loss score behaves under constant predictions and how label reversal after an alarm changes accuracy. In Chapter 7, the observations each representation requires are related to the conclusions its evaluation supports.

8.3 Future Work

The first open item is the HGANN-Mal ablation, which isolates the effect of attention: the non-attentive operator over the identical fused hyperedges, and HGANN-Mal over topological hyperedges alone, on both corpora and under a stated normalisation convention. Both Android systems then need time-ordered splits and attacks through valid package transformations, with clean and attacked performance reported under one protocol. For Hybroid, such an attack would test whether unreachable injected calls leave its traffic features unchanged.

References

- [1] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck. “DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket”. In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2014. DOI: 10.14722/ndss.2014.23247.
- [2] L. Onwuzurike, E. Mariconti, P. Andriotis, E. De Cristofaro, G. Ross, and G. Stringhini. “MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models (Extended Version)”. In: *ACM Transactions on Privacy and Security* 22.2 (2019), pp. 1–34. DOI: 10.1145/3313391.
- [3] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu. “Deep Learning for Android Malware Defenses: A Systematic Literature Review”. In: *ACM Computing Surveys* 55 (2023), pp. 1–36. DOI: 10.1145/3544968.
- [4] V. Chandola, A. Banerjee, and V. Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (2009), pp. 1–58. DOI: 10.1145/1541880.1541882.
- [5] G. Pang, C. Shen, L. Cao, and A. van den Hengel. “Deep Learning for Anomaly Detection: A Review”. In: *ACM Computing Surveys* 54.2 (2021), pp. 1–38. DOI: 10.1145/3439950.
- [6] T. N. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *5th International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1609.02907 [cs.LG].
- [7] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. “Graph Attention Networks”. In: *6th International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903 [stat.ML].
- [8] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui. “A Survey on Malware Detection with Graph Representation Learning”. In: *ACM Computing Surveys* 56.11 (2024), pp. 1–36. DOI: 10.1145/3664649.
- [9] Google. *The Android Show: I/O Edition*. <https://blog.google/products/android/the-android-show-io-2025/>. May 2025.
- [10] A. P. Mathur and N. O. Tippenhauer. “SWaT: A Water Treatment Testbed for Research and Training on ICS Security”. In: *Proceedings of the International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater)*. IEEE, 2016, pp. 31–36. DOI: 10.1109/CySWater.2016.7469060.
- [11] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouretdinov, and L. Cavallaro. “Transcend: Detecting Concept Drift in Malware Classification Models”. In: *USENIX Security Symposium*. 2017, pp. 625–642.

- [12] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro. “TESSERACT: Eliminating Experimental Bias in Malware Classification Across Space and Time”. In: *USENIX Security Symposium*. 2019, pp. 729–746. arXiv: 1807.07838.
- [13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck. “Dos and Don’ts of Machine Learning in Computer Security”. In: *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988. arXiv: 2010.09470. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [14] P. Irolla and A. Dey. “The Duplication Issue within the Drebin Dataset”. In: *Journal of Computer Virology and Hacking Techniques* 14.3 (2018), pp. 245–249. DOI: 10.1007/s11416-018-0316-z.
- [15] Y. Zhao, L. Li, H. Wang, H. Cai, T. F. Bissyandé, J. Klein, and J. Grundy. “On the Impact of Sample Duplication in Machine-Learning-Based Android Malware Detection”. In: *ACM Transactions on Software Engineering and Methodology* 30.3 (2021), 40:1–40:38.
- [16] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus. “Intriguing Properties of Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2014. arXiv: 1312.6199 [cs.CV].
- [17] B. Biggio, I. Corona, D. Maiorca, B. Nelson, N. Šrndić, P. Laskov, G. Giacinto, and F. Roli. “Evasion Attacks against Machine Learning at Test Time”. In: *European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Vol. 8190. Lecture Notes in Computer Science. Springer, 2013, pp. 387–402. DOI: 10.1007/978-3-642-40994-3_25.
- [18] B. Biggio and F. Roli. “Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning”. In: *Pattern Recognition* 84 (2018), pp. 317–331. DOI: 10.1016/j.patcog.2018.07.023.
- [19] A. Athalye, N. Carlini, and D. Wagner. “Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples”. In: *International Conference on Machine Learning (ICML)*. Vol. 80. Proceedings of Machine Learning Research. 2018, pp. 274–283.
- [20] F. Tramèr, N. Carlini, W. Brendel, and A. Madry. “On Adaptive Attacks to Adversarial Example Defenses”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020, pp. 1633–1645. arXiv: 2002.08347.
- [21] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. J. Goodfellow, A. Madry, and A. Kurakin. *On Evaluating Adversarial Robustness*. arXiv:1902.06705. 2019. arXiv: 1902.06705 [cs.LG].
- [22] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro. “Intriguing Properties of Adversarial ML Attacks in the Problem Space”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2020, pp. 1332–1349. DOI: 10.1109/SP40000.2020.00073.
- [23] A. Moser, C. Kruegel, and E. Kirda. “Limits of Static Analysis for Malware Detection”. In: *Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC)*. 2007, pp. 421–430. DOI: 10.1109/ACSAC.2007.21.
- [24] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. “TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones”. In: *ACM Transactions on Computer Systems* 32 (2014), pp. 1–29. DOI: 10.1145/2619091.

- [25] L. Onwuzurike, M. Almeida, E. Mariconti, J. Blackburn, G. Stringhini, and E. De Cristofaro. “A Family of Droids – Android Malware Detection via Behavioral Modeling: Static vs Dynamic Analysis”. In: *Annual Conference on Privacy, Security and Trust (PST)*. 2018, pp. 1–10. DOI: 10.1109/PST.2018.8514191.
- [26] D. Zhou, J. Huang, and B. Schölkopf. “Learning with Hypergraphs: Clustering, Classification, and Embedding”. In: *Advances in Neural Information Processing Systems 19 (NIPS)*. MIT Press, 2006, pp. 1601–1608. DOI: 10.7551/mitpress/7503.003.0205.
- [27] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao. “Hypergraph Neural Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33.01 (2019), pp. 3558–3565. DOI: 10.1609/aaai.v33i01.33013558.
- [28] Y. Gao, Y. Feng, S. Ji, and R. Ji. “HGNN⁺: General Hypergraph Neural Networks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.3 (2023), pp. 3181–3199. DOI: 10.1109/TPAMI.2022.3182052.
- [29] S. Brody, U. Alon, and E. Yahav. “How Attentive Are Graph Attention Networks?” In: *Proceedings of the 10th International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2105.14491.
- [30] M.-O. Pahl and F.-X. Aubet. “All Eyes on You: Distributed Multi-Dimensional IoT Microservice Anomaly Detection”. In: *Proceedings of the 14th International Conference on Network and Service Management (CNSM)*. IEEE, 2018, pp. 72–80. URL: <https://ieeexplore.ieee.org/document/8584985>.
- [31] M.-O. Pahl, F.-X. Aubet, and S. Liebold. “Graph-based IoT Microservice Security”. In: *NOMS 2018 – 2018 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2018, pp. 1–3. DOI: 10.1109/NOMS.2018.8406118.
- [32] F.-X. Aubet, M.-O. Pahl, S. Liebold, and M. R. Norouzian. *Graph-based Anomaly Detection for IoT Microservices*. Two-page contribution, Passive and Active Measurement Conference (PAM). Berlin, Germany, 2018.
- [33] S. Duque Antón, M. R. Norouzian, K. Fischer, D. Friedrich, P. Kasinathan, S. Teuber, R. Hormann, M. Soppa, S. Dukanovic, J. Hachenberger, A. Giehl, S. Plaga, S. Tatschner, and N. Wiedermann. *Anwendungsfall "Visueller Security-Leitstand"*. Abschlussdeliverable, Arbeitspaket 4. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, 2018.
- [34] S. Duque Antón, D. Fraunholz, C. Lipps, D. Angermeier, B. Filipovic, A. Giehl, J. Obermaier, P. Schneider, D. Schuster, N. Wiedermann, M. Rudolph, H. Brandl, F. Schreiner, K. Fischer, D. Friedrich, B. Bierbaumer, M. R. Norouzian, J. Müller, G. Fischer, and W. Neifer. *Werkzeuge*. Abschlussdeliverable, Arbeitspaket 7. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, 2018.
- [35] M. R. Norouzian, P. Xu, C. Eckert, and A. Zarras. “Hybroid: Toward Android Malware Detection and Categorization with Program Code and Network Traffic”. In: *Information Security: 24th International Conference, ISC 2021*. Vol. 13118. Lecture Notes in Computer Science. Springer, 2021, pp. 259–278. DOI: 10.1007/978-3-030-91356-4_14.
- [36] M. R. Norouzian and C. Eckert. “HGANN-Mal: A Hypergraph Attention Neural Network Approach for Android Malware Detection”. In: *Information Security: 28th International Conference, ISC 2025*. Vol. 16186. Lecture Notes in Computer Science. Springer, 2025, pp. 388–408. DOI: 10.1007/978-3-032-08124-7_23.

- [37] SPARTA Consortium. *AI Systems Threat Analysis Mechanisms and Tools*. Project Deliverable D7.1. Editor: E. Ríos (TEC). Contributor: M. R. Norouzian (TUM). Dissemination level: public. 95 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, June 2021. URL: https://www.sparta.eu/assets/deliverables/SPARTA-D7.1-AI-systems-threat-analysis-mechanisms-and-tools-PU-M18_v1.1.pdf.
- [38] SPARTA Consortium. *Final Version of AI Systems Security Mechanisms and Tools*. Project Deliverable D7.5. Editor: B. Addad (TCS). Contributor: M. R. Norouzian (TUM). Dissemination level: public. 129 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, Aug. 2021. URL: <https://www.sparta.eu/assets/deliverables/SPARTA-D7.5-Final-version-of-AI-systems-security-mechanisms-and-tools-PU-M30.pdf>.
- [39] SPARTA Consortium. *Validation and Evaluation Report*. Project Deliverable D7.6. Editor: M. R. Norouzian (TUM). Dissemination level: public. 73 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, Feb. 2022. URL: <https://www.sparta.eu/assets/deliverables/SPARTA-D7.6-Final-version-of-evaluation-and-validation-plan-PU-M36.pdf>.
- [40] M. R. Norouzian. *SAFAIR AI Contest Development Toolkit*. https://git.sec.in.tum.de/Norouzian/safair-ai-contest/-/tree/master/dev_toolkit. 2021.
- [41] R. Hormann, S. Teuber, S. Nikelski, P. Larbig, S. Neff, D. Scheuermann, A. Giehl, S. Plaga, M. R. Norouzian, and A. Malkis. *Anomalieerkennung im Wertschöpfungsnetzwerk; Datenaustauschplattform für das Wertschöpfungsnetzwerk*. Deliverable D4.2-1 / M24. IUNO — Nationales Referenzprojekt IT-Sicherheit in Industrie 4.0, June 2017.
- [42] R. Sommer and V. Paxson. “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection”. In: *Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2010, pp. 305–316. DOI: 10.1109/SP.2010.25.
- [43] S. Axelsson. “The Base-Rate Fallacy and the Difficulty of Intrusion Detection”. In: *ACM Transactions on Information and System Security* 3.3 (2000), pp. 186–205. DOI: 10.1145/357830.357849.
- [44] Android Open Source Project. *Dalvik Executable Format*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime/dex-format> (visited on 08/13/2026).
- [45] Android Open Source Project. *Dalvik Bytecode*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime/dalvik-bytecode> (visited on 08/13/2026).
- [46] Android Open Source Project. *Android Runtime (ART) and Dalvik*. Android Open Source Project. 2026. URL: <https://source.android.com/docs/core/runtime> (visited on 08/13/2026).
- [47] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner. “Android Permissions Demystified”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2011, pp. 627–638. DOI: 10.1145/2046707.2046779.
- [48] Android Open Source Project. *Permissions on Android*. Android Open Source Project. 2026. URL: <https://developer.android.com/guide/topics/permissions/overview> (visited on 08/13/2026).

- [49] K. W. Y. Au, Y. F. Zhou, Z. Huang, and D. Lie. “PScout: Analyzing the Android Permission Specification”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2012.
- [50] A. Sabbah, R. Jarrar, S. Zein, and D. Mohaisen. “Understanding Concept Drift With Deprecated Permissions in Android Malware Detection”. In: *IEEE Transactions on Dependable and Secure Computing* 23.4 (2026), pp. 8292–8306. DOI: 10.1109/TDSC.2026.3682216.
- [51] A. Desnos. “Android: Static Analysis Using Similarity Distance”. In: *Hawaii International Conference on System Sciences (HICSS)*. 2012, pp. 5394–5403. DOI: 10.1109/HICSS.2012.114.
- [52] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan. “Soot: A Java Bytecode Optimization Framework”. In: *CASCON First Decade High Impact Papers*. 2010, pp. 214–224. DOI: 10.1145/1925805.1925818.
- [53] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Ochteau, and P. McDaniel. “FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps”. In: *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2014, pp. 259–269. DOI: 10.1145/2594291.2594299.
- [54] L. K. Yan and H. Yin. “DroidScope: Seamlessly Reconstructing the OS and Dalvik Semantic Views for Dynamic Android Malware Analysis”. In: *USENIX Security Symposium*. 2012, pp. 569–584.
- [55] T. Petsas, G. Voyatzis, E. Athanasopoulos, M. Polychronakis, and S. Ioannidis. “Rage against the Virtual Machine: Hindering Dynamic Analysis of Android Malware”. In: *European Workshop on System Security (EuroSec)*. 2014, pp. 1–6. DOI: 10.1145/2592791.2592796.
- [56] T. Vidas and N. Christin. “Evading Android Runtime Analysis via Sandbox Detection”. In: *ACM Symposium on Information, Computer and Communications Security (AsiaCCS)*. 2014, pp. 447–458. DOI: 10.1145/2590296.2590325.
- [57] M. Lindorfer, M. Neugschwandtner, and C. Platzer. “MARVIN: Efficient and Comprehensive Mobile App Classification through Static and Dynamic Analysis”. In: *Proceedings of the 39th IEEE Annual Computer Software and Applications Conference (COMPSAC)*. 2015, pp. 422–433. DOI: 10.1109/COMPSAC.2015.103.
- [58] D. Wermke, N. Huaman, Y. Acar, B. Reaves, P. Traynor, and S. Fahl. “A Large Scale Investigation of Obfuscation Use in Google Play”. In: *Annual Computer Security Applications Conference (ACSAC)*. 2018, pp. 222–235. DOI: 10.1145/3274694.3274726.
- [59] S. Dong, M. Li, W. Diao, X. Liu, J. Liu, Z. Li, F. Xu, K. Chen, X. Wang, and K. Zhang. “Understanding Android Obfuscation Techniques: A Large-Scale Investigation in the Wild”. In: *International Conference on Security and Privacy in Communication Systems (SecureComm)*. 2018, pp. 172–192. DOI: 10.1007/978-3-030-01701-9_10.
- [60] M. Hammad, J. Garcia, and S. Malek. “A Large-Scale Empirical Study on the Effects of Code Obfuscations on Android Apps and Anti-Malware Products”. In: *International Conference on Software Engineering (ICSE)*. 2018. DOI: 10.1145/3180155.3180228.

- [61] C. Gao, G. Huang, H. Li, B. Wu, Y. Wu, and W. Yuan. “A Comprehensive Study of Learning-Based Android Malware Detectors under Challenging Environments”. In: *International Conference on Software Engineering (ICSE)*. 2024. DOI: 10.1145/3597503.3623320.
- [62] Y. Duan, M. Zhang, A. V. Bhaskar, H. Yin, et al. “Things You May Not Know About Android (Un)Packers: A Systematic Study Based on Whole-System Emulation”. In: *Network and Distributed System Security Symposium (NDSS)*. 2018.
- [63] L. Li, D. Li, T. F. Bissyandé, J. Klein, Y. Le Traon, D. Lo, and L. Cavallaro. “Understanding Android App Piggybacking: A Systematic Study of Malicious Code Grafting”. In: *IEEE Transactions on Information Forensics and Security* 12.6 (2017), pp. 1269–1284. DOI: 10.1109/TIFS.2017.2656460.
- [64] H. Gascon, F. Yamaguchi, D. Arp, and K. Rieck. “Structural Detection of Android Malware Using Embedded Call Graphs”. In: *Proceedings of the 2013 ACM Workshop on Artificial Intelligence and Security (AISec)*. 2013, pp. 45–54. DOI: 10.1145/2517312.2517315.
- [65] H. Dai, B. Dai, and L. Song. “Discriminative Embeddings of Latent Variable Models for Structured Data”. In: *International Conference on Machine Learning (ICML)*. Vol. 48. Proceedings of Machine Learning Research. 2016, pp. 2702–2711.
- [66] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song. “Neural Network-Based Graph Embedding for Cross-Platform Binary Code Similarity Detection”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2017, pp. 363–376. DOI: 10.1145/3133956.3134018.
- [67] U. Alon, M. Zilberstein, O. Levy, and E. Yahav. “code2vec: Learning Distributed Representations of Code”. In: *Proceedings of the ACM on Programming Languages* 3.POPL (2019), pp. 1–29. DOI: 10.1145/3290353.
- [68] S. Agarwal, K. Branson, and S. Belongie. “Higher Order Learning with Graphs”. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. 2006, pp. 17–24. DOI: 10.1145/1143844.1143847.
- [69] U. Chitra and B. J. Raphael. “Random Walks on Hypergraphs with Edge-Dependent Vertex Weights”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019. arXiv: 1905.08287.
- [70] D. Y. Kang, Q. Mei, and S.-W. Kim. “Low Mileage, High Fidelity: Evaluating Hypergraph Expansion Methods by Quantifying the Information Loss”. In: *Proceedings of the ACM Web Conference (WWW)*. 2024, pp. 959–968. DOI: 10.1145/3589334.3645657.
- [71] Y. Wang and J. M. Kleinberg. “From Graphs to Hypergraphs: Hypergraph Projection and Its Reconstruction”. In: *International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=qwYKE3VB2h>.
- [72] F. Jiang, Y. Li, Y. Feng, K. Zheng, L. Niu, B. Ramasubramanian, B. Alomair, L. Bushnell, and R. Poovendran. *The WidthWall: A Strict Expressivity Hierarchy for Hypergraph Neural Networks*. 2026. arXiv: 2605.13690 [cs.LG].
- [73] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry”. In: *International Conference on Machine Learning (ICML)*. 2017, pp. 1263–1272. arXiv: 1704.01212.

- [74] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. “How Powerful are Graph Neural Networks?” In: *International Conference on Learning Representations (ICLR)*. 2019.
- [75] Q. Li, Z. Han, and X.-M. Wu. “Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (2018). DOI: 10.1609/aaai.v32i1.11604.
- [76] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017, pp. 5998–6008. arXiv: 1706.03762.
- [77] T. Fang, T. Gao, C. Wang, Y. Shang, W. Chow, L. Chen, and Y. Yang. “KAA: Kolmogorov-Arnold Attention for Enhancing Attentive Graph Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2025. URL: <https://openreview.net/forum?id=atXCzVSXTJ>.
- [78] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar. “HyperGCN: A New Method for Training Graph Convolutional Networks on Hypergraphs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019, pp. 1509–1520. arXiv: 1809.02589.
- [79] E. Chien, C. Pan, J. Peng, and O. Milenkovic. “You Are AllSet: A Multiset Function Framework for Hypergraph Neural Networks”. In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2106.13264.
- [80] S. Bai, F. Zhang, and P. H. S. Torr. “Hypergraph Convolution and Hypergraph Attention”. In: *Pattern Recognition* 110 (2021), p. 107637. DOI: 10.1016/j.patcog.2020.107637.
- [81] M. Li, Y. Fang, D. Shen, H. Feng, X. Zhuang, K. Xia, and P. Liò. “High-Pass Matters: Theoretical Insights and Sheaflet-Based Design for Hypergraph Neural Networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. Vol. 40. 2026, pp. 23039–23046. DOI: 10.1609/aaai.v40i27.39469.
- [82] Y. Gao, J. Yan, Y. Feng, X. Han, S. Ying, Z. Wu, and H. Hu. “HGNNv2: Stable Hypergraph Neural Networks”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 48.5 (2026), pp. 5311–5327. DOI: 10.1109/TPAMI.2026.3652225.
- [83] J. Kim, S. Oh, S. Cho, and S. Hong. “Equivariant Hypergraph Neural Networks”. In: *Computer Vision – ECCV 2022. Lecture Notes in Computer Science*. 2022, pp. 86–103. DOI: 10.1007/978-3-031-19803-8_6.
- [84] J. Kim, S. Oh, and S. Hong. “Transformers Generalize DeepSets and Can Be Extended to Graphs & Hypergraphs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021, pp. 28016–28028. URL: <https://proceedings.neurips.cc/paper/2021/hash/ec0f40c389aeef789ce03eb814facc6c-Abstract.html>.
- [85] A. Sperotto, G. Schaffrath, R. Sadre, C. Morariu, A. Pras, and B. Stiller. “An Overview of IP Flow-Based Intrusion Detection”. In: *IEEE Communications Surveys & Tutorials* 12.3 (2010), pp. 343–356. DOI: 10.1109/SURV.2010.032210.00054.
- [86] V. Paxson. “Bro: A System for Detecting Network Intruders in Real-Time”. In: *Computer Networks* 31.23–24 (1999), pp. 2435–2463. DOI: 10.1016/S1389-1286(99)00112-7.
- [87] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander. “LOF: Identifying Density-Based Local Outliers”. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. ACM, 2000, pp. 93–104. DOI: 10.1145/342009.335388.

- [88] F. T. Liu, K. M. Ting, and Z.-H. Zhou. “Isolation Forest”. In: *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17.
- [89] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller. “A Unifying Review of Deep and Shallow Anomaly Detection”. In: *Proceedings of the IEEE* 109.5 (2021), pp. 756–795. DOI: 10.1109/JPROC.2021.3052449.
- [90] L. Akoglu, H. Tong, and D. Koutra. “Graph Based Anomaly Detection and Description: A Survey”. In: *Data Mining and Knowledge Discovery* 29.3 (2015), pp. 626–688. DOI: 10.1007/s10618-014-0365-y.
- [91] R. R. R. Barbosa, R. Sadre, and A. Pras. “Flow Whitelisting in SCADA Networks”. In: *International Journal of Critical Infrastructure Protection* 6.3–4 (2013), pp. 150–158. DOI: 10.1016/j.ijcip.2013.08.003.
- [92] M.-K. Yoon and G. F. Ciocarlie. “Communication Pattern Monitoring: Improving the Utility of Anomaly Detection for Industrial Control Systems”. In: *Proceedings of the NDSS Workshop on Security of Emerging Networking Technologies (SENT)*. Internet Society, 2014. DOI: 10.14722/sent.2014.23012.
- [93] M. Iliofotou, P. Pappu, M. Faloutsos, M. Mitzenmacher, S. Singh, and G. Varghese. “Network Monitoring Using Traffic Dispersion Graphs (TDGs)”. In: *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement (IMC)*. ACM, 2007, pp. 315–320. DOI: 10.1145/1298306.1298349.
- [94] Y. Jin, E. Sharafuddin, and Z.-L. Zhang. “Unveiling Core Network-Wide Communication Patterns through Application Traffic Activity Graph Decomposition”. In: *ACM SIGMETRICS Performance Evaluation Review* 37.1 (2009), pp. 49–60. DOI: 10.1145/2492101.1555356.
- [95] P. Bahl, R. Chandra, A. Greenberg, S. Kandula, D. A. Maltz, and M. Zhang. “Towards Highly Reliable Enterprise Network Services via Inference of Multi-Level Dependencies”. In: *ACM SIGCOMM Computer Communication Review* 37.4 (2007), pp. 13–24. DOI: 10.1145/1282427.1282383.
- [96] X. Chen, M. Zhang, Z. M. Mao, and P. Bahl. “Automating Network Application Dependency Discovery: Experiences, Limitations, and New Solutions”. In: *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008, pp. 117–130.
- [97] A. Natarajan, P. Ning, Y. Liu, S. Jajodia, and S. E. Hutchinson. “NSDMiner: Automated Discovery of Network Service Dependencies”. In: *Proceedings of the IEEE INFOCOM*. IEEE, 2012, pp. 2507–2515. DOI: 10.1109/INFOCOM.2012.6195642.
- [98] M. Kravchik and A. Shabtai. “Detecting Cyber Attacks in Industrial Control Systems Using Convolutional Neural Networks”. In: *Proceedings of the 2018 Workshop on Cyber-Physical Systems Security and Privacy (CPS-SPC)*. ACM, 2018, pp. 72–83. DOI: 10.1145/3264888.3264896.
- [99] E. Maali, O. Alrawi, and J. A. McCann. “Evaluating Machine Learning-Based IoT Device Identification Models for Security Applications”. In: *Network and Distributed System Security Symposium (NDSS)*. 2025.

- [100] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani. “A Detailed Analysis of the KDD CUP 99 Data Set”. In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.
- [101] R. Flood, G. Engelen, D. Aspinall, and L. Desmet. “Bad Design Smells in Benchmark NIDS Datasets”. In: *IEEE European Symposium on Security and Privacy (EuroS&P)*. 2024, pp. 658–675. DOI: 10.1109/EUROSP60621.2024.00042.
- [102] D. Kus, E. Wagner, J. Pennekamp, K. Wolsing, I. B. Fink, M. Dahlmanns, K. Wehrle, and M. Henze. “A False Sense of Security? Revisiting the State of Machine Learning-Based Industrial Intrusion Detection”. In: *Proceedings of the 8th ACM Cyber-Physical System Security Workshop (CPSS)*. ACM, 2022, pp. 73–84. DOI: 10.1145/3494107.3522773.
- [103] K. Wolsing, L. Thiemt, C. van Sloun, E. Wagner, K. Wehrle, and M. Henze. “Can Industrial Intrusion Detection Be SIMPLE?”. In: *Computer Security (ESORICS 2022)*. Lecture Notes in Computer Science. Springer, 2022, pp. 574–594. DOI: 10.1007/978-3-031-17143-7_28.
- [104] B. Biggio, G. Fumera, and F. Roli. “Security Evaluation of Pattern Classifiers under Attack”. In: *IEEE Transactions on Knowledge and Data Engineering* 26.4 (2014), pp. 984–996. DOI: 10.1109/TKDE.2013.57.
- [105] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. Tech. rep. NIST AI 100-2e2025. National Institute of Standards and Technology, Mar. 2025. DOI: 10.6028/NIST.AI.100-2e2025.
- [106] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami. “Practical Black-Box Attacks against Machine Learning”. In: *ACM Asia Conference on Computer and Communications Security (ASIACCS)*. 2017, pp. 506–519. DOI: 10.1145/3052973.3053009.
- [107] K. Grosse, L. Bieringer, T. R. Besold, and A. Alahi. “Towards More Practical Threat Models in Artificial Intelligence Security”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024.
- [108] B. Biggio, B. Nelson, and P. Laskov. “Poisoning Attacks against Support Vector Machines”. In: *International Conference on Machine Learning (ICML)*. 2012, pp. 1807–1814. arXiv: 1206.6389.
- [109] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li. “Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2018, pp. 19–35. DOI: 10.1109/SP.2018.00057.
- [110] T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg. “BadNets: Evaluating Backdooring Attacks on Deep Neural Networks”. In: *IEEE Access* 7 (2019), pp. 47230–47244. DOI: 10.1109/ACCESS.2019.2909068.
- [111] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart. “Stealing Machine Learning Models via Prediction APIs”. In: *USENIX Security Symposium*. 2016, pp. 601–618. arXiv: 1609.02943.
- [112] S. Song, X. Xie, R. Feng, Q. Guo, and S. Chen. “FCGHunter: Towards Evaluating Robustness of Graph-Based Android Malware Detection”. In: *IEEE Transactions on Software Engineering* 52.2 (2026), pp. 428–448. DOI: 10.1109/TSE.2025.3626788.

- [113] K. Grosse, N. Papernot, P. Manoharan, M. Backes, and P. McDaniel. “Adversarial Examples for Malware Detection”. In: *European Symposium on Research in Computer Security (ESORICS)*. Lecture Notes in Computer Science. Springer, 2017, pp. 62–79. DOI: 10.1007/978-3-319-66399-9_4.
- [114] I. J. Goodfellow, J. Shlens, and C. Szegedy. “Explaining and Harnessing Adversarial Examples”. In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6572 [stat.ML].
- [115] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. “Towards Deep Learning Models Resistant to Adversarial Attacks”. In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1706.06083 [stat.ML].
- [116] T. Chow, M. D’Onghia, L. Linhardt, Z. Kan, D. Arp, L. Cavallaro, and F. Pierazzi. “Beyond the TESSERACT: Trustworthy Dataset Curation for Sound Evaluations of Android Malware Classifiers”. In: *arXiv preprint arXiv:2506.23814* (2025).
- [117] Z. Kan, S. McFadden, D. Arp, F. Pendlebury, R. Jordaney, J. Kinder, F. Pierazzi, and L. Cavallaro. “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Extended Version)”. In: *arXiv preprint arXiv:2402.01359* (2024).
- [118] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon. “Towards a Rigorous Evaluation of Time-Series Anomaly Detection”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 36.7 (2022), pp. 7194–7201. DOI: 10.1609/aaai.v36i7.20680.
- [119] S. Sørnbø and M. Ruocco. “Navigating the Metric Maze: A Taxonomy of Evaluation Metrics for Anomaly Detection in Time Series”. In: *Data Mining and Knowledge Discovery* 38.3 (2024), pp. 1027–1068. DOI: 10.1007/s10618-023-00988-8.
- [120] Q. Liu and J. Paparrizos. “The Elephant in the Room: Towards A Reliable Time-Series Anomaly Detection Benchmark”. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2024.
- [121] T. Bilot, B. Jiang, Z. Li, N. El Madhoun, K. Al Agha, A. Zouaoui, and T. Pasquier. “Sometimes Simpler is Better: A Comprehensive Analysis of State-of-the-Art Provenance-Based Intrusion Detection Systems”. In: *USENIX Security Symposium*. 2025, pp. 7193–7212.
- [122] N. Daoudi, K. Allix, T. F. Bissyandé, and J. Klein. “Lessons Learnt on Reproducibility in Machine Learning Based Android Malware Detection”. In: *Empirical Software Engineering* 26 (2021), p. 74. DOI: 10.1007/s10664-021-09955-7.
- [123] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur. “A Dataset to Support Research in the Design of Secure Water Treatment Systems”. In: *Critical Information Infrastructures Security (CRITIS)*. Vol. 10242. Lecture Notes in Computer Science. Springer, 2017, pp. 88–99. DOI: 10.1007/978-3-319-71368-7_8.
- [124] M.-O. Pahl, G. Carle, and G. Klinker. “Distributed Smart Space Orchestration”. In: *NOMS 2016 – 2016 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2016, pp. 979–984. DOI: 10.1109/NOMS.2016.7502936.
- [125] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

- [126] N. Moustafa and J. Slay. “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)”. In: *2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942.
- [127] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng. “MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks”. In: *International Conference on Artificial Neural Networks (ICANN)*. Vol. 11730. Lecture Notes in Computer Science. Springer, 2019, pp. 703–716. DOI: 10.1007/978-3-030-30490-4_56.
- [128] A. Geiger, D. Liu, S. Alnegheimish, A. Cuesta-Infante, and K. Veeramachaneni. “TadGAN: Time Series Anomaly Detection Using Generative Adversarial Networks”. In: *IEEE International Conference on Big Data (Big Data)*. 2020, pp. 33–43. DOI: 10.1109/BigData50022.2020.9378139.
- [129] A. Arora, S. Garg, and S. K. Peddoju. “Malware Detection Using Network Traffic Analysis in Android Based Mobile Devices”. In: *International Conference on Next Generation Mobile Apps, Services and Technologies (NGMAST)*. 2014, pp. 66–71. DOI: 10.1109/NGMAST.2014.57.
- [130] A. Zulkifli, I. R. A. Hamid, W. M. Shah, and Z. Abdullah. “Android Malware Detection Based on Network Traffic Using Decision Tree Algorithm”. In: *International Conference on Soft Computing and Data Mining (SCDM)*. 2018, pp. 485–494. DOI: 10.1007/978-3-319-72550-5_46.
- [131] Y. Aafer, W. Du, and H. Yin. “DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android”. In: *International Conference on Security and Privacy in Communication Systems (SecureComm)*. 2013, pp. 86–103. DOI: 10.1007/978-3-319-04283-1_6.
- [132] N. Peiravian and X. Zhu. “Machine Learning for Android Malware Detection Using Permission and API Calls”. In: *IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*. 2013, pp. 300–305. DOI: 10.1109/ICTAI.2013.53.
- [133] C. Li, K. Mills, D. Niu, R. Zhu, H. Zhang, and H. Kinawi. “Android Malware Detection Based on Factorization Machine”. In: *IEEE Access* 7 (2019), pp. 184008–184019. DOI: 10.1109/ACCESS.2019.2958927.
- [134] C. Chen, Y. Liu, B. Shen, and J.-J. Cheng. “Android Malware Detection Based on Static Behavior Feature Analysis”. In: *Journal of Computers* 29.6 (2018), pp. 243–253. DOI: 10.3966/199115992018122906024.
- [135] P. M. Comparetti, G. Salvaneschi, E. Kirda, C. Kolbitsch, C. Kruegel, and S. Zanero. “Identifying Dormant Functionality in Malware Programs”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2010, pp. 61–76. DOI: 10.1109/SP.2010.12.
- [136] G. Canfora, A. De Lorenzo, E. Medvet, F. Mercaldo, and C. A. Visaggio. “Effectiveness of Opcode Ngrams for Detection of Multi Family Android Malware”. In: *International Conference on Availability, Reliability and Security (ARES)*. 2015, pp. 333–340. DOI: 10.1109/ARES.2015.57.
- [137] N. McLaughlin, J. Martinez del Rincon, B. Kang, S. Yerima, P. Miller, S. Sezer, Y. Safaei, E. Trickel, Z. Zhao, A. Doupé, and G.-J. Ahn. “Deep Android Malware Detection”. In: *ACM Conference on Data and Application Security and Privacy (CODASPY)*. 2017, pp. 301–308. DOI: 10.1145/3029806.3029823.

- [138] A. Narayanan, C. Soh, L. Chen, Y. Liu, and L. Wang. “apk2vec: Semi-Supervised Multi-View Representation Learning for Profiling Android Applications”. In: *IEEE International Conference on Data Mining (ICDM)*. 2018, pp. 357–366. DOI: 10.1109/ICDM.2018.00051.
- [139] J. Yan, G. Yan, and D. Jin. “Classifying Malware Represented as Control Flow Graphs Using Deep Graph Convolutional Neural Network”. In: *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2019, pp. 52–63. DOI: 10.1109/DSN.2019.00020.
- [140] Y. Bengio, A. Courville, and P. Vincent. “Representation Learning: A Review and New Perspectives”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1798–1828. DOI: 10.1109/TPAMI.2013.50.
- [141] N. N. Tran, A. Said, W. Abbas, T. Derr, and X. D. Koutsoukos. *Quantifying the Generalization Gap: A New Benchmark for Out-of-Distribution Graph-Based Android Malware Classification*. 2025. DOI: 10.48550/arXiv.2508.06734. arXiv: 2508.06734.
- [142] J. Malik and R. Kaushal. “CREDROID: Android Malware Detection by Network Traffic Analysis”. In: *ACM Workshop on Privacy-Aware Mobile Computing (PAMCO)*. 2016, pp. 28–36. DOI: 10.1145/2940343.2940348.
- [143] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng. “Malware Traffic Classification Using Convolutional Neural Network for Representation Learning”. In: *International Conference on Information Networking (ICOIN)*. 2017, pp. 712–717. DOI: 10.1109/ICOIN.2017.7899588.
- [144] G. Marín, P. Casas, and G. Capdehourat. “DeepMAL – Deep Learning Models for Malware Traffic Detection and Classification”. In: *Data Science – Analytics and Applications*. Wiesbaden: Springer Vieweg, 2021, pp. 105–112. DOI: 10.1007/978-3-658-32182-6_16.
- [145] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani. “Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification”. In: *International Carnahan Conference on Security Technology (ICCST)*. 2018, pp. 1–7. DOI: 10.1109/ICCST.2018.8585560.
- [146] L. Taheri, A. F. A. Kadir, and A. H. Lashkari. “Extensible Android Malware Detection and Family Classification Using Network-Flows and API-Calls”. In: *International Carnahan Conference on Security Technology (ICCST)*. 2019, pp. 1–8. DOI: 10.1109/ICCST.2019.8888430.
- [147] S. Jain, A. Arora, and D. Kumar. “Hybrid Analysis in Android Malware Detection: A Systematic and Comparative Review”. In: *Computer Science Review* 62 (2026), p. 100980. DOI: 10.1016/J.COSREV.2026.100980.
- [148] J. Feng, L. Shen, Z. Chen, Y. Lei, and H. Li. “HGDetector: A Hybrid Android Malware Detection Method Using Network Traffic and Function Call Graph”. In: *Alexandria Engineering Journal* 114 (2025), pp. 30–45. DOI: 10.1016/j.aej.2024.11.068.
- [149] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. “Distributed Representations of Words and Phrases and Their Compositionality”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2013, pp. 3111–3119.
- [150] I. U. Haq and J. Caballero. “A Survey of Binary Code Similarity”. In: *ACM Computing Surveys* 54.3 (2021), pp. 1–38. DOI: 10.1145/3446371.

- [151] QoSient, LLC. *Argus: The Network Audit Record Generation and Utilization System*. openargus. 2026. URL: <https://openargus.org/> (visited on 08/14/2026).
- [152] S. Pourali, N. Samarasinghe, and M. Mannan. “Hidden in Plain Sight: Exploring Encrypted Channels in Android Apps”. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2022, pp. 2445–2458. DOI: 10.1145/3548606.3560665.
- [153] G. Dai, J. Ge, M. Cai, D. Xu, and W. Li. “SVM-Based Malware Detection for Android Applications”. In: *ACM Conference on Security & Privacy in Wireless and Mobile Networks (WiSec)*. 2015, pp. 1–2. DOI: 10.1145/2766498.2774991.
- [154] Android Open Source Project. *Network Security Configuration*. Android Developers. 2026. URL: <https://developer.android.com/privacy-and-security/security-config> (visited on 08/14/2026).
- [155] H. Li, B. Wu, W. Zhou, W. Yuan, C. Gao, X. You, and X. Luo. “An Efficient Adversarial Attack on FCG-Based Android Malware Detection Systems”. In: *IEEE Transactions on Information Forensics and Security* 20 (2025), pp. 9413–9426. DOI: 10.1109/TIFS.2025.3607270.
- [156] D. Zhang, X. Wu, E. He, X. Guo, X. Yang, R. Li, and H. Li. “Android Malware Detection Based on Hypergraph Neural Networks”. In: *Applied Sciences* 13.23 (2023), p. 12629. DOI: 10.3390/app132312629.
- [157] Y. He, Y. Liu, L. Wu, Z. Yang, K. Ren, and Z. Qin. “MsDroid: Identifying Malicious Snippets for Android Malware Detection”. In: *IEEE Transactions on Dependable and Secure Computing* 20 (2023), pp. 2025–2039. DOI: 10.1109/TDSC.2022.3168285.
- [158] J. Liu, J. Zeng, F. Pierazzi, Z. Yang, L. Cavallaro, and Z. Liang. “Unraveling the Key of Machine Learning-Based Android Malware Detection”. In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3809491.
- [159] X. Zhang, Y. Zhang, M. Zhong, D. Ding, Y. Cao, Y. Zhang, M. Zhang, and M. Yang. “Enhancing State-of-the-Art Classifiers with API Semantics to Detect Evolved Android Malware”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2020.
- [160] K. Zhao, H. Zhou, Y. Zhu, X. Zhan, K. Zhou, J. Li, L. Yu, W. Yuan, and X. Luo. “Structural Attack against Graph Based Android Malware Detection”. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2021, pp. 3218–3235. DOI: 10.1145/3460120.3485387.
- [161] S. MahdaviFar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani. “Dynamic Android Malware Category Classification Using Semi-Supervised Deep Learning”. In: *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC)*. 2020, pp. 515–522. DOI: 10.1109/DASC-PICom-CBDCoM-CyberSciTech49142.2020.00094.
- [162] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon. “AndroZoo: Collecting Millions of Android Apps for the Research Community”. In: *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2016, pp. 468–471. DOI: 10.1145/2901739.2903508.

- [163] SPARTA Consortium. *Validation and Evaluation Plan*. Project Deliverable D7.3. Editor: M. R. Norouzzian (TUM). Dissemination level: public. 19 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, June 2021. URL: <https://www.sparta.eu/assets/deliverables/SPARTA-D7.3-Validation-and-evaluation-plan-PU-M21-v1.1.pdf>.
- [164] M. R. Norouzzian. *Adversarial Machine Learning Benchmark Tool*. Source code, revision 2fac62b (8 November 2021). GitHub. 2021. URL: <https://github.com/reza-norouzzian/Adversarial-Benchmark-Tool/tree/2fac62b947ed> (visited on 09/10/2026).
- [165] M. R. Norouzzian. *SAFAIR AI Contest*. Chair for IT Security, Technical University of Munich. 2021. URL: <https://www.sec.in.tum.de/i20/projects/sparta-safair-ai-contest> (visited on 08/15/2026).
- [166] A. Kurakin, I. J. Goodfellow, and S. Bengio. “Adversarial Examples in the Physical World”. In: *International Conference on Learning Representations (ICLR) Workshop Track*. 2017. arXiv: 1607.02533 [cs.CV].
- [167] N. Carlini and D. Wagner. “Towards Evaluating the Robustness of Neural Networks”. In: *IEEE Symposium on Security and Privacy (S&P)*. 2017, pp. 39–57. DOI: 10.1109/SP.2017.49.
- [168] A. Kurakin, I. Goodfellow, S. Bengio, Y. Dong, F. Liao, M. Liang, T. Pang, J. Zhu, X. Hu, C. Xie, J. Wang, Z. Zhang, Z. Ren, A. Yuille, S. Huang, Y. Zhao, Y. Zhao, Z. Han, J. Long, Y. Berdibekov, T. Akiba, S. Tokui, and M. Abe. “Adversarial Attacks and Defences Competition”. In: *The NIPS ’17 Competition: Building Intelligent Systems*. The Springer Series on Challenges in Machine Learning. Springer, 2018, pp. 195–231. DOI: 10.1007/978-3-319-94042-7_11.
- [169] Z. Liu, P. Luo, X. Wang, and X. Tang. “Deep Learning Face Attributes in the Wild”. In: *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 3730–3738. DOI: 10.1109/ICCV.2015.425.
- [170] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller. *Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments*. Technical Report 07-49. University of Massachusetts, Amherst, Oct. 2007.
- [171] SPARTA Consortium. *Preliminary Description of AI Systems Security Mechanisms and Tools*. Project Deliverable D7.2. Editor: M. Pawlicki (ITTI). Dissemination level: public. 85 pages. SPARTA Project, EU Horizon 2020 grant agreement No. 830892, July 2020. URL: <https://www.sparta.eu/assets/deliverables/SPARTA-D7.2-Preliminary-description-of-AI-systems-security-mechanisms-and-tools-PU-M18.pdf>.
- [172] J. Rauber, W. Brendel, and M. Bethge. *Foolbox: A Python Toolbox to Benchmark the Robustness of Machine Learning Models*. arXiv:1707.04131. 2017.
- [173] J. Rauber. *Foolbox: Attack Base Classes, Version 3.3.1*. Source file `foolbox/attacks/base.py`; release of 23 February 2021. GitHub. 2021. URL: <https://github.com/bethgelab/foolbox/blob/v3.3.1/foolbox/attacks/base.py> (visited on 09/10/2026).
- [174] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard. “DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2574–2582. DOI: 10.1109/CVPR.2016.282.

Feature Importance of the NADICS Extraction Fields

Figure 3.4 in Section 3.4.1 names only the fifteen most important fields of the NADICS extraction schema. Table A.1 lists all 63 in rank order, with the importance of each field and the cumulative importance down to its rank. The cumulative column gives the thresholds of panel (c) of Figure 3.4 exactly: the first ten fields hold 54.47 per cent of the total, the first 24 hold 80.72 per cent and the first 36 hold 90.41 per cent. The remaining 27 fields share 9.59 per cent. Fields of equal importance keep the order of the source record, and the highest process field, `setpoint_change_count`, stands at rank 19.

Table A.1. Importance of the 63 fields of the NADICS extraction schema in the random forest of Section 3.4.2, in rank order. Importance is the normalised share of the total, and the cumulative column sums the importances down to that rank. The letter of the identifier gives the group of Table A.2, and the bar colours follow Figure 3.4.

Rank	ID	Field	Group	Importance (%)	Cumulative (%)	
1	C2	s7_function	Protocol	11.93	11.93	
2	D6	role_violation	Relational	7.95	19.88	
3	B6	period_deviation	Timing	6.76	26.64	
4	B11	unanswered_req_ratio	Timing	4.77	31.41	
5	C12	err_rate	Protocol	4.57	35.98	
6	D1	is_known_edge	Relational	4.37	40.36	
7	C5	write_read_ratio	Protocol	3.78	44.14	
8	A9	rate	Flow	3.58	47.71	
9	D3	n_distinct_dst_1min	Relational	3.48	51.19	
10	C9	addr_entropy	Protocol	3.28	54.47	
11	C11	s7_error_code	Protocol	2.88	57.36	
12	B3	iat_cv	Timing	2.58	59.94	
13	B7	cycle_count_gap	Timing	2.49	62.43	
14	D4	n_distinct_ports_1min	Relational	2.39	64.81	
15	B10	resp_latency_max	Timing	2.19	67.00	
16	C4	n_write_items	Protocol	2.09	69.09	
17	B8	cycle_jitter	Timing	1.79	70.87	
18	B9	resp_latency_mean	Timing	1.79	72.66	
19	E4	setpoint_change_count	Process	1.69	74.35	
20	C10	n_distinct_db	Protocol	1.59	75.94	
21	C15	n_tsap_tried	Protocol	1.29	77.24	
22	A3	service	Flow	1.19	78.43	
23	A4	state	Flow	1.14	79.57	
24	C3	n_read_items	Protocol	1.14	80.72	
25	A1	duration	Flow	1.09	81.81	

continued on the next page

Table A.1. (continued)

Rank	ID	Field	Group	Importance (%)	Cumulative (%)	
26	D5	role_pair	Relational	1.09	82.90	■
27	C6	s7_area	Protocol	0.89	83.80	■
28	D7	dcp_identify_count	Relational	0.84	84.64	■
29	A14	src_loss	Flow	0.80	85.44	■
30	B5	period_power	Timing	0.80	86.23	■
31	A5	src_pkt	Flow	0.75	86.98	■
32	B2	iat_std	Timing	0.75	87.72	■
33	A10	src_load	Flow	0.70	88.42	■
34	C1	s7_rosctr	Protocol	0.70	89.12	■
35	A6	dst_pkt	Flow	0.65	89.76	■
36	D2	edge_age	Relational	0.65	90.41	■
37	A7	src_bytes	Flow	0.60	91.00	■
38	B13	is_new_session	Timing	0.60	91.60	■
39	C18	opcua_session_rate	Protocol	0.60	92.20	■
40	A15	dst_loss	Flow	0.55	92.74	■
41	B4	dominant_period	Timing	0.55	93.29	■
42	A8	dst_bytes	Flow	0.50	93.79	■
43	D9	new_mac_seen	Relational	0.50	94.28	■
44	A12	src_mean_sz	Flow	0.45	94.73	■
45	B1	iat_mean	Timing	0.45	95.18	■
46	C7	s7_db_number	Protocol	0.45	95.63	■
47	E2	value_delta	Process	0.45	96.07	■
48	A11	dst_load	Flow	0.40	96.47	■
49	E5	value_freeze_len	Process	0.40	96.87	■
50	B12	session_age	Timing	0.35	97.22	■
51	E3	value_out_of_range	Process	0.35	97.56	■
52	A13	dst_mean_sz	Flow	0.30	97.86	■
53	C8	s7_start_addr	Protocol	0.30	98.16	■
54	C14	cotp_tsap_slot	Protocol	0.30	98.46	■
55	A2	protocol	Flow	0.25	98.71	■
56	C13	pdu_ref_gap	Protocol	0.25	98.96	■
57	C17	opcua_sec_policy_none	Protocol	0.25	99.20	■
58	A18	tcp_rtt	Flow	0.20	99.40	■
59	C16	opcua_msg_type	Protocol	0.15	99.55	■
60	D8	arp_conflict_count	Relational	0.15	99.70	■
61	A16	syn_ack	Flow	0.12	99.82	■
62	E1	tag_value	Process	0.10	99.92	■
63	A17	ackdat	Flow	0.08	100.00	■

Table A.2 gives the group totals behind panel (b) of Figure 3.4, together with the importance per field that Section 3.4.1 discusses and the rank of the highest field in each group.

Table A.2. Importance by field group. Both shares are percentages of the 63-field total. Importance per field is the group share divided by the number of fields in the group, and the last column gives the rank of the group's highest field in Table A.1.

Group	Fields	Importance share (%)	Field share (%)	Importance per field (%)	Highest rank
A ■ Flow base	18	13.32	28.57	0.74	8
B ■ Timing / periodicity	13	25.84	20.63	1.99	3
C ■ Protocol semantics	18	36.43	28.57	2.02	1
D ■ Relational	9	21.42	14.29	2.38	2
E ■ Process derived	5	2.98	7.94	0.60	19
All groups	63	100.00	100.00	1.59	